

Zvýšenie bezpečnosti vybraných šifrovacích algoritmov v prístupových systémoch

¹*Oleksandr MYKHAILYSHYN*

¹Katedra počítačov a informatiky, Fakulta elektrotechniky a informatiky, Technická univerzita v Košiciach, Slovenská republika

¹oleksandr.mykhailyshyn@student.tuke.sk

Abstrakt – Práca sa zaoberá bezpečnostnými mechanizmami, ktoré sú relevantné pre moderné systémy kontroly prístupu využívajúce bezkontaktné identifikátory RFID/NFC. Teoretická časť nevystupuje iba ako všeobecný prehľad, ale priamo nadväzuje na vytvorený prototyp: čítačku založenú na ESP32 a RC522, serverovú aplikáciu v jazyku C++ a lokálne úložisko SQLite s auditnou stopou. Jadrom návrhu je otázka, ako zvýšiť dôveryhodnosť celého toku udalostí od načítania UID až po rozhodnutie o prístupe a neskoršie overenie auditu.

V texte preto rozoberám autentifikáciu hardvérových požiadaviek pomocou HMAC-SHA-256, ochranu proti opakovaniu správ cez monotónne `hw_seq`, vnútorný frame chránený mechanizmom AEAD (ChaCha20-Poly1305 a XChaCha20-Poly1305) a auditný log navrhnutý tak, aby bolo možné odhaliť dodatočnú manipuláciu. V experimentálnej časti sa porovnáva šesť konfiguračných profilov, ktoré kombinujú oba šifrovacie algoritmy s tromi stratégiami generovania nonce (náhodný, deterministický HMAC a deterministický s runtime anomálnou detekciou). Súčasne otvorene pomenúvam aj hranice prototypu: UID karty je v tomto riešení identifikátor, nie kryptografický dôkaz identity, databáza je uložená lokálne a prototyp nevyžaduje povinné TLS. Tieto obmedzenia nie sú obchádzané, ale slúžia ako súčasť realistického hodnotenia toho, čo prototyp chráni dobre a kde zostáva priestor na ďalšie rozšírenie.

Kľúčové slová – kontrola prístupu, RFID, NFC, kryptografia, AEAD, ChaCha20-Poly1305, XChaCha20-Poly1305, anomálna detekcia, HMAC, HKDF, anti-replay, audit log.

Enhancing Security of Selected Encryption Algorithms in Access Control Systems

Oleksandr Mykhailyshyn

Technical University of Košice, Faculty of Electrical Engineering and Informatics, Slovakia

Abstract – The thesis addresses security mechanisms relevant to modern RFID/NFC access control systems. The theoretical part does not serve merely as a general overview but directly underpins the implemented prototype: an ESP32 and RC522-based reader, a C++ server application, and a local SQLite storage with an audit trail. The core design question is how to increase the trustworthiness of the entire event flow from UID reading to the access decision and subsequent audit verification.

The text therefore examines the authentication of hardware requests using HMAC-SHA-256, protection against message replay via monotonic `hw_seq`, an internal frame protected by AEAD (ChaCha20-Poly1305 and XChaCha20-Poly1305), and an audit log designed to detect post-hoc manipulation. The experimental section compares six configuration profiles that combine both encryption algorithms with three nonce generation strategies (random, deterministic HMAC, and deterministic with runtime anomaly detection). The thesis also explicitly identifies the prototype's limitations: the card UID is an identifier in this solution, not a cryptographic proof of identity; the database is stored locally; and TLS is not mandatory. These constraints are not circumvented but serve as part of a realistic assessment of what the prototype protects well and where room for further improvement remains.

Keywords – access control, RFID, NFC, cryptography, AEAD, ChaCha20-Poly1305, XChaCha20-Poly1305, anomaly detection, HMAC, HKDF, anti-replay, audit log.

ČESTNÉ VYHLÁSENIE

Vyhlasujem, že som celú diplomovú prácu vypracoval samostatne a uviedol som všetku použitú literatúru.

.....
Oleksandr Mykhailyshyn

Košice, April 9, 2026

POĎAKOVANIE

Ďakujem vedúcemu diplomovej práce za odborné vedenie, cenné rady a trpezlivú podporu počas celého procesu vypracovania tejto práce.

CONTENTS

I	Úvod	11
II	Motivácia, ciele a prínos práce	11
II-A	Motivácia	11
II-B	Ciele práce	12
II-C	Prínos	12
II-D	Rozsah riešenia a vedomé obmedzenia	12
II-E	Metodika hodnotenia a analytické ciele	12
III	Všeobecné bezpečnostné pojmy a terminológia	13
III-A	CIA triáda a rozšírené bezpečnostné ciele	13
III-B	AAA model: Authentication, Authorization, Accounting	14
III-C	Riziko, hrozba, zraniteľnosť	14
III-D	Modelovanie hrozieb	14
III-E	Autentifikačné faktory a pojem identity	15
III-F	Zodpovednosť, nepopierateľnosť a forenzná pripravenosť	15
III-G	Bezpečnostné požiadavky ako merateľné tvrdenia	15
IV	Systémy kontroly prístupu a prístupové modely	15
IV-A	Fyzická vs. logická kontrola prístupu	15
IV-B	Modely kontroly prístupu	16
IV-C	Bezpečnostné požiadavky na API a server	16
IV-D	Zóny, kontext a väzby medzi komponentmi	16
IV-E	Princíp najmenej oprávnení a oddelenie právomocí	16
V	Bezpečnostné hrozby a špecifiká RFID/NFC technológií	16
V-A	Typické útoky na RFID/NFC	17
V-B	Mitigácie a odporúčané protiopatrenia	17
V-C	Príklad známych slabín: MIFARE Classic	17
V-D	Bezpečnostné dôsledky pre návrh prototypu	18
V-E	Hrozby na komunikačnej vrstve a v sieťovej infraštruktúre	18
V-F	Hrozby na serverovej strane a v API	18
V-G	Útoky po incidente: manipulácia auditnej stopy	18
VI	Kryptografické základy relevantné pre prístupové systémy	18
VI-A	Hashovacie funkcie a integrita	18
VI-B	MAC a HMAC	19
VI-C	Výber parametrov a bezpečnostná úroveň	19
VI-D	Bezpečná práca s pamäťou a citlivými údajmi	19
VI-E	Symetrické šifrovanie a AEAD	19
VI-E1	ChaCha20 a Poly1305	19
VI-E2	ChaCha20-Poly1305, XChaCha20-Poly1305 a nonce manažment	20
VI-E3	Associated Data (AAD)	20
VI-E4	Porovnanie s AES-GCM	20
VI-F	Náhodnosť, nonce a praktické generovanie	21
VI-G	Domain separation a kontext v derivácii kľúčov	21
VI-H	Kryptografia vs. bezpečnosť implementácie	21
VI-I	Bezpečnostné modely AEAD a nonce-misuse resistance	21
VII	Zásady návrhu bezpečných protokolov	22
VII-A	Fail-closed správanie	22
VII-B	Jednoznačná serializácia a kanonizácia	22
VII-C	Verzovanie formátu a algoritmov	22
VII-D	Bezpečné spracovanie chýb a jednotné odpovede	22
VII-E	Ochrana proti DoS a vyčerpaniu zdrojov	22
VII-F	Bezpečnosť knižníc a závislostí	22
VII-G	Mapovanie bezpečnostných cieľov na mechanizmy	22

VIII	Správa kryptografických kľúčov	23
VIII-A	Oddelenie kľúčov podľa účelu	23
VIII-B	Verzovanie a rotácia kľúčov	23
VIII-C	Ukladanie kľúčov a bezpečnosť servera	23
VIII-D	Provisioning a registrácia zariadení	23
VIII-E	Kompromitácia kľúča a postup obnovy	23
VIII-F	Evidencia verzií a migračné stratégie	23
IX	Návrh bezpečného komunikačného protokolu	24
IX-A	End-to-end tok udalostí	24
IX-B	Formát správ, validácia vstupu a obrana pred parser útokmi	24
IX-C	Hranice dôvery a spracovanie metadát	24
IX-D	Škálovanie anti-replay pri viacerých zariadeniach	24
IX-E	Časové okná, oneskorenie a „freshness“	25
IX-F	Autentifikácia hardvéru cez HMAC	25
IX-G	Anti-replay: sekvencie a <i>ReplayWindow</i>	25
IX-G1	Prečo je replay útok nebezpečný	25
IX-G2	Monotónne <i>hw_seq</i>	25
IX-G3	Sliding window na serveri	25
IX-H	Freshness a časová pečiatka	25
IX-I	Poznámka k dôvernosti komunikácie	25
X	Bezpečnosť vstavaného zariadenia a spracovanie tajomstiev	25
X-A	Ukladanie tajomstiev na zariadení	26
X-B	Monotónne počítadlá a trvácnosť stavu	26
X-C	Bezpečnosť firmware a aktualizácie	26
X-D	Fyzické útoky a praktický hardening	26
X-E	Bezpečnosť Wi-Fi konfigurácie a lokálnej siete	26
X-F	Bezpečnosť logovania na zariadení	27
XI	Bezpečnosť transportnej vrstvy a použitie TLS	27
XI-A	Prečo nestačí iba TLS	27
XI-B	mTLS vs. symetrické tajomstvá	27
XI-C	Validácia certifikátu a <i>certificate pinning</i>	27
XI-D	Praktické nasadenie TLS v IoT	27
XI-E	Kombinácia vrstiev: kedy dáva zmysel HMAC aj pri TLS	28
XII	Bezpečnosť úložiska, exportov a životný cyklus dát	28
XII-A	Minimizácia dát a pseudonymizácia	28
XII-B	Export/import a integritné kontroly	28
XII-C	Prístupové práva k databáze a zálohovanie	28
XII-D	Retencia a ochrana súkromia v audite	28
XIII	Ochrana súkromia identifikátorov a bezpečnosť úložiska	28
XIII-A	Prečo neukladať surový UID	28
XIII-B	HMAC(card_id) s <i>pepper</i>	29
XIII-C	Pepper vs. salt a odolnosť voči slovníkovým útokom	29
XIII-D	Minimalizácia údajov v audite a súkromnostné riziká	29
XIII-E	Poznámka k právnym aspektom (GDPR)	29
XIII-F	Integrita databázy a hranice SQLite	29
XIV	Tamper-evident audit log	29
XIV-A	Základné typy útokov na logy	29
XIV-B	Hash chain s HMAC	30
XIV-C	Audit anchor a ochrana proti truncation	30
XIV-D	Forward integrity a správa auditného kľúča	30
XIV-E	Uloženie anchorov a dôveryhodný referenčný bod	30
XIV-F	Limity a alternatívy: Merkle stromy a verifikovateľné výňatky	30
XIV-G	Verifikácia a nástroj <i>audit_verify</i>	30
XV	Odporúčania a štandardy relevantné pre návrh	30
XV-A	Kryptografické a protokolárne štandardy	31
XV-B	Bezpečnostné odporúčania pre systémy	31
XV-C	Súlad prototypu s odporúčaniami	31

XVI	Riadenie rizika a návrh protiopatrení	31
XVI-A	Pravdepodobnosť a dopad	31
XVI-B	Vrstvené zabezpečenie (defense-in-depth)	31
XVI-C	Bezpečnosť ako proces	31
XVI-D	Predpoklady a zvyškové riziko	32
XVI-E	Komunikačné kompromisy a použiteľnosť	32
XVII	Bezpečnosť administrátorského rozhrania a prevádzkové aspekty	32
XVII-A	Typické webové hrozby: XSS, CSRF a spracovanie vstupu	32
XVII-B	Autentifikácia administrátora a správa relácií	32
XVII-C	Autorizácia operácií a audit administrácie	32
XVII-D	Bezpečnostná konfigurácia a minimalizácia „debug“ rozhraní	32
XVII-E	Monitoring a detekcia anomálií	32
XVIII	Súvisiace prístupy a typické architektúry systémov kontroly prístupu	32
XVIII-A	Centralizované vs. distribuované rozhodovanie	33
XVIII-B	Online vs. offline režim a bezpečnostné kompromisy	33
XVIII-C	Integrácia s identitnými systémami	33
XVIII-D	Audit a forenzná pripravenosť v bežných riešeniach	33
XVIII-E	Komunikačné protokoly medzi čítačkou a kontrolérom	33
XVIII-F	Topológie: inteligentná čítačka vs. kontrolér pri dverách	33
XVIII-G	Cloud vs. on-premise a dôvera v infraštruktúru	34
XVIII-H	Postavenie prototypu v tomto kontexte	34
XIX	Kontrolný zoznam bezpečného návrhu prístupového systému	34
XIX-A	Autentifikácia a autorizácia	34
XIX-B	Validácia vstupu a robustnosť	34
XIX-C	Kryptografia a manažment kľúčov	34
XIX-D	Anti-replay a časové údaje	34
XIX-E	Úložisko, súkromie a audit	34
XIX-F	Prevádzka a monitoring	34
XX	Zhrnutie teoretických východísk	35
XX-A	Kľúčové bezpečnostné princípy a ich realizácia	35
XX-B	Identifikované medzery a odporúčania pre produkčné nasadenie	35
XX-C	Prepojenie teórie s experimentálnou analýzou	35
XXI	Praktická implementácia navrhnutého systému	35
XXI-A	Architektúra implementácie	35
XXI-B	Modulárne členenie projektu	36
XXI-C	Externé závislosti	37
XXI-D	Zdôvodnenie použitia knižnice libsodium	37
XXI-D1	Súlad so štandardmi	38
XXI-D2	Bezpečnostné audity a nasadenie	38
XXI-D3	Ochrana proti bočným kanálom	38
XXI-D4	Kompilácia zo zdrojového kódu	38
XXI-D5	Záver k validite analýzy	39
XXI-E	Tok spracovania prístupovej udalosti	39
XXI-F	Poznámka k implementačným rozhodnutiam a k ďalšiemu spracovaniu textu	39
XXI-G	Autentifikácia hardvérovej požiadavky	40
XXI-H	AEAD šifrovanie interného frame	40
XXI-H1	Voľba algoritmu	40
XXI-H2	Konštrukcia nonce	40
XXI-H3	AAD väzba hlavičky	41
XXI-I	Derivácia kľúčov a domain separation	41
XXI-J	Anti-replay mechanizmus	42
XXI-K	Detektor protokolových anomálií	43
XXI-L	Ochrana identifikátorov kariet	43
XXI-M	Tamper-evident auditná stopa	44
XXI-M1	Konštrukcia HMAC hash chain	44
XXI-M2	Anchor a ochrana proti truncation	44
XXI-N	Rozhodovacia logika a model RBAC	44
XXI-O	Konfigurácia systému a bezpečnostné profily	44
XXI-P	Webové administrátorské rozhranie	45
XXI-Q	Prepojenie mechanizmov: prečo vrstvy nestoja samostatne	45

XXII Verifikácia kvality kódu pred experimentmi	46
XXII-A Statická analýza: clang-tidy	46
XXII-B Dynamická analýza: sanitizers	46
XXII-C Testovacia sada a pokrytie	47
XXII-D Výsledky dynamickej analýzy	47
XXII-E Záver verifikácie	48
XXIII Experimentálna analýza bezpečnostných profilov	48
XXIII-A Matica konfiguračných profilov	48
XXIII-B Architektúra experimentálneho harnessu	49
XXIII-B1 Hlavné komponenty	49
XXIII-B2 Fázy experimentu	49
XXIII-B3 Reprodukovateľnosť	49
XXIII-B4 Vzťah harnessu k produkčnému systému	50
XXIII-C Popis útokových scenárov	50
XXIII-C1 S1: Replay útok	50
XXIII-C2 S2: Manipulácia poľa v hlavičke (AAD tampering)	50
XXIII-C3 S3a/S3b: Zneužitie nonce (RNG fault)	50
XXIII-C4 S4: Rollback sekvenčného čísla	50
XXIII-C5 S5: Tag probe (forgery resistance)	51
XXIII-C6 S6: Throughput benchmark	51
XXIII-D Metodika zberu a spracovania dát	51
XXIII-D1 Zber dát	51
XXIII-D2 Odvodzované metriky	51
XXIII-D3 Štatistická robustnosť	51
XXIII-E Výsledky: Úspešnosť útokov	51
XXIII-E1 Celkový prehľad	51
XXIII-E2 Závislosť od počtu útočných frame	51
XXIII-E3 Rýchlosť karantény	53
XXIII-F Výsledky: Baseline korektnosť	53
XXIII-G Výsledky: Latencia a priepustnosť	53
XXIII-G1 Throughput benchmark (S6)	54
XXIII-G2 Overhead detektora na baseline frame	54
XXIII-G3 Distribúcia latencie podľa scenárov	54
XXIII-H Diskusia a interpretácia výsledkov	54
XXIII-H1 Tézis 1: Deterministický nonce bez verifikácie nie je dostatočný	54
XXIII-H2 Tézis 2: ProtocolAnomalyDetector materiálne zvyšuje bezpečnosť	55
XXIII-H3 Tézis 3: Replay window a anomálna detekcia sú komplementárne	55
XXIII-H4 Tézis 4: AAD binding poskytuje ochranu nezávisle od režimu nonce	57
XXIII-H5 Tézis 5: Defense-in-depth cez karanténu	57
XXIII-H6 Tézis 6: Overhead detektora je prijateľný	57
XXIII-I Porovnanie s minimalistickými implementáciami	57
XXIII-J Zhrnutie experimentov	58
XXIV Záver	58
XXIV-A Súhrn realizovaných prác	58
XXIV-B Hlavné výsledky a závery	60
XXIV-C Splnenie cieľov práce	60
XXIV-D Otvorené otázky a obmedzenia	61
XXIV-E Možnosti rozšírenia	61
XXIV-F Záverečné zhodnotenie	61
References	62

LIST OF FIGURES

1	Zjednodušená end-to-end schéma prototypu a hraníc dôvery medzi vrstvami.	11
2	ChaCha20-Poly1305 AEAD: tok šifrovania (vľavo) a dešifrovania (vpravo). Poly1305 autentifikuje AAD aj CipherText spoločne. Pri dešifrovaní sa overenie tagu vykoná <i>pred</i> dešifrovaním.	20
3	Závislosťová hierarchia modulov projektu <i>access-security</i> ($A \rightarrow B$: modul A závisí na B).	37
4	Derivácia kľúčov z master secret cez HKDF. Každá vetva má vlastný info reťazec (domain separation).	42
5	Úspešnosť útokov podľa scenára a profilu (priemer cez 5 behov). Zelené bunky: útok zablokovaný (0%). Červené bunky: útok úspešný.	52
6	Závislosť úspešnosti útoku od počtu útočných frame N . Pásma: rozsah cez 5 behov (nulová šírka = konzistentný deterministický výsledok).	53
7	Počet frame od začiatku útoku do prvej karantény (len R2 profily, priemer $\pm$ std). .	53
8	Priepustnosť pri $N = 50,000$ frame (priemer $\pm$ std cez 5 behov) a percentily latencie.	54
9	Porovnanie latencie R1 (bez detektora) a R2 (s detektorom) na validných baseline frame.	55
10	Distribúcia latencie podľa scenára a profilu (baseline vs. attack fáza, $N = 500$). . .	56
11	Súhrnná tabuľka výsledkov experimentov: scenár $\times$ profil.	59

ZOZNAM SKRATIEK

AES	Authenticated Encryption with Associated Data
ABAC	Attribute-Based Access Control
AAA	Authentication, Authorization, Accounting
ARX	Add-Rotate-XOR
CIA	Confidentiality, Integrity, Availability
DAC	Discretionary Access Control
DoS	Denial of Service
ESP32	mikrokontrolér Espressif Systems ESP32
FEI	Fakulta elektrotechniky a informatiky
FIPS	Federal Information Processing Standard
GDPR	General Data Protection Regulation
HKDF	HMAC-based Key Derivation Function
HMAC	Hash-based Message Authentication Code
HSM	Hardware Security Module
IoT	Internet of Things
KPI	Katedra počítačov a informatiky
LSan	LeakSanitizer
MAC	Message Authentication Code
MITM	Man-in-the-Middle (útok)
mTLS	mutual Transport Layer Security
NFC	Near Field Communication
NIST	National Institute of Standards and Technology
NVS	Non-Volatile Storage
OSDP	Open Supervised Device Protocol
OTA	Over-The-Air (bezdrôtová aktualizácia)
OWASP	Open Web Application Security Project
PKI	Public Key Infrastructure
PSK	Pre-Shared Key
RBAC	Role-Based Access Control
RC522	čítačka RFID kariet (NXP Semiconductors)
RFC	Request for Comments
RFID	Radio Frequency Identification
SIMD	Single Instruction, Multiple Data
SQLite	odľahčená relačná databáza (serverless)
STRIDE	Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege
TLS	Transport Layer Security
TUKE	Technická univerzita v Košiciach
UBSan	UndefinedBehaviorSanitizer
UID	Unique Identifier

I. ÚVOD

Systémy kontroly prístupu (*Access Control Systems*) sú jedným z tých prvkov infraštruktúry, pri ktorých sa fyzická a informačná bezpečnosť stretávajú priamo v jednom bode. Na prvý pohľad ide iba o rozhodnutie, či sa dvere otvoria alebo nie. V skutočnosti sa za tým skrýva identifikácia používateľa, vyhodnotenie pravidiel, práca s tajomstvami, komunikácia po sieti a napokon aj audit, ktorý má po incidente vysvetliť, čo sa stalo.

Práve pri jednoduchších riešeniach býva rozdiel medzi funkčným a bezpečným systémom najvýraznejší. V praxi nie je problém zostaviť čítačku, ktorá po načítaní UID pošle hodnotu na server a čaká na odpoveď. Takýto návrh je však zraniteľný takmer okamžite: stačí zachytiť požiadavku, zopakovať ju v inom čase, podvrhnúť vlastný identifikátor alebo upraviť dáta v úložisku. Ak systém ukladá surové UID a pritom nemá dôveryhodný audit, po incidente zostáva iba veľmi slabá stopa.

Táto diplomová práca vychádza práve z takéhoto praktického pozorovania. Cieľom nebolo navrhnúť teoreticky maximálne silný systém bez ohľadu na cenu a zložitosť, ale postaviť prototyp, na ktorom sa dá ukázať, ktoré bezpečnostné mechanizmy majú v prístupových systémoch skutočný prínos. Výsledkom je kombinácia hardvéru (ESP32 + RC522), serverovej aplikácie v jazyku C++ a webového rozhrania, v ktorom sa spravujú čítačky, dvere, roly, karty aj auditné záznamy.

Bezpečnosť riešenia stojí na niekoľkých navzájom previazaných vrstvách:

- autentifikácia hardvérových požiadaviek pomocou HMAC-SHA-256[1],
- šifrovanie a integrita interného payloadu pomocou AEAD (ChaCha20-Poly1305[2] a XChaCha20-Poly1305[3], [4]),
- ochrana proti replay útokom cez monotónne sekvencie a *ReplayWindow*,
- ochrana súkromia identifikátorov kariet pomocou HMAC hashovania s *pepper*,
- auditná stopa navrhnutá tak, aby bolo možné odhaliť dodatočnú manipuláciu pomocou HMAC hash chain a anchor bodov[5].

Zjednodušená schéma celého toku udalosti je znázornená na obrázku 1.

Dôležité je, že jednotlivé vrstvy v prototypu neplnia tú istú úlohu. Hardvérová čítačka neposiela na server priamo AEAD šifrovaný payload; odosiela HTTP požiadavku autentifikovanú pomocou HMAC a monotónneho *hw_seq*. Až server po overení tejto požiadavky vytvára interný kryptograficky chránený frame, ktorý vstupuje do rozhodovacej logiky. Táto hranica dôvery je v texte pomenovaná zámerne, pretože ovplyvňuje aj interpretáciu výsledkov: niečo iné znamená ochrana proti podvrhnutiu HTTP požiadavky a niečo iné ochrana interného frame alebo auditnej stopy.

Bezpečnostná analýza pokrýva oblasti s priamym dopadom na implementovaný systém: model hrozieb, kryptografické primitíva, autentifikáciu zariadenia, anti-replay mechanizmy, správu kľúčov a auditovateľnosť. Overenie prínosu navrhnutých mechanizmov vychádza z experimentálneho porovnania šiestich konfiguračných profilov, ktoré variujú šifrovací algoritmus, stratégiu generovania nonce a prítomnosť runtime anomálnej detekcie.

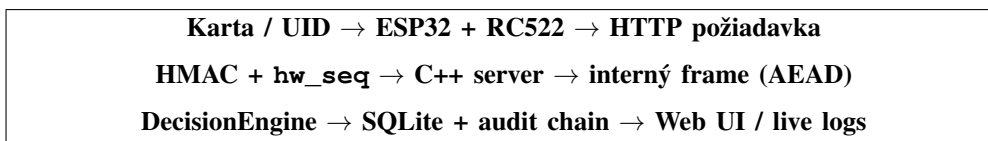


Fig. 1 Zjednodušená end-to-end schéma prototypu a hraníc dôvery medzi vrstvami.

II. MOTIVÁCIA, CIELE A PRÍNOS PRÁCE

A. Motivácia

Motivácia tejto práce nie je iba teoretická. Počas návrhu a skladania prototypu sa opakovane ukazovalo, že najjednoduchšia funkčná verzia systému by sa dala postaviť veľmi rýchlo: prečítať UID, poslať ho na server a podľa odpovede otvoriť dvere. Takýto návrh však rieši iba funkcionálnosť, nie dôveryhodnosť. Hneď ako sa do úvahy vezme bežná lokálna sieť, databáza s kartami a potreba spätne vysvetliť incident, začnú byť zaujímavé celkom iné otázky.

Reálne prostredie totiž zahŕňa:

- nepredvídateľnú sieť (Wi-Fi, zdieľané siete, kompromitované zariadenia),
- úložisko v podobe databázy, ktorú je možné kopírovať a analyzovať,
- insider scenáre (administrátor, ktorý môže zneužiť oprávnenia alebo zmeniť logy),

- vstavané zariadenia s obmedzenými zdrojmi, ktoré často nemajú plnú podporu pre komplexné bezpečnostné protokoly.

Ak systém nepoužíva kryptograficky zabezpečené logovanie, útočník môže po incidente vymazať stopy. Ak systém neobsahuje anti-replay, je možné prístup opakovať bez karty. Ak systém ukladá surové UID, únik databázy priamo ohrozuje používateľov.

B. Ciele práce

Hlavné ciele práce sú:

- 1) Navrhnuť bezpečnostné požiadavky a model hrozieb pre prototyp prístupového systému.
- 2) Zaviesť kryptografické mechanizmy na úrovni komunikácie a spracovania:
 - AEAD (ChaCha20-Poly1305 a XChaCha20-Poly1305) pre dôvernosť a integritu dát[2], [3],
 - HMAC-SHA-256 pre autentifikáciu požiadaviek hardvéru[1],
 - HKDF pre deriváciu kľúčov a oddelenie účelov[6].
- 3) Implementovať anti-replay mechanizmus založený na monotónnom `hw_seq` (perzistentne uloženom na ESP32) a *ReplayWindow* na serveri.
- 4) Navrhnuť a implementovať tamper-evident audit log s možnosťou verifikácie a detekcie truncation.
- 5) Stanoviť analytické ciele hodnotenia: porovnať šesť konfiguračných profilov (dva AEAD algoritmy $\times$ tri nonce stratégie) v šiestich útokových a výkonnostných scenároch, zbierať metriky (úspešnosť útoku, dôvody zamietnutia, latenciu a priepustnosť) a formulovať závery o príspevku jednotlivých bezpečnostných mechanizmov.

C. Prínos

Prínos práce je možné čítať v dvoch rovinách. Prvá je technická: vznikol funkčný prototyp, na ktorom sa dá ukázať, že aj pomerne malý prístupový systém môže mať jasne oddelené bezpečnostné vrstvy. Druhá rovina je metodická: práca nevychádza len z tvrdenia, že zvolený mechanizmus je „bezpečnejší“, ale pripravuje priestor na jeho meranie a porovnanie s jednoduchšou verziou riešenia.

Konkrétne ide o:

- demonštrácia realistického návrhu bezpečnej komunikácie pre IoT/embedded zariadenia,
- dôsledné oddelenie autentifikácie hardvéru od administrátorských oprávnení (HW nepoužíva admin token),
- ochrana súkromia používateľov pomocou hashovania identifikátorov kariet,
- auditovateľnosť a detekcia manipulácie logov pomocou kryptografických reťazcov,
- modulárna architektúra C++ projektu (kryptografia, protokol, rozhodovanie, úložisko, UI).

D. Rozsah riešenia a vedomé obmedzenia

V tejto práci nepovažujem za poctivé tváriť sa, že prototyp rieši všetky problémy produkčného prístupového systému. Niektoré voľby boli urobené zámerne, pretože cieľom bolo overiť konkrétne bezpečnostné mechanizmy a nie simulovať kompletne enterprise nasadenie.

Použitá RFID karta pracuje v jednoduchom režime, takže UID vystupuje ako identifikátor, nie ako kryptografický dôkaz identity. Databázová vrstva je postavená na SQLite, čo je pre prototyp a experimenty praktické, ale neznamena to vysokú dostupnosť, centralizovanú správu tajomstiev ani pohodlné škálovanie. Komunikácia hardvéru so serverom je v základnom režime chránená autentifikáciou a integritou na aplikačnej vrstve (HMAC + `hw_seq`), zatiaľ čo dôvernosť transportu cez TLS zostáva rozšírením pre ďalšie nasadenie.

Tieto obmedzenia teda nie sú slabinou textu, ktorú by bolo potrebné zakryť. Naopak, vytvárajú presný rámec pre interpretáciu výsledkov. Ak sa v analytickej časti ukáže zlepšenie odolnosti voči podvrhnutiu, replay alebo manipulácii s auditom, bude zrejmé, že ide o prínos navrhnutých mechanizmov v rámci definovaného prototypu, nie o všeobecné tvrdenie o všetkých prístupových systémoch.

E. Metodika hodnotenia a analytické ciele

Pri podobných prototypoch sa často končí pri ukážke, že systém „funguje“. To však pre tému zameranú na bezpečnosť nestačí. Preto je hodnotenie od začiatku postavené tak, aby bolo možné overiť, či navrhnuté mechanizmy skutočne menia správanie systému pri útoku alebo chybe.

Table 1
Vedomé obmedzenia prototypu a ich význam pre interpretáciu výsledkov

Oblasť	Aktuálne riešenie	Dôsledok pre prácu
Identita karty	UID slúži ako identifikátor, nie ako kryptografický dôkaz	Práca sa zameriava najmä na bezpečnosť komunikácie, serverovej logiky a auditu, nie na bezpečné smart karty.
Úložisko	Lokálna SQLite databáza	Riešenie je vhodné pre prototyp a experimenty; vysoká dostupnosť a centralizovaná správa tajomstiev nie sú predmetom práce.
Transport	Povinné TLS nie je v základnom režime nasadené	Dôvernoscť prenosu UID nie je hlavnou demonštrovanou vlastnosťou; dôraz je na autentifikáciu zariadenia, anti-replay a auditovateľnosť.
Správa tajomstiev	Tajomstvá sú spravované pragmaticky pre potreby prototypu	Výsledky treba interpretovať ako experimentálny návrh, nie ako plnohodnotné produkčné nasadenie.

V praktickej časti sa preto nebude sledovať iba to, či server vráti `allow` alebo `deny`. Dôležité bude aj to, prečo k rozhodnutiu došlo, aká je úspešnosť jednotlivých útokov, akú latenciu zavádzajú ochranné mechanizmy a ako sa riešenie správa pri opakovaných alebo chybných požiadavkách. Porovnanie základného a zabezpečeného variantu je zvolené práve preto, aby analytická časť prirodzene vyrástla z cieľov definovaných v teórii a nepôsobila ako neskôr doplnený experimentálny dodatok.

Table 2
Prepojenie analytických cieľov s meranými ukazovateľmi

Scenár	Mechanizmus	Pozorovaný ukazovateľ
Podvrhnutie HW požiadavky	HMAC-SHA-256	Miera zamietnutia, dôvod <code>bad_signature</code> .
Replay zachytenej požiadavky	<code>hw_seq</code> + <code>ReplayWindow</code>	Miera zamietnutia, dôvod <code>replay</code> alebo <code>too_old</code> .
Manipulácia interného frame	AEAD + AAD	Neúspešné overenie integrity a zamietnutie správy.
Zásah do auditnej stopy	HMAC hash chain + anchor	Úspešnosť detekcie pri verifikácii auditu.
Prevádzková záťaž	Validácia vstupu + rozhodovanie	Latencia spracovania a správanie systému pri opakovaných požiadavkách.

III. VŠEOBECNÉ BEZPEČNOSTNÉ POJMY A TERMINOLÓGIA

Táto kapitola predstavuje základné pojmy, ktoré sa používajú v ďalších častiach práce.

A. CIA triáda a rozšírené bezpečnostné ciele

CIA triáda (dôvernoscť, integrita, dostupnosť) je základný rámec, od ktorého sa pri bezpečnostnom návrhu zvyčajne začína. V prístupovom systéme však sama osebe nestačí. Pri praktickej implementácii sa veľmi rýchlo ukáže, že rovnako podstatné je vedieť, kto správu vytvoril, či ju nemožno znovu použiť a či sa dá po incidente spätne overiť história udalostí. Preto je vhodné k CIA doplniť ešte niekoľko cieľov:

- autentickosť (*authenticity*) – schopnosť overiť pôvod správy alebo akcie,
- auditovateľnosť (*auditability*) – schopnosť spätne preukázať rozhodnutia a udalosti,
- odolnosť voči replay (*replay resistance*) – schopnosť odmietnuť opakovanie starej správy.

V prototypu sa tieto ciele prejavujú veľmi konkrétne:

- Dôvernoscť: útočník nemá získať citlivé údaje (napr. identifikátory, tokeny, kľúče).
- Integrita: útočník nemá zmeniť požiadavku alebo audit záznam bez detekcie.

- Dostupnosť: systém musí zvládnuť prevádzku bez výpadkov; zároveň sa musí brániť proti DoS v rámci možností prototypu.

Table 3
Mapovanie bezpečnostných cieľov na mechanizmy použité v prototypu

Cieľ	Mechанизmus v prototypu	Príklad narušenia
Dôvernosť	AEAD frame, neukladanie surového UID	Čítanie citlivých údajov z payloadu alebo databázy.
Integrita	HMAC požiadavky, AEAD tag, hash chain auditu	Zmena uid, door_id alebo auditnej udalosti bez detekcie.
Dostupnosť	Limity vstupu, fail-closed, validácia	Pád parsera alebo vyčerpanie zdrojov pri chybných vstupoch.
Autentickosť	HMAC-SHA-256 pre hardvér	Vydávanie sa za legitímnu čítačku.
Odolnosť voči replay	hw_seq + ReplayWindow	Opakované použitie starej platnej požiadavky.
Auditovateľnosť	Tamper-evident audit log + verify nástroj	Dodatočné popretie udalosti alebo tichá úprava histórie.

B. AAA model: Authentication, Authorization, Accounting

Model AAA je v literatúre známy a pomerne stručný, no v kontexte tejto práce je užitočný práve preto, že sa dá priamo premietnuť do implementácie. Namiesto abstraktného delenia sa dá pri prototypu ukázať, kde sa ktorá časť naozaj nachádza:

- Authentication (autentifikácia) – overenie identity alebo pôvodu.
- Authorization (autorizácia) – rozhodnutie, či identita môže vykonať akciu.
- Accounting (účtovanie/audit) – záznam akcií a ich výsledkov.

V tomto prototypu je autentifikácia rozdelená na dve vrstvy. Karta slúži predovšetkým ako identifikátor používateľa, zatiaľ čo samotné zariadenie sa voči serveru preukazuje podpisom HMAC. Autorizácia prebieha v *DecisionEngine* na základe rolí a väzieb medzi čítačkou a dverami. Posledná časť modelu, teda accounting, nie je iba pomocný log pre administrátora; je to auditná stopa, z ktorej má byť možné spätne overiť, čo sa stalo a či s históriou nebolo manipulované. V tomto bode sa teoretický model AAA prirodzene prepája s odporúčaniami pre digitálnu identitu a správu logov.[7], [8]

C. Riziko, hrozba, zraniteľnosť

V ďalších kapitolách sa opakovane pracuje s pojmami hrozba, zraniteľnosť a riziko, preto má zmysel ich hneď na začiatku oddeliť. Bez tohto rozlíšenia by bolo ťažké vysvetliť, prečo sa niektoré mechanizmy zavádzajú priamo do protokolu a iné iba ako odporúčanie pre budúce nasadenie.

Bezpečnostná analýza preto rozlišuje:

- hrozbu (threat) – potenciálnu príčinu incidentu (napr. útočník schopný zachytiť sieťovú komunikáciu),
- zraniteľnosť (vulnerability) – slabinu systému (napr. absencia anti-replay),
- riziko (risk) – kombináciu pravdepodobnosti a dopadu.

V tejto práci nie je cieľom vytvoriť úplný korporátny register rizík. Dôležitejšie je ukázať, ktoré hrozby sú pre zvolený prototyp realistické a ktoré mechanizmy ich vedú znížiť na prijateľnú mieru.

D. Modelovanie hrozieb

Pri návrhu prototypu sa ukázalo, že hrozby sa dajú najlepšie udržať pod kontrolou vtedy, keď sú pomenované ešte pred implementáciou. Ako orientačný rámec je preto vhodný model STRIDE[9] (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege). Nie všetky jeho časti sú v tejto práci rovnako dôležité; najviac sa týkajú tých vrstiev, ktoré prototyp naozaj obsahuje:

- Spoofing: podvrhnutie požiadavky (riešené HMAC),
- Tampering: zmena dát alebo logov (riešené AEAD, audit hash chain),

- Repudiation: popieranie udalostí (riešené auditom),
- Information disclosure: únik identifikátorov (riešené hashovaním UID v DB a odporúčaním TLS),
- DoS: zahltenie endpointu (čiastočne riešené limitmi a validáciou).

E. Autentifikačné faktory a pojem identity

V prístupových systémoch sa často hovorí o tom, že používateľ sa „identifikuje kartou“. Je dôležité rozlišovať dva pojmy: identifikácia (tvrdenie „kto som“) a autentifikácia (dôkaz, že tvrdenie je pravdivé). V praxi sa autentifikácia realizuje pomocou *faktorov*:

- niečo, čo mám (karta, token, zariadenie),
- niečo, čo viem (PIN, heslo),
- niečo, čím som (biometria).

RFID/NFC karta v jednoduchom režime (iba UID) je teda skôr identifikátor než plnohodnotný autentifikátor. V produkčných systémoch sa tento problém zvykne riešiť ďalším faktorom, napríklad PIN-om, alebo kartou s vlastným kryptografickým protokolom. V tejto práci sa tento rozdiel neskrýva; naopak, je súčasťou interpretácie výsledkov. Riziko sa kompenzuje inde – najmä autentifikáciou zariadenia voči serveru, anti-replay mechanizmami a auditovateľnosťou. Z pohľadu odporúčaní pre digitálnu identitu ide teda o prototyp s jednoduchším poverením používateľa, ale so silnejšou ochranou komunikačnej a serverovej vrstvy.[7], [10]

F. Zodpovednosť, nepopierateľnosť a forenzná pripravenosť

V praxi nestačí iba rozhodnúť *ALLOW/DENY*. Organizácia potrebuje vedieť *kedy, kde a prečo* k rozhodnutiu došlo. Tento aspekt sa často označuje ako accountability (zodpovednosť) a je úzko spojený s auditovateľnosťou.

Pojem *nepopierateľnosť* (non-repudiation) sa v kryptografii spája najmä s digitálnymi podpismi, ale v kontexte prístupových systémov má aj praktický význam: systém by mal vedieť preukázať konzistentný reťaz udalostí a minimalizovať možnosti spätnej manipulácie. To je motivácia pre tamper-evident logovanie a pre dôsledné zaznamenávanie kontextu (identifikátor čítačky, dverí, čas, výsledok, dôvod zamietnutia).

G. Bezpečnostné požiadavky ako merateľné tvrdenia

Pri návrhu bezpečnostného riešenia je užitočné formulovať požiadavky ako overiteľné tvrdenia. Napríklad:

- *Systém odmietne opakovanie tej istej požiadavky aj v prípade, že bola zachytená na sieti.*
- *Únik databázy nesmie priamo odhaliť surové identifikátory kariet.*
- *Po manipulácii s auditným záznamom je možné určiť prvé miesto nekonzistencie.*

Takto formulované požiadavky sa dajú priamo mapovať na mechanizmy (HMAC, anti-replay, HMAC hashovanie UID, hash chain) a v ďalších častiach práce poskytujú jasné kritériá, čo sa považuje za „zvýšenie bezpečnosti“.

IV. SYSTÉMY KONTROLY PRÍSTUPU A PRÍSTUPOVÉ MODELY

Táto kapitola opisuje základné modely riadenia prístupu a požiadavky, na ktoré sa opiera implementácia prototypu. Pohľad na prístupové systémy z rôznych uhlov — fyzické vs. logické, centralizované vs. distribuované, rôzne modely autorizácie — poskytuje kontext potrebný pre pochopenie návrhových rozhodnutí v prototypovej implementácii.

A. Fyzická vs. logická kontrola prístupu

Fyzická kontrola prístupu rieši vstupy do budov a zón. Logická kontrola prístupu rieši prístup k dátam a službám. V oboch prípadoch sa používa rovnaký princíp: subjekt (používateľ) vykoná akciu, systém overí identitu a rozhodne podľa politiky.[11]

B. Modely kontroly prístupu

Z teórie sú známe rôzne modely:

- DAC (Discretionary Access Control) – vlastník objektu určuje prístup.
- MAC (Mandatory Access Control) – prístup riadi centrálna politika (napr. bezpečnostné úrovne).
- RBAC (Role-Based Access Control) – oprávnenia sú priradené rolám, používatelia majú roly.
- ABAC (Attribute-Based Access Control) – rozhodovanie podľa atribútov subjektu, objektu a kontextu.

V prototypovej implementácii je použitý RBAC prístup: karty majú roly (napr. *admin*), dvere majú povolené roly a existuje väzba medzi *reader_id* a *door_id*. Tento model je dostatočne jednoduchý pre demo, ale zároveň realistický a dobre nadväzuje na klasické RBAC modely popísané v literatúre.[11], [12]

C. Bezpečnostné požiadavky na API a server

Keďže prototyp obsahuje webové API (administrátorské aj hardvérové), je vhodné reflektovať typické API riziká, napríklad podľa OWASP API Security Top 10 a širšieho rámca OWASP ASVS.[13], [14] V kontexte prístupového systému sú kritické najmä:

- Broken Authentication: slabé overenie identity klienta,
- Broken Object Level Authorization: nedostatočné overenie oprávnení pri práci s objektmi (dvere, čítačky, karty),
- Security Misconfiguration: debug endpointy alebo slabé nastavenia.

Praktická implementácia preto oddelí uje admin token (iba pre UI) od HW autentifikácie (HMAC).

D. Zóny, kontext a väzby medzi komponentmi

Vo fyzickej kontrole prístupu sa často pracuje so zónami (napr. verejná zóna, kancelárska zóna, technická zóna). Politika prístupu potom vyjadruje, ktoré roly alebo skupiny používateľov môžu vstupovať do konkrétnych zón a v akom čase. Aj keď prototyp používa zjednodušený model *dvere* → *povolené roly*, ide o rovnaký princíp: dvere reprezentujú hranicu zóny.

Dôležitým bezpečnostným prvkom je väzba (binding) medzi čítačkou a dverami. Ak by útočník vedel poslať na server udalosť s ľubovoľným *door_id*, mohol by zneužiť legitímnu čítačku na otvorenie iných dverí. Preto sa v prístupových systémoch používa:

- fyzické opatrenie (umiestnenie čítačky pri konkrétnych dverách),
- konfiguračná väzba (server pozná, ku ktorým dverám čítačka patrí),
- a v ideálnom prípade aj kryptografická identita zariadenia.

V prototypovej architektúre sa väzba realizuje databázovým pravidlom *reader_id* ↔ *door_id*. Všeobecne ide o ukážku toho, ako sa *kontext* stáva súčasťou autorizácie.

E. Princíp najmenej oprávnení a oddelenie právomocí

Princíp *least privilege*[15] hovorí, že každá komponenta má mať len také oprávnenia, ktoré nevyhnutne potrebuje. V prístupovom systéme to znamená, že:

- hardvérové zariadenie nemá mať administrátorské oprávnenia,
- administrátorské rozhranie a jeho tokeny majú byť oddelené od mechanizmov zariadenia,
- a databázové operácie majú byť obmedzené na minimum potrebné pre rozhodovanie a audit.

Oddelenie právomocí znižuje dopad kompromitácie jednej časti systému. Ak by napríklad útočník získal kontrolu nad čítačkou, stále by nemal automaticky získať možnosť meniť prístupové pravidlá alebo exportovať databázu.

V. BEZPEČNOSTNÉ HROZBY A ŠPECIFIKÁ RFID/NFC TECHNOLOGIÍ

RFID a NFC pôsobia na prvý pohľad jednoducho: karta sa priloží, čítačka prečíta identifikátor a systém rozhodne. Práve táto jednoduchosť však zvädza k podceneniu hrozieb. V praxi nebýva problémom iba samotná karta, ale celý reťazec od rádiovej komunikácie až po serverové spracovanie. NIST SP 800-98 preto nepozera na RFID ako na izolovaný prvok, ale ako na súčasť väčšieho bezpečnostného systému[16].

Pre túto prácu je dôležité jedno rozlíšenie. Prototyp nepoužíva kryptograficky silnú kartu s mutual authentication; pracuje s UID ako s identifikátorom, nad ktorým sa až následne uplatnia serverové bezpečnostné mechanizmy. To je vedomé rozhodnutie a treba ho vnímať už pri čítaní celej kapitoly.

A. Typické útoky na RFID/NFC

Pri RFID/NFC sa často spomínajú tie isté triedy útokov, ale nie všetky majú v každom projekte rovnakú váhu. V našom prototypu sú najdôležitejšie najmä tie, ktoré priamo súvisia s tým, že karta neposkytuje kryptografický dôkaz identity.

- **Skimming a eavesdropping:** útočník sa snaží zachytiť komunikáciu medzi čítačkou a kartou. Krátky dosah NFC pomáha, ale s lepšou anténou alebo v rušnom prostredí sa nedá rátať s tým, že odpočúvanie je „prakticky nemožné“.
- **Cloning a emulácia:** ak systém stojí iba na UID, z klonovania sa stáva veľmi realistický problém. Útočník nepotrebuje poznať prístupovú politiku; stačí mu vedieť napodobniť identifikátor.
- **Relay útoky:** tu nejde o kopírovanie karty, ale o predĺženie jej dosahu. Čítačka tak komunikuje s legitímnou kartou, ktorá je fyzicky inde.
- **Manipulácia čítačky:** ak útočník nahradí čítačku vlastným zariadením alebo sa pripojí na jej výstup, problém sa presúva z rádiovkej vrstvy na aplikačnú a sieťovú vrstvu.

B. Mitigácie a odporúčané protiopatrenia

Nie je realistické tvrdiť, že jeden mechanizmus vyrieši celý problém RFID/NFC. Obrana sa preto skladá z viacerých vrstiev:

- **Kryptografická karta:** ak karta vie preukázať znalosť tajomstva, znižuje sa význam samotného UID.
- **Druhý faktor:** PIN alebo biometria dávajú zmysel najmä tam, kde by jediné zneužitie karty malo vysoký dopad.
- **Obrana proti relay:** distance bounding a prísnejšie časové limity sú technicky zaujímavé, ale v jednoduchom prototypu by zbytočne zvyšovali zložitosť.
- **Organizačné opatrenia:** fyzická kontrola vstupov, kamera alebo revízia oprávnení sú menej elegantné než kryptografia, no v reálnej prevádzke bývajú veľmi účinné.

V tejto práci sa preto nehrá na to, že samotná karta je „silný autentifikátor“. Posilňuje sa až vyššia vrstva: čítačka sa musí serveru preukázať pomocou HMAC, požiadavka má monotónne hw_seq a výsledok spracovania sa zapisuje do auditu tak, aby ho bolo možné dodatočne overiť.

Table 4
Vybrané hrozby a ich väzba na protiopatrenia v prototypu

Hrozba	Protiopatrenie	Poznámka
Spoofing čítačky	HMAC podpis požiadavky	Chráni pôvod a integritu HTTP požiadavky.
Replay požiadavky	Monotónne hw_seq + ReplayWindow	Účinné aj po reštarte zariadenia pri zachovaní perzistentného stavu.
Manipulácia interných dát	AEAD (ChaCha20/XChaCha20-Poly1305) + AAD	Chráni payload a viaže hlavičku na šifrované dáta.
Únik databázy	HMAC(card_id) s pepper	Obmedzuje priamu použiteľnosť uniknutých identifikátorov.
Mazanie alebo úprava logov	HMAC hash chain + anchor	Umožňuje neskoršiu detekciu nekonzistencie auditu.

C. Príklad známych slabín: MIFARE Classic

Historicky boli široko nasadené karty typu MIFARE Classic[17], ktoré používajú proprietárny algoritmus CRYPTO1. Výskum Garcia *et al.*[18] ukázal vážne slabiny v tomto mechanizme a umožnil praktické útoky vedúce k získaniu kľúčov a klonovaniu kariet. Následné práce[19] rozšírili tieto útoky na ďalšie systémy používajúce podobné proprietárne šifry. Z toho vyplýva dôležitá poučka: spoliehanie sa na utajenie algoritmu alebo na UID ako jediný faktor je nedostatočné. Aj keď konkrétny prototyp v práci nepoužíva autentifikáciu na úrovni karty, návrh kompenzuje riziká na úrovni servera a protokolu (HMAC, anti-replay, audit).

D. Bezpečnostné dôsledky pre návrh prototypu

Z predchádzajúcich bodov pre prototyp vyplývajú štyri jednoduché pravidlá. Po prvé, UID sa používa len ako identifikátor vstupu do politiky. Po druhé, dôveru prenášame z karty na čítačku a serverové spracovanie. Po tretie, každá požiadavka musí nieť stopu čerstvosti, inak by ju bolo možné opakovať. A napokon, ani databáza nesmie obsahovať viac údajov, než je skutočne potrebné. Tento rámec sa potom priamo odráža v praktickej implementácii.

E. Hrozby na komunikačnej vrstve a v sieťovej infraštruktúre

Aj keď je fyzická technológia (RFID/NFC) často v centre pozornosti, v moderných riešeniach je rovnako kritická sieťová komunikácia medzi čítačkou a serverom. Medzi typické hrozby patria:

- Man-in-the-Middle (MITM): útočník zmení alebo presmeruje komunikáciu (napr. kompromitovaný prístupový bod Wi-Fi).
- Replay útoky: zachytená platná požiadavka sa opakuje s cieľom dosiahnuť rovnaký výsledok.
- Podvrhnutie klienta: útočník sa vydáva za čítačku a posielá vlastné požiadavky.
- Injekcia a deserializácia: nevalidovaný vstup môže viesť k pádom, obchádzaniu logiky alebo k zneužitiu parsera.

Práve tieto hrozby sú motiváciou pre použitie HMAC na autentifikáciu zariadenia a pre dôsledné limity a validáciu vstupov na serveri.

F. Hrozby na serverovej strane a v API

Server spracúva rozhodnutia a drží konfiguráciu (dvere, roly, väzby). To z neho robí najcennejší cieľ. Okrem klasických sieťových útokov (DoS) sú relevantné najmä:

- nesprávna autorizácia: chyby v kontrole oprávnení pri operáciách typu *CRUD*,
- únik tajomstiev: logovanie kľúčov, nesprávne nastavené prístupové práva, nechcené debug endpointy,
- privilege escalation: získanie administrátorských práv cez slabú autentifikáciu alebo zraniteľnosť v UI,
- kompromitácia závislostí: použitie knižníc s bezpečnostnými chybami.

Z teoretického hľadiska je dôležité uplatniť *defense-in-depth*: aj keď jedna vrstva zlyhá, ďalšie vrstvy majú minimalizovať dopad.

G. Útoky po incidente: manipulácia auditnej stopy

Po úspešnom preniknutí do systému sa útočník často snaží znížiť šancu na odhalenie. Najčastejšie to znamená upraviť logy alebo ich čiastočne odstrániť. Preto je auditovateľnosť a integrita logov dôležitou súčasťou bezpečnostného návrhu, nie iba „doplňkom“. Mechanizmy ako hash chain a anchor umožňujú neskoršie overenie, či auditná stopa bola zachovaná v konzistentnom stave.

VI. KRYPTOGRAFICKÉ ZÁKLADY RELEVANTNÉ PRE PRÍSTUPOVÉ SYSTÉMY

V tejto časti nejde o „prehľad všetkej kryptografie“. Potrebné je skôr pochopiť, čo z použitých mechanizmov v prototypu skutočne robí a čo od nich naopak očakávať nemožno. Práve zámena týchto rolí býva častým zdrojom chýb: hash sa zamieňa za autentifikáciu, TLS za autorizáciu a šifrovanie za celkovú bezpečnosť systému.

A. Hashovacie funkcie a integrita

Hashovacia funkcia je v bezpečnostnom návrhu užitočná, ale jej možnosti sú obmedzené. Vie vytvoriť pevne dlhý odtlačok vstupu a dobre odhalí náhodnú alebo úmyselnú zmenu dát. Nevyrieši však otázku pôvodu správy. Ak útočník zmení obsah, nič mu nebráni prepočítať aj nový hash. Preto samotný hash nestačí všade tam, kde chceme vedieť nielen to, že sa správa zmenila, ale aj to, kto ju vytvoril.

B. MAC a HMAC

MAC je mechanizmus, ktorý s využitím tajného kľúča zabezpečuje integritu a autentickosť správy. HMAC je konkrétna konštrukcia MAC nad hashovacou funkciou a je štandardizovaná vo FIPS 198-1; pôvodná všeobecná konštrukcia HMAC je popísaná aj v RFC 2104.[1], [20] HMAC sa používa v prototypovej implementácii pre autentifikáciu požiadaviek hardvéru.

Dôležité implementačné poznámky:

- Kanonizácia vstupu: HMAC sa musí počítať nad jednoznačnou reprezentáciou správy. Ako príklad jednoznačnej kanonizácie sa môže použiť reťazec "POST /api/hw/uid\n" + raw JSON body. Táto voľba zabezpečuje, že aj zmena whitespace alebo poradia polí (ak by JSON serializácia nebola stabilná) môže ovplyvniť podpis. Prakticky preto zariadenie posieľa raw telo a server podpisuje rovnaký reťazec.
- Konštantnočasové porovnanie: výsledný HMAC sa musí porovnávať konštantným časom, aby sa minimalizovalo riziko timing útokov.
- Oddelenie kľúčov: kľúč pre HW autentifikáciu nesmie byť zdieľaný s inými účelmi (napr. audit). HKDF umožňuje oddeliť podkľúče[6].

C. Výber parametrov a bezpečnostná úroveň

Pri použití kryptografických primitív je vhodné uvažovať o bezpečnostnej úrovni (napr. 128-bitová bezpečnosť) a o tom, aké parametre to implikuje. V praxi to znamená:

- používať dostatočne dlhé kľúče (typicky 256 bitov pre symetrické primitíva),
- nepoužívať zastarané hashovacie funkcie (napr. MD5, SHA-1) pre bezpečnostné účely,
- a vyhnúť sa „zvláštnym“ vlastným formátom, ktoré môžu skryť chyby v spracovaní.

V prístupových systémoch je často kľúčová dlhodobá udržateľnosť: zariadenia môžu byť nasadené roky. Preto je vhodné mať v protokole priestor pre kryptografickú agilitu (verzovanie kľúčov, možnosť rotácie) a vyhnúť sa algoritmom, ktoré sú viazané na špecifický hardvér.

D. Bezpečná práca s pamäťou a citlivými údajmi

Kryptografické kľúče a medzi-výsledky (napr. HMAC tag, plaintext pred šifrovaním) predstavujú citlivé údaje aj v pamäti procesu. Z teoretického hľadiska sa odporúča:

- minimalizovať dobu, počas ktorej sú citlivé údaje v pamäti,
- prepisovať buffere po použití, ak knižnica poskytuje bezpečné API,
- a vyhýbať sa kopírovaniu citlivých údajov do logov alebo výnimiek.

Tieto zásady sú obzvlášť relevantné na serverovej strane, kde proces môže bežať dlhodobo a kde sa môžu objaviť *core dumps* alebo diagnostické výpisy.

E. Symetrické šifrovanie a AEAD

Symetrické šifrovanie je vhodné pre embedded zariadenia aj servery kvôli výkonu. Samotné šifrovanie však nerieši integritu. V praxi sa preto používa AEAD, ktoré spája dôvernosť a integritu do jednej operácie.

1) *ChaCha20 a Poly1305*: ChaCha20 je prúdová šifra z rodiny Salsa20, navrhnutá D. J. Bernsteinom s cieľom dosiahnuť vysoký výkon v softvéri bez závislosti na hardvérovej akcelerácii[21]. Poly1305 je jednorázový MAC. Ich kombinácia je štandardizovaná v RFC 8439 a zapadá do modelu AEAD rozhrania podľa RFC 5116[2], [22].

a) *Interná štruktúra ChaCha20*.: Stav ChaCha20 je matica 4×4 32-bitových slov (512 bitov celkovo), zložená zo štyroch konštantných slov, ôsmich slov 256-bitového kľúča, jedného slova počítadla a troch slov nonce (pri RFC 8439; XChaCha20 používa dve slová nonce po HChaCha20 kroku).

Základnou operáciou je *quarter round* $QR(a, b, c, d)$ — ARX konštrukcia (Add-Rotate-XOR):

$$\begin{aligned} a &+= b; & d &\oplus= a; & d &\lll 16 \\ c &+= d; & b &\oplus= c; & b &\lll 12 \\ a &+= b; & d &\oplus= a; & d &\lll 8 \\ c &+= d; & b &\oplus= c; & b &\lll 7 \end{aligned}$$

Dvadsať kôl (10 stĺpcových + 10 diagonálnych) transformuje počiatočný stav na výstupný keystream blok (512 bitov). Ten sa XOR-uje s plaintext-om[2]. Kľúčová bezpečnostná vlastnosť ARX konštrukcie: všetky operácie sú identicky časované naprieč hodnotami vstupu — žiadne

podmienené vetvenie ani tabuľkové vyhľadávania. To zaručuje *konštantný čas* výpočtu, čím sa eliminuje trieda *timing side-channel* útokov, ktorá je reálnym rizikom softvérových implementácií AES bez hardvérovej podpory[21].

b) *Poly1305 MAC*.: Poly1305 je jednorázový autentifikátor postavený na polynomiálnom výpočte v konečnom poli $\mathbb{F}_{2^{130}-5}$. Správa sa rozdelí na 16-bajtové bloky $m_1, m_2, \dots, m_n$ (s prídavným bitom), vyhodnotí sa polynóm s tajomným kľúčom r a výsledok sa maskuje hodnotou s :

$$\text{tag} = ((m_1 r^n + m_2 r^{n-1} + \dots + m_n r) \bmod (2^{130} - 5) + s) \bmod 2^{128} \quad (1)$$

Hodnoty r a s (32 B spolu) sú odvodené z prvého keystream bloku ChaCha20 (counter = 0). Každý kľúč (r, s) sa musí použiť iba raz — pri opakovanom použití r pod rovnakou správou by útočník mohol extrahovať r a kovať MAC[2].

c) *RFC 8439 kompozícia*.: ChaCha20-Poly1305 AEAD podľa RFC 8439 funguje takto: ChaCha20 s counter = 0 vygeneruje 64-bajtový blok; prvých 32 B tvorí Poly1305 kľúč (r, s) . Šifrovanie prebieha s counter ≥ 1 . Poly1305 autentifikuje konkaténáciu AAD (s výplňou na 16 B), šifrovaného textu (s výplňou) a dĺžkových polí (8 B + 8 B). Výsledný 16-bajtový tag pokrýva kontext aj obsah súčasne[2], [22]. Postup šifrovania a dešifrovania je zobrazený na obrázku 2.

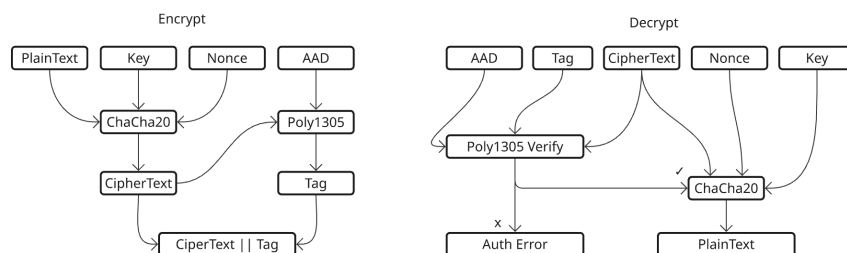


Fig. 2 ChaCha20-Poly1305 AEAD: tok šifrovania (vľavo) a dešifrovania (vpravo). Poly1305 autentifikuje AAD aj CipherText spoločne. Pri dešifrovaní

2) *ChaCha20-Poly1305, XChaCha20-Poly1305 a nonce manažment*: AEAD konštrukcie sú „krehké“ voči zneužitiu nonce: opakovanie nonce pod rovnakým kľúčom môže viesť k vážnym únikom informácií. XChaCha20-Poly1305 rozširuje nonce na 192 bitov, čo umožňuje bezpečnejšie používanie náhodných nonce pre dlhodobu žijúce kľúče[3], [4]. ChaCha20-Poly1305 IETF (RFC 8439) používa 96-bitový nonce s nižším bezpečným limitom pri náhodnom generovaní ($\sim 2^{48}$ správ)[2]. V prototypovej implementácii sú podporené obe varianty, pričom nonce môže byť generovaný náhodne alebo deterministicky cez HMAC — porovnanie oboch prístupov je predmetom experimentov.

3) *Associated Data (AAD)*: AEAD podporuje *Associated Authenticated Data* (AAD): dáta, ktoré sa nešifrujú, ale ich integrita je kryptograficky chránená. V prístupovom protokole je AAD použité pre hlavičku frame. Tým sa zabezpečí, že zmena *reader_id*, *door_id* alebo *seq* spôsobí zlyhanie overenia tagu.

4) *Porovnanie s AES-GCM*: AES-GCM (Galois/Counter Mode) je dominantnou AEAD konštrukciou v TLS 1.3 a podnikových systémoch[23]. Pre kontext tejto práce je relevantné porozumieť, prečo bol zvolený ChaCha20-Poly1305 namiesto AES-GCM.

Table 5
Porovnanie ChaCha20-Poly1305 a AES-GCM

Vlastnosť	ChaCha20-Poly1305	AES-GCM
Veľkosť nonce	96 b (IETF) / 192 b (XChaCha20)	96 b
Nonce reuse	Únik XOR plaintextov (two-time pad)	Kritické: extrahovateľný GHASH kľúč[22]
Softvér bez HW	Konštantný čas (ARX)[21]	Timing side-channel bez AES-NI
HW akcelerácia	AVX2/SSSE3 (SIMD)	AES-NI (Intel/ARM)
Vhodnosť pre IoT	Vysoká	Iba s HW akcelerátorom
Štandardizácia	RFC 8439, TLS 1.3	RFC 5116, TLS 1.3

Kľúčovým rozdielom pre softvérové nasadenie je bezpečnosť implementácie bez hardvérovej podpory. AES pracuje so S-boxom, ktorého tabuľkové implementácie vykazujú závislosti na

vstupe v čase prístupu do cache — tzv. *cache timing* útok[21]. ChaCha20 je čistá ARX konštrukcia bez tabuľkových vyhľadávaní, kde sú všetky operácie identicky časované.

Rozdiel pri zneužití nonce je obzvlášť dôležitý: pri AES-GCM vedie opakovaný nonce k tzv. *forbidden attack*, kde zo dvoch šifrovaných textov pod rovnakým nonce a kľúčom možno analyticky extrahovať hodnotu použitú v GHASH a kovať MAC tagy pre ľubovoľné správy[22]. Pri ChaCha20-Poly1305 vedie nonce reuse k úniku XOR plaintextov, čo je vážne, ale Poly1305 kľúč r zostáva neodhalený — degradácia bezpečnosti je teda iného charakteru.

Pre prototyp nasadzovaný na ESP32 (bez AES-NI) je ChaCha20-Poly1305 konzistentne bezpečná voľba bez platformovej závislosti.

F. Náhodnosť, nonce a praktické generovanie

Mnohé bezpečnostné protokoly stoja na predpoklade, že niektoré hodnoty sú nepredvídateľné (napr. nonce, kľúče). V embedded prostredí však môže byť generovanie kvalitnej náhodnosti problematické: zariadenie nemusí mať dostatok entropie a môže sa spoliehať na deterministické zdroje. Existujú tri základné stratégie generovania nonce:

- **Náhodný nonce** — jednoduchý, ale závisí na kvalite RNG a pri krátkom nonce (96 bitov u ChaCha20) hrozí kolízia pri vysokom počte správ.
- **Deterministický nonce (HMAC)** — nonce sa odvodí ako $\text{HMAC}(K_{\text{nonce}}, \text{context})$, kde kontext zahŕňa hlavičku správy. Eliminuje závislosť na RNG, ale vyžaduje zdieľanie nonce kľúča medzi odosielateľom a overovateľom.
- **Deterministický nonce s runtime verifikáciou** — rovnaký HMAC nonce, ale server navyše overuje, že prijatý nonce zodpovedá očakávanej hodnote. Ak nezodpovedá, zariadenie sa zakaranténuje.

Rozšírený nonce pri XChaCha20-Poly1305 znižuje riziko kolízie aj pri náhodných noncech[4], čo je výhodné v systémoch s dlhšou životnosťou kľúčov. Deterministický HMAC nonce túto výhodu posúva ešte ďalej: pri správnej konštrukcii sa nonce neopakuje ani pri chybnom RNG.

G. Domain separation a kontext v derivácii kľúčov

Aj keď je HKDF formálne jednoduchý, v praxi je kľúčové správne nastaviť *context/label* (parameter *info*). Domain separation znamená, že z rovnakého master secret sa odvodí rôzne podkľúče tak, aby sa nedali zameniť. Typickým prístupom je použiť popis účelu, napríklad "hw-auth", "aead-frame" alebo "audit-chain". Takéto označovanie je jednoduché, ale výrazne znižuje riziko, že sa jeden kľúč „nechtiac“ použije inde.

H. Kryptografia vs. bezpečnosť implementácie

Bezpečnosť algoritmov môže byť oslabená implementáciou. Typické chyby sú porovnávanie MAC ne-konštantným časom, únik citlivých údajov do logov, neobmedzené spracovanie veľkých vstupov (vyčerpanie zdrojov) a nedostatočné ošetrovanie chýb. Preto sa v bezpečnom návrhu uplatňuje zásada: primitíva sú nutné, ale nestačia. Rovnako dôležité je robustné spracovanie vstupov, limity a jednotné chybové správanie.

I. Bezpečnostné modely AEAD a nonce-misuse resistance

Porozumenie formálnym bezpečnostným modelom pomáha presne formulovať, čo zaručujú kryptografické protokoly a kde sú hranice ich bezpečnosti.

a) *Štandardný model (nAE)*.: AEAD bezpečnosť sa formálne definuje dvoma vlastnosťami[22]:

- **IND-CPA** (Indistinguishability under Chosen-Plaintext Attack): šifrovaný text je nerozoznateľný od náhodných dát aj keď útočník môže vybrať plaintexty.
- **INT-CTXT** (Integrity of Ciphertext): útočník nedokáže skovať platný šifrovaný text s platným tagom, ani pri adaptívnych dotazoch na šifrovaní.

Kombinácia IND-CPA + INT-CTXT tvorí *AE* bezpečnosť — presne to, čo poskytujú ChaCha20-Poly1305 a XChaCha20-Poly1305 pri unikátnych nonce[2]. Štandardný model (*nonce-based AE*, *nAE*) predpokladá, že nonce sa neopakuje. Porušenie tohto predpokladu degraduje bezpečnostné záruky spôsobom závislým od konkrétnej konštrukcie.

b) *Nonce-misuse resistance*.: Rogaway a Shrimpton[24] formalizovali silnejší model, kde šifra zachováva *dôvernoscť* aj pri opakovanom nonce — za cenu, že útočník rozpozná zhodné plaintexty pri rovnakom nonce. Konštrukcie dosahujúce tento cieľ (napr. SIV alebo DAE)[25] vyžadujú spravidla dva priechody správou.

Táto práca nepoužíva nonce-misuse resistant primitívum, ale namiesto toho kompenzuje na protokolovej úrovni: profily R1/R2 používajú deterministický HMAC nonce odvodený z kontextu vrátane unikátneho seq — nonce sa neopakuje, kým sa neopakuje celý kontext. Profil R2 navyše aktívne detekuje nonce mismatch na serveri a zakaranténuje zariadenie. Ide o praktický kompromis: zachováva nAE primitívum (bez overhead SIV), ale znižuje riziko zneužitia nonce prostredníctvom deterministickej konštrukcie a serverovej verifikácie[24], [25].

VII. ZÁSADY NÁVRHU BEZPEČNÝCH PROTOKOLOV

Pri čítaní bezpečnostnej literatúry je ľahké nadobudnúť dojem, že rozhoduje najmä výber algoritmu. Pri praktickom protokole to tak nebýva. O výsledku často rozhodne obyčajná vec: nejednoznačný formát, nevhodná chybová hláška alebo príliš dôverčivý parser. Preto má táto kapitola možno menej „kryptografický“ charakter, ale pre výslednú bezpečnosť je rovnako dôležitá.

A. *Fail-closed správanie*

Pri prístupovom systéme býva prirodzené pýtať sa, ako zabezpečiť, aby sa dvere otvorili vždy, keď majú. Bezpečnostný návrh si však musí rovnako tvrdo položiť opačnú otázku: čo sa stane, keď si systém nie je istý? V tejto práci je odpoveď jednoznačná. Ak zlyhá HMAC, validácia vstupu, anti-replay alebo interná konzistencia dát, výsledkom nie je „skúsme to ešte nejako zachrániť“, ale zamietnutie. Takýto prístup je menej pohodlný pri ladení, no bezpečnejší v prevádzke.

B. *Jednoznačná serializácia a kanonizácia*

Pri podpisovaní alebo šifrovaní je kritické, aby všetky strany pracovali s rovnakou reprezentáciou dát. Ak by napríklad hardvér podpisoval JSON s iným poradím polí než server, mohlo by dôjsť k chybnému odmietnutiu. Naopak, ak by server akceptoval viacero reprezentácií, útočník môže hľadať *gadgets* na bypass podpisu. Preto prototyp podpisuje *raw body* a server používa rovnaký presný vstup do HMAC.

C. *Verzovanie formátu a algoritmov*

Bezpečné systémy musia počítať s evolúciou: zmeny formátu, rotácia kľúčov, prípadne migrácia algoritmov. Preto protokol obsahuje polia ako `key_version` a interné limity (maximálne dĺžky polí, verzovanie), čo znižuje riziko „tichých“ nekompatibilití.

D. *Bezpečné spracovanie chýb a jednotné odpovede*

Chybové správy sú často podceňovaný komunikačný kanál. Ak systém vracia detailné dôvody zlyhania (napr. „nesprávny podpis“, „neznáma čiarka“, „príliš staré seq“), útočník môže tieto informácie využiť na *enumeráciu* a postupné zlepšovanie útoku. Zároveň však prevádzka potrebuje vedieť, čo sa deje. Rozumný kompromis je: jednotná odpoveď smerom k nedôveryhodnému klientovi a detailný dôvod iba v internom logu a administrácii.

E. *Ochrana proti DoS a vyčerpaniu zdrojov*

DoS (Denial of Service) je relevantný aj pre prístupové scenáre, pretože môže spôsobiť výpadok vstupov alebo núdzový režim. Na úrovni protokolu a servera je vhodné uplatniť limity veľkosti požiadaviek, rýchlu validáciu pred náročnými operáciami (napr. pred dešifrovaním), a jednoduchú ochranu proti floodu (rate limiting). Takéto opatrenia nezrušia DoS úplne, ale zvyšujú odolnosť.

F. *Bezpečnosť knižníc a závislostí*

Kryptografické primitíva je vhodné realizovať cez overené knižnice. Z teoretického hľadiska to znamená minimalizovať „vlastnú kryptografiu“, sledovať aktualizácie a mať testy, ktoré zachytia regresie. V prototypovej implementácii je kryptografická vrstva postavená na knižnici `libsodium`. [4]

G. *Mapovanie bezpečnostných cieľov na mechanizmy*

Tabuľka 6 sumarizuje, ktorý mechanizmus v prototypovom systéme poskytuje akú bezpečnostnú vlastnosť.

Bezpečnostná vlastnosť	Mechanizmus v prototypu
Autentickosť požiadavky od HW	HMAC-SHA-256 nad HTTP požiadavkou
Integrita metadát (reader/door/seq)	AEAD AAD väzba hlavičky frame
Dôvernosť interného payloadu	AEAD ChaCha20/XChaCha20-Poly1305
Ochrana proti replay	hw_seq + ReplayWindow (sliding window)
Súkromie identifikátora v DB	HMAC(card_id) s pepper
Detekcia manipulácie auditu	HMAC hash chain + audit anchor

Table 6

Mapovanie bezpečnostných cieľov na mechanizmy v prototypovom systéme

VIII. SPRÁVA KRYPTOGRAFICKÝCH KLÚČOV

V prototypoch sa o klúčoch často píše iba jednou vetou: „uložia sa do konfigurácie“. To síce môže byť pravda, ale z bezpečnostného pohľadu to nič nevysvetľuje. Aj jednoduchý systém potrebuje vedieť, odkiaľ sa klúče berú, na čo presne slúžia, kedy sa menia a čo sa stane po incidente. NIST SP 800-57 tento problém opisuje ako životný cyklus klúča[26]; v praxi je to skôr séria nepríjemných, ale nevyhnutných prevádzkových otázok.

A. Oddelenie klúčov podľa účelu

Dobrá prax je používať odlišné klúče pre odlišné účely:

- klúč pre HW autentifikáciu (HMAC podpis HTTP),
- klúč pre AEAD šifrovanie payloadu,
- klúč pre audit hash chain,
- *pepper* pre hashovanie identifikátorov kariet.

Použitie jedného klúča pre viac účelov môže viesť k *cross-protocol* útokom alebo k nejasnostiam pri rotácii. Preto je vhodné odvodiť podklúče z hlavného tajomstva pomocou HKDF[6].

B. Verzovanie a rotácia klúčov

V prototypovej implementácii má čítačka priradenú `key_version`. Pri rotácii sa v databáze nastaví nová verzia a server môže dočasne akceptovať aj predchádzajúcu verziu (*allow-previous*), aby sa minimalizoval výpadok pri prechode.

C. Ukladanie klúčov a bezpečnosť servera

Klúče sú v prototypovej verzii uložené v konfiguračnom súbore servera (ako hex reťazce) a načítané pri štarte. Ide o pragmatické riešenie vhodné pre experimentálny prototyp, no z pohľadu produkčnej bezpečnosti predstavuje vedomý kompromis. V produkčnom nasadení by bolo vhodné použiť dedikované úložisko tajomstiev (napr. HSM, Vault), obmedziť prístupové práva procesu a oddeliť správu tajomstiev od aplikačného servera.

D. Provisioning a registrácia zariadení

V produkčnom systéme je dôležitá fáza provisioningu: ako sa čítačka prvýkrát dostane do systému a ako sa jej priradia tajomstvá. Teoreticky existujú dva prístupy: predzdieľané tajomstvo (PSK) vložené pri výrobe/inštalácii alebo bootstrap protokol (napr. na báze certifikátov). PSK je jednoduchšie, ale horšie škáluje a vyžaduje disciplinované nakladanie s tajomstvom počas celej životnosti zariadenia.

E. Kompromitácia klúča a postup obnovy

Aj pri dobrej implementácii treba počítať s kompromitáciou zariadenia alebo únikom klúča. Incident plán by mal riešiť, ako rýchlo vieme zablokovať kompromitované zariadenie, ako prebehnú rotácia klúčov a ako identifikujeme udalosti, ktoré mohli byť ovplyvnené kompromitáciou.

F. Evidencia verzií a migračné stratégie

Migrácia bezpečnostných mechanizmov je citlivá, pretože výpadok znamená praktický problém v prevádzke. Z teoretického hľadiska sa často používa fázovaný rollout, krátke obdobie dual-acceptance (prijíma sa stará aj nová verzia) a následná revokácia starších verzií.

Pre digitálne identity a autentifikátory sú relevantné aj odporúčania NIST SP 800-63-4, ktoré zdôrazňujú riadenie autentifikátorov, životný cyklus a procesy obnovy.

IX. NÁVRH BEZPEČNÉHO KOMUNIKAČNÉHO PROTOKOLU

Bezpečnostné princípy z predchádzajúcich kapitol sa musia premietnuť do konkrétneho protokolu, ktorý bude fungovať v reálnych podmienkach. Táto kapitola popisuje tok spracovania udalosti, formát správ a kľúčové dizajnové rozhodnutia, ktoré sú základom pre implementovaný prototyp.

A. End-to-end tok udalosti

V tejto práci sa z bezpečnostného hľadiska oplatí sledovať udalosť od úplného začiatku. Čítačka prečíta UID, zariadenie zvýši `hw_seq`, vytvorí krátku HTTP požiadavku a podpíše ju. Server po prijatí ešte stále „nepovoľuje vstup“; najskôr overí pôvod a základnú konzistenciu správy. Až potom sa z požiadavky vytvorí interný secure frame, ktorý vstupuje do rozhodovacej logiky.

Prakticky teda tok vyzerá takto:

- 1) RC522 načíta UID karty.
- 2) ESP32 inkrementuje perzistentné `hw_seq`.
- 3) Do `/api/hw/uid` sa odošle JSON s `uid`, `reader_id`, `door_id` a `hw_seq`.
- 4) Hlavička `X-HW-Signature` nesie HMAC nad presným telom požiadavky.
- 5) Server overí podpis a validuje vstup.
- 6) Zo správy vytvorí interný frame, kde sa `hw_seq` použije ako `seq`.
- 7) *DecisionEngine* vyhodnotí pravidlá a výsledok sa uloží do auditu.
- 8) *EventBus* sprístupní runtime udalosti pre administráciu.

Takéto rozdelenie vrstiev je dôležité aj pri interpretácii výsledkov. HMAC medzi zariadením a serverom rieši autenticitu a integritu požiadavky. AEAD frame na serverovej strane potom rieši vnútorné spracovanie, väzbu metadát a auditovateľnosť ďalších krokov.

B. Formát správ, validácia vstupu a obrana pred parser útokmi

V prístupovom systéme predstavuje vstupná správa (napr. JSON požiadavka) hranicu medzi nedôveryhodným svetom a internou logikou servera. Aj keď je správa podpísaná, server musí počítať s tým, že útočník sa môže pokúsiť o:

- poslať extrémne veľké alebo špeciálne formátované vstupy (vyčerpanie pamäte),
- využiť rozdiely v parsovaní (napr. rozdiel medzi „číslo“ a „reťazec“),
- obísť kontrolu pomocou duplicitných polí alebo neštandardných kódovaní.

Z teoretického hľadiska sa odporúča spracovať vstup v dvoch krokoch:

- 1) syntaktická validácia (správny JSON, maximálna veľkosť, povolené znaky),
- 2) semantická validácia (povinné polia, rozsahy hodnôt, typy, konzistentnosť).

Až po úspešnej validácii má zmysel vykonávať ďalšie operácie (napr. dešifrovanie, databázové dotazy). Tento postup znižuje riziko DoS a zlepšuje robustnosť.

C. Hranice dôvery a spracovanie metadát

Nie všetky polia v správe majú rovnaký význam. Napríklad `uid` je identifikátor karty, no nie je dôkazom autenticity. Naopak `reader_id` a `door_id` sú metadátá, ktoré určujú kontext autorizácie a musia byť chránené proti manipulácii. V návrhu je dôležité definovať, ktoré polia sa:

- považujú za nedôveryhodné a slúžia iba ako vstup do politiky,
- musia byť kryptograficky viazané (napr. cez HMAC alebo AEAD AAD),
- musia byť odvodené na serveri (napr. časová pečiatka).

Takéto rozdelenie znižuje riziko, že server začne dôverovať údajom, ktoré útočník vie ovplyvniť.

D. Škálovanie anti-replay pri viacerých zariadeniach

Anti-replay mechanizmus musí byť v praxi navrhnutý tak, aby fungoval pre viac zariadení súčasne. To znamená, že *ReplayWindow* sa typicky udržiava per identitu zariadenia (napr. per `reader_id`). Pri návrhu je potrebné rozhodnúť:

- aká je veľkosť okna (koľko hodnôt tolerujeme),
- či tolerujeme out-of-order doručenie (napr. pri výpadkoch siete),
- a ako riešime reštart zariadenia alebo stratu stavu.

Z teoretického hľadiska je vhodné, aby server odmietol nečakaný pokles sekvencie, ale zároveň poskytol administrátorovi diagnostiku (napr. zariadenie bolo resetované). Vtedy je možné zvoliť bezpečný proces obnovy (napr. re-provisioning alebo manuálne potvrdenie resetu).

E. Časové okná, oneskorenie a „freshness“

Ak sa používa časová pečiatka, treba počítať s oneskorením siete a s tým, že klient nemusí mať presný čas. Praktický prístup je, že server generuje čas pre interné správy a pri prijíme požiadavky používa čas iba pre audit a pre detekciu anomálií (napr. nezvyčajne dlhé oneskorenie). Ak sa predsa používa klientsky čas, musí existovať tolerancia (maximálny *skew*) a fail-closed správanie mimo tolerancie.

F. Autentifikácia hardvéru cez HMAC

HMAC podpis požiadavky zabezpečuje, že iba zariadenie s tajným kľúčom vie vytvoriť platnú požiadavku. To bráni podvrhnutiu (spoofing). V prototypovej architektúre je dôležité, že zariadenie nepoužíva administrátorský token (ten je určený iba pre UI), čo znižuje riziko zneužitia v prípade kompromitácie zariadenia.

G. Anti-replay: sekvencie a ReplayWindow

1) *Prečo je replay útok nebezpečný*: Replay útok je praktický najmä tam, kde je výsledok udalosti deterministický (napr. *ALLOW*) a útočník vie zachytiť komunikáciu. Bez anti-replay by stačilo raz zachytiť platnú požiadavku a opakovať ju.

2) *Monotónne hw_seq*: Zariadenie používa monotónne sekvenčné číslo *hw_seq*, ktoré sa inkrementuje pri každom skene a perzistentne ukladá do NVS. V prípade reštartu zariadenia sa hodnota obnoví. Monotónnosť je dôležitá: ak by sa *hw_seq* resetoval, útočník by mohol znovu použiť staré hodnoty.

3) *Sliding window na serveri*: Server používa mechanizmus *sliding window* (ReplayWindow). Udržiava: (1) *maxSeen* a (2) bitset použitia hodnôt v rozsahu [*maxSeen* - *window*, *maxSeen*].

Ak príde *seq* príliš staré, server ho odmietne ako *too old*. Ak spadá do okna, ale už bolo použité, odmietne ho ako *replay*. Tento model je bežný v protokoloch, ktoré musia tolerovať out-of-order doručenie, ale zároveň odmietnuť duplicitné správy.

H. Freshness a časová pečiatka

V embedded zariadení nie je zaručená presnosť času. Preto server generuje *ts_unix_ms* a používa ho v hlavičke frame. Čas slúži pre audit, pre detekciu anomálií a pre *freshness* logiky (napr. maximálne povolené oneskorenie správy).

I. Poznámka k dôvernosti komunikácie

V prototypovej implementácii je HW požiadavka autentifikovaná (integrita a pôvod), ale dôvernosť prenosu UID na úrovni HTTP nie je v základnom režime kryptograficky chránená. V praxi je vhodné použiť TLS (HTTPS) alebo poslať už šifrovaný payload z hardvéru. V tejto práci sa dôvernosť posilňuje najmä na úrovni úložiska (neukladanie surového UID) a na úrovni interného protokolu (AEAD frame), pričom absencia povinného TLS je vedomé zjednodušenie prototypu a nie deklarovaná bezpečnostná vlastnosť.

X. BEZPEČNOSŤ VSTAVANÉHO ZARIADENIA A SPRACOVANIE TAJOMSTIEV

Čítačka je z hľadiska bezpečnosti nepohodlný prvok. Nie je pod takou kontrolou ako server, býva fyzicky dostupná a zároveň nemá veľa výkonu ani priestoru na zložitejšie protiopatrenia. Pri návrhu preto nepomáha tváriť sa, že ide o plnohodnotne dôveryhodný uzol. Užitočnejšie je počítať s tým, že zariadenie môže byť slabším článkom a že server musí jeho správy vždy znovu overiť.

Model hrozieb preto zahŕňa situácie, v ktorých útočník získa:

- krátkodobý fyzický prístup (napr. otvorenie krytu, pripojenie k rozhraniam),
- možnosť resetovať napájanie alebo opakovane reštartovať zariadenie,
- možnosť odoberať firmware z pamäte (v závislosti od ochrany),
- možnosť ovplyvniť prostredie (Wi-Fi, rušenie, pokusy o MITM).

V praxi to znamená, že zariadenie sa nemá považovať za „dôveryhodné“ v rovnakom zmysle ako server. Zariadenie je skôr klient, ktorého správy sa musia overiť a limitovať.

A. Ukladanie tajomstiev na zariadení

Najväčším problémom pri jednoduchých schémach je dlhodobé uloženie tajomstva (napr. PSK pre HMAC). Ak útočník tajomstvo získa, vie zariadenie plnohodnotne emulovať. Preto sa v teórii odporúča:

- používať per-device tajomstvá (každé zariadenie má vlastný kľúč),
- oddeliť tajomstvá podľa účelu (autentifikácia, šifrovanie, audit),
- a uvažovať o hardvérových mechanizmoch (secure element, flash encryption).

V prostredí ESP32 existujú mechanizmy ako *Secure Boot* a *Flash Encryption*, ktoré dokážu sťažiť extrakciu firmware a tajomstiev. Pre prototyp nie je cieľom implementovať kompletný hardening, ale v teoretickej rovine je dôležité tieto možnosti poznať a zväžiť ich v produkčnom návrhu.

B. Monotónne počítadlá a trvácnosť stavu

Anti-replay mechanizmy často pracujú so sekvenčným číslom alebo časom. Aby mali význam, musia byť trvácne aj po reštarte. Z toho vyplývajú dve praktické otázky:

- Ako ukladať stav tak, aby sa pri výpadku napájania nestratil?
- Ako zabrániť rollbacku stavu (útočník obnoví starú hodnotu)?

Jednoduché perzistentné ukladanie (napr. NVS) rieši prvú otázku. Druhá otázka je zložitejšia a v produkcii sa rieši napríklad cez secure storage, monotónne čítače v hardvéri alebo serverové pravidlá, ktoré odmietnu nečakaný pokles sekvencie.

C. Bezpečnosť firmware a aktualizácie

Pri mikrokontroléri býva firmware zároveň funkciou aj bezpečnostnou hranicou. Keď sa v ňom objaví chyba, nestačí povedať, že ju „niekedy opravíme“; bez rozumného mechanizmu aktualizácie zostane slabé miesto v zariadení prakticky po celý jeho životný cyklus. Na druhej strane, ak sa aktualizácie navrhnu zle, stanú sa najkratšou cestou k prevzatiu zariadenia.

Preto sa v odbornej literatúre opakuje niekoľko rovnakých zásad: nový firmware má byť podpísaný, zariadenie má vedieť odmietnuť staršiu zraniteľnú verziu a vývojové rozhrania nemajú zostať otvorené v nasadení. V tejto práci nie je implementovaný plnohodnotný secure boot ani OTA aktualizácia; bolo by nepresné tvrdiť opak. Z pohľadu témy je však dôležité pomenovať, že práve firmware rozhoduje o tom, či sa zachová správne počítadlo `hw_seq`, či sa tajomstvo použije iba na podpis a či zariadenie neposiela na sieť viac údajov, než má.

D. Fyzické útoky a praktický hardening

Keď je čítačka pri dverách fyzicky dostupná, model hrozieb sa mení. Útočník už nemusí útočiť cez sieť; môže skúšať debug port, napájanie, vnútornú zbernicu alebo jednoducho vymeniť modul za vlastný. V laboratórnom prototypu sa proti takémuto súperovi nedá dosiahnuť rovnaká odolnosť ako v komerčnom kontroléri so secure elementom a chráneným kryptom. Dáva preto zmysel hovoriť o primeranom hardeningu, nie o úplnej fyzickej odolnosti.

Prakticky to znamená aspoň tri veci. Po prvé, vypnúť alebo obmedziť rozhrania, ktoré boli užitočné pri vývoji, ale v prevádzke by iba rozširovali útočnú plochu. Po druhé, premýšľať nad tým, čo sa stane po kompromitácii jednej čítačky; v dobre navrhnutom systéme to nesmie automaticky znamenať kompromitáciu celej serverovej časti. A po tretie, pri interpretácii výsledkov práce otvorene priznať, že fyzická ochrana zariadenia tu nie je hlavným predmetom riešenia.

E. Bezpečnosť Wi-Fi konfigurácie a lokálnej siete

Wi-Fi býva v prototypoch často najpraktickejšia voľba, ale zároveň prináša typický problém: zariadenie sa ocitne v sieti, kde nie je samo. Vedľa neho môžu byť telefóny, notebooky aj iné IoT prvky, ktoré nemusia byť dôveryhodné. Práve preto nestačí pozerieť sa iba na obsah správy; dôležité je aj to, z akej siete zariadenie komunikuje a s kým vôbec smie hovoriť.

Pre tento typ riešenia je rozumné oddeliť zariadenia aspoň do samostatnej siete alebo VLAN, zúžiť komunikáciu na nevyhnutné endpointy a nenechať otvorené všeobecné služby, ktoré prototyp nepotrebuje. Silné Wi-Fi zabezpečenie a monitorovanie neúspešných spojení sú už skôr prevádzkové disciplíny, ale v praxi často rozhodnú o tom, či incident zostane lokálny, alebo sa rozšíri ďalej. Sieťová segmentácia sama osebe kryptografiu nenahrádza; iba kupuje čas a znižuje dosah problému.

F. Bezpečnosť logovania na zariadení

Pri embedded zariadení je lákavé logovať všetko, najmä počas ladenia. Presne tu však vzniká bežná chyba: do konzoly sa dostanú informácie, ktoré tam po nasadení nemajú čo robiť. V tomto prototype majú logy slúžiť hlavne na diagnostiku čítačky a komunikácie, nie na prenos citlivých údajov. Záznamy preto nemajú obsahovať tajomstvá, celé podpisy ani zbytočne detailné vnútorné stavy.

Rovnako dôležité je, aby sa zo servisného logovania nestal trvalý režim prevádzky. To, čo pomáha pri vývoji, môže po nasadení fungovať ako jednoduchý únikový kanál. Aj pri tak malej súčasti systému sa teda ukazuje rovnaký princíp ako inde: diagnostika je potrebná, ale musí byť podriadená bezpečnostnému cieľu, nie naopak.

XI. BEZPEČNOSŤ TRANSPORTNEJ VRSTVY A POUŽITIE TLS

Pri komunikácii medzi čítačkou a serverom sa ľahko zmiešajú dve odlišné vrstvy ochrany. Jedna otázka znie, či je dôveryhodná samotná požiadavka — teda či server vie, kto ju vytvoril a či ju nikto po ceste neupravil. Druhá otázka sa týka celého prenosu po sieti: či niekto nevie čítať alebo meniť komunikáciu na transportnej vrstve. HMAC a AEAD riešia prvý problém na úrovni správy; TLS rieši druhý na úrovni spojenia. Obe vrstvy sú užitočné, ale ich úloha nie je zameniteľná.[23], [27]

A. Prečo nestačí iba TLS

Samotné TLS neodpovie na všetko. Ak je zariadenie kompromitované, vie stále vytvárať úplne korektné HTTPS požiadavky. Ak je chyba v serverovej autorizácii, TLS ju nezakryje. A ak je v architektúre prítomná proxy alebo reverzný front-end, transportná ochrana sa môže ukončiť ešte pred aplikáciou, ktorá rozhoduje o prístupe.

Preto dáva zmysel ponechať si aj aplikačnú autentifikáciu. V tomto prototype túto úlohu plní HMAC nad telom požiadavky a väzba na `hw_seq`. Aj keby sa transportná vrstva neskôr zmenila, server stále vie overiť, že prijatá udalosť zodpovedá tomu, čo odoslalo konkrétne zariadenie.

B. mTLS vs. symetrické tajomstvá

Pri autentifikácii zariadení sa v praxi stretávajú dve cesty. Prvá je postavená na symetrickom tajomstve, z ktorého sa počíta HMAC; druhá používa certifikáty a mutual TLS. mTLS sa lepšie hodí do väčších infraštruktúr, kde treba riešiť životný cyklus identity zariadenia, revokáciu a správu certifikátov. Má však aj cenu: PKI, distribúcia dôvery a komplikovanejšie nasadenie na strane klienta.

Pre tento prototyp bol zvolený jednoduchší model so zdieľaným tajomstvom. Nie preto, že by mTLS nebolo bezpečné, ale preto, že cieľom práce nebolo stavať certifikačnú infraštruktúru. HMAC stačí na demonštráciu autenticity zariadenia, integrity požiadavky a ochrany proti replay na aplikačnej vrstve.

C. Validácia certifikátu a certificate pinning

Ak sa TLS použije, najslabším miestom býva paradoxne klient. V embedded praxi sa stále objavujú implementácie, ktoré certifikát servera neoverujú dôsledne, alebo validáciu vypnú úplne, lebo je „jednoduchšie rozbehať spojenie“. V takom prípade ostane z TLS iba šifrovaný tunel bez skutočnej identity servera.

Pre stabilné prostredia prichádza do úvahy aj *certificate pinning*, teda uloženie očakávaného verejného kľúča alebo fingerprintu. Tým sa znižuje závislosť od všeobecného trust store, ale zároveň sa komplikuje rotácia certifikátu. Je to typický príklad mechanizmu, ktorý vyzerá čisto bezpečnostne, no v praxi sa rýchlo zmení na prevádzkový problém, ak sa zle naplánuje.[27]

D. Praktické nasadenie TLS v IoT

V teórii sa TLS často spomína ako samozrejímá voľba. V malom IoT zariadení však za tým nasledujú konkrétne otázky: kde bude uložený CA certifikát, ako sa vyrieši jeho obmena, čo sa stane po expirácii a kto zabezpečí aktualizáciu zariadenia bez výpadku. Práve tu sa ukazuje rozdiel medzi „bezpečným mechanizmom“ a „udržateľným nasadením“.

Z tohto dôvodu ostáva v prototype základná komunikácia hardvéru riešená ako HTTP + HMAC. Neznamená to, že TLS nie je dôležité; znamená to iba to, že ťažisko práce je inde. Produkčné riešenie by malo pridať TLS alebo mTLS aj kvôli dôvernosti prenášaného identifikátora.

E. Kombinácia vrstiev: kedy dáva zmysel HMAC aj pri TLS

Otázka „prečo HMAC, keď už mám TLS?“ sa objavuje prirodzene. Odpoveď závisí od toho, čo presne chceme overiť. TLS chráni spojenie medzi dvoma bodmi. HMAC v tomto návrhu viaže konkrétnu požiadavku na zariadenie a na jej obsah. To je užitočné napríklad vtedy, keď sa transportná vrstva v budúcnosti presunie na proxy, load balancer alebo iný medzičlánok.

Inými slovami: TLS by vylepšilo dôveryhodnosť a transportnú integritu, no HMAC zostáva nositeľom aplikačnej identity zariadenia. Tieto vrstvy sa preto nevylučujú; každá rieši inú časť problému.

XII. BEZPEČNOSŤ ÚLOŽISKA, EXPORTOV A ŽIVOTNÝ CYKLUS DÁT

Úložisko prístupového systému má dve hlavné úlohy: udržiava konfiguráciu (karty, roly, väzby) a uchováva audit udalostí. Z toho vyplýva, že únik alebo manipulácia s úložiskom má vysoký dopad.

A. Minimizácia dát a pseudonymizácia

Princíp minimalizácie hovorí, že systém má ukladať iba to, čo potrebuje. Ukladanie HMAC(card_id) namiesto surového UID je príkladom pseudonymizácie: záznamy nie sú priamo čitateľné bez tajomstva (pepper). To znižuje dopad úniku databázy a zároveň umožňuje systém používať (vyhl'adávanie karty podľa UID prebehne tak, že sa UID prepočíta na hash).

B. Export/import a integritné kontroly

Pri administrácii sa často objaví potreba exportovať databázu alebo migrovať konfiguráciu. Export/import je však rizikový: môže obísť bežné API kontroly a v prípade neovereného importu viesť k injekcii neplatných pravidiel. Z teoretického hľadiska je preto vhodné:

- vykonať integrity check databázy po importe,
- overifikovať auditnú reťaz, ak je súčasťou exportu,
- a auditovať samotné administrátorské operácie export/import.

C. Prístupové práva k databáze a zálohovanie

Databáza je často obyčajný súbor (pri SQLite). Z bezpečnostného hľadiska to znamená, že prístupové práva na úrovni operačného systému sú kritické: proces servera má mať práva iba na čítanie/zápis, ktoré potrebuje, a administrátorské exporty majú byť auditované. Pri zálohovaní treba myslieť na to, že záloha obsahuje rovnaké citlivé dáta ako primárna databáza.

V produkcii je vhodné uvažovať aj o šifrovaní úložiska alebo o databáze s podporou šifrovania. Šifrovanie v pokoji (at-rest) však nenahrádza auditnú integritu: útočník s právami procesu môže stále dáta meniť, preto je tamper-evident logovanie doplnkovým mechanizmom.

D. Retencia a ochrana súkromia v audite

Audit záznamy sú z pohľadu bezpečnosti užitočné, ale môžu obsahovať citlivé informácie o pohybe používateľov. V produkcii je preto potrebné definovať:

- obdobie uchovávanía (retenciu),
- prístupové práva k auditu,
- a prípadné anonymizačné alebo agregované reporty.

Tieto aspekty sú najmä procesné, ale priamo súvisia s dôveryhodnosťou systému a s právnymi požiadavkami.

XIII. OCHRANA SÚKROMIA IDENTIFIKÁTOROV A BEZPEČNOSŤ ÚLOŽISKA

Ochrana identifikátorov kariet a integrity databázy sú kľúčové aspekty, ktoré priamo ovplyvňujú dôveryhodnosť systému aj pri úniku dát. Táto kapitola analyzuje riziká spojené s ukladáním surových identifikátorov a opisuje mechanizmy pseudonymizácie implementované v prototypu.

A. Prečo neukladať surový UID

UID je identifikátor, ktorý môže byť v čase stabilný a unikátny. Pri úniku databázy by bolo možné:

- získať zoznam platných identifikátorov,
- korelovať pohyb používateľov podľa auditných záznamov,
- v niektorých prípadoch použiť UID na emuláciu/klonovanie.

Preto prototyp ukladá iba odvodenú hodnotu (HMAC hash).

B. HMAC(card_id) s pepper

Systém ukladá do databázy hodnotu:

$$\text{card_hash} = \text{HMAC}(\text{pepper}, \text{card_id}) \quad (2)$$

kde pepper je tajomstvo uložené mimo databázy. Ak útočník získa SQLite súbor, bez pepper nevie efektívne zistiť pôvodné card_id. Tento prístup je analogický s odporúčaním „oddeliť tajomstvo od databázy“.

C. Pepper vs. salt a odolnosť voči slovníkovým útokom

Pri ochrane identifikátorov sa často spomína salt. Salt je náhodná hodnota, ktorá sa ukladá spolu s hashom a bráni tomu, aby rovnaký vstup mal rovnaký hash. V prístupových systémoch však často chceme, aby systém vedel vyhľadávať kartu podľa UID (t. j. aby rovnaký UID viedol na rovnaký card_hash). Preto sa v tejto práci používa pepper — tajomstvo, ktoré sa do databázy neukladá.

Pepper chráni proti tomu, aby útočník po úniku databázy spustil efektívny slovníkový útok nad priestorom UID. Pri niektorých typoch kariet môže byť priestor UID relatívne malý alebo štruktúrovaný; práve preto je dôležité, aby útočník nemal prístup k tajomstvu, ktoré hashovanie „posilňuje“. Tento prístup je analogický k bezpečnému ukladaniu hesiel, s tým rozdielom, že tu nejde o heslo používateľa, ale o identifikátor.

D. Minimalizácia údajov v audite a súkromnostné riziká

Aj keď sa surové UID do databázy neukladá, audit môže stále obsahovať informácie, ktoré umožňujú korelovať správanie používateľov (čas, miesto, frekvencia prístupov). Z toho vyplýva, že prístup k auditu má byť obmedzený a v produkcii je vhodné zaviesť role-based prístup k auditným záznamom, pravidlá retencie a v reportoch preferovať agregované údaje pred detailnými trasami.

E. Poznámka k právnym aspektom (GDPR)

V európskom prostredí sa pri spracovaní identifikátorov a auditných záznamov často uplatňuje GDPR. Aj keď prototyp nie je produkčný systém, teoreticky je dôležité, aby návrh rešpektoval princípy minimalizácie, účelového viazania a primeranej doby uchovávanía. Hashovanie UID s pepper znižuje riziko pri úniku databázy, ale auditné záznamy stále môžu predstavovať osobné údaje v závislosti od kontextu organizácie. Preto je vhodné v praxi doplniť interné pravidlá, kto má prístup k auditu a na aký účel.

F. Integrita databázy a hranice SQLite

SQLite[28] poskytuje transakcie, obmedzenia a integrity check, ale nepodpisuje obsah databázy a sama osebe nerieši ani distribuovanú dostupnosť či správu prístupov vo väčšej infraštruktúre. Preto pri lokálnej manipulácii súboru nie je možné zaručiť integritu bez dodatočných mechanizmov. V prototypu je táto slabina čiastočne kompenzovaná tamper-evident audit logom, pričom v práci sa otvorene uvádza, že ide o lokálne úložisko vhodné pre experimentálnu a validačnú fázu.

XIV. TAMPER-EVIDENT AUDIT LOG

Audit v prístupovom systéme nemá slúžiť iba na to, aby si administrátor vedel spätne pozrieť, kto prišiel ku dverám. Má byť aj zdrojom dôkazu po incidente. To mení požiadavky na jeho návrh. Nestačí, že sa udalosti zapisujú; dôležité je, aby sa dalo neskôr rozpoznať, či niekto históriu nemenil alebo neorezal. Výskum v oblasti bezpečného logovania preto zdôrazňuje detekciu manipulácie a, pri silnejších schémach, aj forward integrity[5], [8].

A. Základné typy útokov na logy

- Modifikácia záznamu: zmena výsledku ALLOW/DENY alebo času.
- Vloženie záznamu: pridanie falošnej udalosti.
- Vymazanie (truncation): odstránenie časti logu, typicky konca.
- Reorder: zmena poradia udalostí.

B. Hash chain s HMAC

Prototyp používa jednoduchú, ale účinnú konštrukciu: každý záznam obsahuje HMAC hash predošlého záznamu. Zjednodušene:

$$H_i = \text{HMAC}(K_{\text{audit}}, \text{encode}(\text{record}_i) \parallel H_{i-1}) \quad (3)$$

kde K_{audit} je tajný auditný kľúč. Vďaka tomu zmena ktoréhokoľvek záznamu spôsobí nekonzistenciu v následných hodnotách.

C. Audit anchor a ochrana proti truncation

Hash chain sama o sebe nezabráni útoku, pri ktorom útočník odstráni posledných n záznamov a ponechá konzistentný prefix. Preto prototyp používa audit anchor: periodicky sa ukladá hodnota H_i a index i do samostatnej štruktúry. Pri verifikácii sa kontroluje, že audit log obsahuje všetky záznamy minimálne po posledný anchor.

D. Forward integrity a správa auditného kľúča

Výskum bezpečného logovania používa pojem *forward integrity*: aj keď útočník v čase t získa aktuálny kľúč, nemal by byť schopný spätne upraviť staršie záznamy bez detekcie. Jednoduchý hash chain s fixným kľúčom poskytuje detekciu modifikácie, ale pri kompromitácii kľúča útočník môže vytvoriť alternatívnu históriu od miesta kompromitácie. Silnejšie konštrukcie preto používajú evolúciu kľúča (kľúč sa po každom zázname alebo po epochách aktualizuje jednosmerným spôsobom).[5]

V prototypovej práci je cieľom demonštrovať tamper-evident vlastnosti a detekciu manipulačných útokov. Pre produkčný systém by bolo vhodné zvážiť aj forward-integrity varianty a pravidlá rotácie auditného kľúča.

E. Uloženie anchorov a dôveryhodný referenčný bod

Anchor zvyšuje odolnosť proti truncation, ale vytvára otázku: *kde anchor uložiť tak, aby ho útočník nevedel jednoducho prepisovať?* Teoreticky existujú možnosti: uložiť anchor mimo hlavnej databázy, replikovať anchor do externého systému alebo periodicky exportovať anchor ako „checkpoint“ mimo dosahu bežného procesu.

F. Limity a alternatívy: Merkle stromy a verifikovateľné výňatky

Hash chain je jednoduchý a efektívny, ale verifikácia typicky vyžaduje prejsť celý log (alebo aspoň od anchoru). Pre veľké objemy dát sa používajú aj Merkle stromy, ktoré umožňujú verifikovať vybrané výňatky bez nutnosti spracovať celé dáta. Tieto prístupy sú zložitejšie na implementáciu, ale môžu byť vhodné, ak audit rastie do veľkých rozmerov alebo ak sa vyžaduje časté overovanie.

G. Verifikácia a nástroj `audit_verify`

Verifikácia prebieha offline alebo cez admin panel:

- načítanie záznamov a rekonštrukcia reťaze,
- kontrola konzistencie H_i ,
- kontrola anchor proti truncation,
- reportovanie prvého indexu, kde reťaz zlyhá.

XV. ODPORÚČANIA A ŠTANDARDY RELEVANTNÉ PRE NÁVRH

Pri návrhu bezpečnostných mechanizmov je dôležité opierať sa o etablované odporúčania a štandardy. Táto kapitola sumarizuje hlavné zdroje odporúčaní relevantné pre tento prototyp a popisuje, ako sú zohľadnené v návrhu.

A. Kryptografické a protokolárne štandardy

Kryptografická vrstva prototypu stojí na štandardizovaných primitívach. HMAC-SHA-256 je špecifikovaný vo FIPS 198-1[1] a RFC 2104[20]; HKDF je definovaný v RFC 5869[6]; ChaCha20-Poly1305 a XChaCha20-Poly1305 vychádzajú z RFC 8439[2] a Internet-Draftu XChaCha[3]. Voľba štandardizovaných algoritmov namiesto vlastných konštrukcií je základnou bezpečnostnou zásadou: implementácie štandardizovaných primitív sú externe auditované, výskumná komunita ich aktívne preveruje a aktualizácie sa riadia jasným štandardizačným procesom.

Pre komunikačný protokol je kľúčovým referenčným dokumentom RFC 5116[22], ktorý definuje všeobecné rozhranie pre AEAD konštrukcie vrátane ich sémantiky bezpečnosti. Použitie konštrukcie sú s týmto rozhraním kompatibilné, čo umožňuje migráciu na iný algoritmus bez zmeny protokolárnej logiky. TLS 1.3 (RFC 8446)[23] a odporúčania pre jeho bezpečné nasadenie (RFC 9325)[27] sú referenčnými dokumentmi pre transportnú vrstvu; hoci prototyp nepoužíva TLS v základnom režime, návrh je pripravený na jeho doplnenie.

B. Bezpečnostné odporúčania pre systémy

NIST SP 800-57[26] definuje odporúčaný životný cyklus kryptografických kľúčov vrátane oddelenia kľúčov podľa účelu, verzovania a plánovanej rotácie. V prototypu je tento princíp realizovaný cez HKDF deriváciu s domain separation: pre AEAD šifrovanie, nonce generovanie, pepper identifikátorov a audit HMAC existujú nezávislé podkľúče s explicitnými informáciami.

NIST SP 800-98[16] sa priamo venuje bezpečnosti RFID systémov a odporúča kombináciu vrstiev ochrany: autentifikáciu zariadení, ochranu komunikácie na aplikačnej vrstve a auditovateľnosť udalostí. Tieto tri aspekty sú jadrom navrhnutého prototypu — HMAC autentifikácia zariadenia, AEAD frame ochrana a tamper-evident audit zodpovedajú odporúčanej stratégii vrstvenia.

NIST SP 800-63B-4[7] pokrýva digitálnu identitu a autentifikáciu. Pre prototyp je relevantný najmä princíp oddelenia autentifikácie zariadenia (HMAC) od autorizácie prístupu (RBAC). NIST SP 800-92[8] definuje odporúčania pre správu bezpečnostných logov vrátane integrity záznamu a ochrany pred modifikáciou — priama motivácia pre tamper-evident audit log v prototypu.

C. Súlad prototypu s odporúčaniami

Cieľom nie je odcitovať štandardy, ale použiť ich ako rámec pre overenie návrhových rozhodnutí. Všetky kľúčové mechanizmy prototypu — oddelenie kľúčov podľa účelu, rotácia, dôsledná validácia vstupov, fail-closed správanie a minimalizácia dôvery v klienta — vychádzajú z princípov etablovaných v uvedených štandardoch. Oblasť, kde prototyp závedome nepokrýva plnú produkčnú úroveň odporúčaní (absencia TLS, lokálna SQLite databáza, experimentálna správa tajomstiev), sú v práci explicitne pomenované ako vedomé obmedzenia s cieľom zachovať experimentálnu reprodukovateľnosť.

XVI. RIADENIE RIZIKA A NÁVRH PROTIOPATRENÍ

Pri hodnotení navrhnutého systému nie je užitočné tváriť sa, že všetky hrozby majú rovnakú váhu. V malej laboratórnej zostave je realistickejšie riešiť replay, podvrhnutie požiadavky alebo úpravu auditu než napríklad veľmi sofistikované fyzické útoky na kartu. Táto kapitola preto neponúka univerzálny katalóg rizík, ale skôr vysvetľuje, prečo boli vybrané práve tie protiopatrenia, ktoré dávajú zmysel pre tento prototyp.

A. Pravdepodobnosť a dopad

Pri hodnotení rizika sa používa matica pravdepodobnosť $\times$ dopad. Napríklad replay útok je často pravdepodobný (stačí odpočúvanie siete) a jeho dopad môže byť vysoký (neoprávnený vstup). Naopak, niektoré fyzické útoky na RFID môžu mať nižšiu pravdepodobnosť v kontrolovanom prostredí, ale stále sa oplatí ich zvážiť.

B. Vrstvené zabezpečenie (defense-in-depth)

Vrstvené zabezpečenie znamená, že systém nestojí na jednom mechanizme. V návrhu sa to prejavuje tak, že HMAC chráni pôvod požiadavky,[1] anti-replay chráni pred opakovaním, AEAD chráni interné správy a integritu metadát[2], [4] a tamper-evident audit log chráni dôveryhodnosť histórie.[5]

C. Bezpečnosť ako proces

Bezpečnosť nie je jednorazová vlastnosť, ale proces: zahŕňa aktualizácie, rotáciu kľúčov, revíziu pravidiel, monitoring a reakciu na incidenty. Teoretická časť preto zdôrazňuje aj prevádzkové aspekty, nielen kryptografické primitíva.

D. Predpoklady a zvyškové riziko

Každý návrh implicitne stojí na predpokladoch. V tejto práci sa typicky predpokladá, že server je lepšie chránený než čítačka a že administrátorské rozhranie je prístupné iba autorizovaným osobám. Aj pri splnení predpokladov však ostáva zvyškové riziko (residual risk), napríklad kompromitácia jednej čítačky alebo dlhodobý DoS. Dôležité je, aby systém v takom prípade zlyhal bezpečne (fail-closed) a poskytol auditné stopy pre analýzu.

E. Komunikačné kompromisy a použiteľnosť

Prístupové systémy musia byť použiteľné: rýchla odozva, nízky počet falošných zamietnutí a jednoduchá správa oprávnení. To vytvára kompromisy. Napríklad príliš úzka anti-replay *window* môže spôsobovať zamietnutia pri nestabilnej sieti; príliš široká *window* zas znižuje bezpečnosť. Teoreticky je vhodné tieto parametre odvíjať od prostredia a mať možnosť ich meniť bez zásahu do protokolu (konfigurácia na serveri).

XVII. BEZPEČNOSŤ ADMINISTRÁTORSKÉHO ROZHRAANIA A PREVÁDZKOVÉ ASPEKTY

Okrem samotného protokolu je v prístupových systémoch kritické aj administrátorské rozhranie a prevádzkové procesy. Administrácia spravuje roly, väzby, karty a často umožňuje export alebo diagnostiku. Z toho vyplýva, že musí byť chránená silnejšie než bežné klienty.

A. Typické webové hrozby: XSS, CSRF a spracovanie vstupu

Administrátorské rozhranie je webová aplikácia a dedí typické webové hrozby. Aj keď ide o interné UI, zraniteľnosti ako XSS (Cross-Site Scripting) alebo CSRF (Cross-Site Request Forgery) môžu viesť k tomu, že útočník vykoná administrátorské operácie bez vedomia používateľa. Z teoretického hľadiska je preto vhodné:

- sanitizovať a escapovať výstupy v UI,
- používať CSRF tokeny pri stavových operáciách,
- validovať vstupy aj v administrátorských endpointoch (nie iba na HW endpointoch).

Pri vývoji je bežné zobrazovať logy a audit v reálnom čase. Práve tu sa často objavia XSS riziká, ak sa zobrazujú nesanitizované reťazce pochádzajúce z vonkajších vstupov.

B. Autentifikácia administrátora a správa relácií

Administrátorské rozhranie by malo používať silnú autentifikáciu (ideálne viacfaktorovú), obmedzenú platnosť tokenov a bezpečné ukladanie relácií. Pri citlivých operáciách je vhodné vyžadovať opätovné overenie alebo rotáciu relácie.

C. Autorizácia operácií a audit administrácie

Operácie typu *pridať kartu*, *zmeniť rolu* alebo *zmeniť väzbu čítačka*→*dvere* majú priamy dopad na bezpečnosť. Preto je vhodné mať oddelené administrátorské roly a logovať administrátorské zmeny do auditnej stopy.

D. Bezpečnostná konfigurácia a minimalizácia „debug“ rozhraní

Debug endpointy a vývojové nastavenia sú častým zdrojom incidentov. Z teoretického hľadiska platí, že musia byť v produkcii vypnuté alebo striktne obmedzené. OWASP API Security Top 10 upozorňuje najmä na *Security Misconfiguration*.^[13]

E. Monitoring a detekcia anomálií

Monitorovanie je doplnkom ku kryptografii: nárast *HMAC fail*, *replay* udalostí alebo zlyhaní verifikácie auditu môže indikovať útok alebo chybu konfigurácie. Dobre navrhnutý systém umožní nielen odmietnuť podozrivú požiadavku, ale aj poskytnúť podklady pre analýzu a zlepšenie politiky.

XVIII. SÚVISIACE PRÍSTUPY A TYPICKÉ ARCHITEKTÚRY SYSTÉMOV KONTROLY PRÍSTUPU

Nie všetky prístupové systémy vyzerajú rovnako a nebolo by správne predstavovať navrhnutý prototyp ako jediný rozumný model. Táto kapitola preto slúži skôr na zasadenie riešenia do kontextu. Ukazuje, kde sa náš prototyp podobá bežným architektúram a kde je naopak zjednodušený práve preto, že ide o akademický a experimentálny systém.

A. Centralizované vs. distribuované rozhodovanie

Pri porovnávaní architektúr je najpraktickejšie položiť si jednoduchú otázku: kde sa prijíma rozhodnutie o otvorení dverí? V centralizovanom modeli sa udalosť pošle na server, ktorý pozná pravidlá, stav systému aj auditný kontext. To zjednodušuje správu oprávnení a revokácií, ale systém sa stáva citlivejší na výpadok siete alebo servera.

Distribuované riešenie posúva časť logiky bližšie k dverám. Výhodou je vyššia autonómia pri výpadku konektivity. Cena za to je menej pohodlná správa politiky, viac lokálnych tajomstiev a väčšia dôležitosť fyzickej bezpečnosti edge zariadenia. V praxi preto mnohé systémy skončia niekde medzi týmito pólmi: rozhodovanie je čiastočne lokálne, ale audit, konfigurácia a správa identít zostávajú centralizované.

B. Online vs. offline režim a bezpečnostné kompromisy

Offline režim pôsobí na prvý pohľad lákavo, pretože dvere „fungujú aj bez siete“. Lenže práve vtedy sa ukáže, čo všetko musí viesť samotné zariadenie: uchovávať lokálne kópie oprávnení, riešiť ich zastaranie a po obnovení spojenia korektne zlúčiť audit. To nie je detail, ale zásadný bezpečnostný a prevádzkový problém.

Online model je prísnejší na dostupnosť infraštruktúry, no zjednodušuje zmeny oprávnení a robí audit okamžitejší a konzistentnejší. Preto je v tejto práci zvolená práve online, centralizovaná cesta. Neznamená to, že offline režim je nesprávny; iba posúva ťažisko problému zo servera na zariadenie a na lokálnu správu dôvery.

C. Integrácia s identitnými systémami

V reálnych organizáciách býva prístupový systém iba jednou časťou širšieho identity ekosystému. Oprávnenia často nevznikajú priamo v ňom, ale prichádzajú z HR alebo z centrálného IAM riešenia. Z bezpečnostného pohľadu je potom rozhodujúce určiť, ktorý systém je „zdroj pravdy“, ako rýchlo sa prenášajú zmeny a kto nesie zodpovednosť za revokáciu práv.

Pre prototyp takáto integrácia nie je kľúčová, ale je dobré ju pomenovať už v teórii. Bez tohto kontextu by sa mohlo zdať, že správa kariet a rolí je izolovaný problém, hoci v praxi je úzko previazaná s personálnymi a prevádzkovými procesmi.

D. Audit a forenzná pripravenosť v bežných riešeniach

Audit býva v marketingových materiáloch takmer samozrejmosťou. Keď sa však objaví incident, ukáže sa rozdiel medzi bežným logovaním a auditom, ktorému sa dá veriť. Na to nestačí, aby systém iba „niečo zapisoval“. Záznamy musia mať jasný pôvod, časový kontext, konzistentný formát a ochranu proti neskoršej úprave.

Práve preto má v tomto prototypu význam tamper-evident prístup. Nie je to náhrada kompletnej forenznej platformy, ale je to praktický krok od obvyčajného logovania k auditu, ktorý sa dá dodatočne verifikovať. V akademickom prototypu je to primeraná hranica: zvyšuje dôveryhodnosť výsledkov bez toho, aby bolo potrebné budovať veľkú SIEM infraštruktúru.

E. Komunikačné protokoly medzi čítačkou a kontrolérom

Nie každá čítačka komunikuje priamo po IP. V mnohých nasadeniach je medzi ňou a serverom ešte kontrolér pri dverách a samotná čítačka používa jednoduchý lokálny protokol. Historicky boli takéto rozhrania navrhnuté najmä na prenos identifikátora, nie na kryptografickú ochranu. Dobrým príkladom je Wiegand, kde problém nevzniká až na úrovni backendu, ale už na samotnom vedení medzi zariadeniami.

Modernejšie protokoly, napríklad OSDP Secure Channel, ukazujú ten istý princíp v bezpečnejšej forme: aj na krátkej lokálnej trase má zmysel riešiť autenticitu, integritu a ochranu proti replay. V tomto zmysle sa komunikácia „čítačka → kontrolér“ veľmi nelíši od komunikácie „zariadenie → server“; mení sa skôr médium a prevádzkový kontext než samotná bezpečnostná logika.

F. Topológia: inteligentná čítačka vs. kontrolér pri dverách

Rozdiel medzi inteligentnou čítačkou a jednoduchou čítačkou s externým kontrolérom je dôležitý hlavne z pohľadu miesta, kde sa nachádzajú citlivé rozhodnutia a tajomstvá. Inteligentná čítačka je architektonicky čistejšia na kabeláž a často aj jednoduchšia na prototypovanie. Zároveň však znamená, že väčšia časť dôvery sa presúva do fyzicky dostupného zariadenia.

Model s kontrolérom pri dverách presúva rozhodovanie do chránenejšej zóny, ale zase vytvára potrebu zabezpečiť ďalší komunikačný úsek. Ani jedna z možností nie je automaticky „správna“; vhodnosť závisí od prostredia, ceny a od toho, aký silný fyzický útok považujeme za realistický.

G. Cloud vs. on-premise a dôvera v infraštruktúru

Aj pri prístupových systémoch sa dnes stretávajú dva prevádzkové svety. Cloud zjednodušuje centralizovanú správu a dostupnosť z viacerých lokalít, no zároveň presúva časť dôvery mimo organizáciu. On-premise dáva väčšiu kontrolu nad auditom a databázou, ale vyžaduje disciplinovanú správu infraštruktúry, záloh a aktualizácií.

Z pohľadu tejto práce je podstatné skôr niečo iné: aby bezpečnostné mechanizmy neboli úplne závislé od konkrétneho umiestnenia servera. Message-level HMAC, vnútorný AEAD frame a tamper-evident audit majú zmysel v oboch modeloch. Rozdiel je hlavne v tom, komu a do akej miery organizácia dôveruje na úrovni prevádzky.

H. Postavenie prototypu v tomto kontexte

Navrhnutý prototyp je najbližší centralizovanému online systému. Čítačka posielala udalosť na server, server rozhoduje a zároveň vytvára auditnú stopu. Pre potreby diplomovej práce je to výhodné, lebo bezpečnostné mechanizmy zostávajú dobre pozorovateľné: dá sa ukázať podpis požiadavky, anti-replay logika, interné spracovanie frame aj verifikácia auditu.

Ak by sa riešenie posúvalo ďalej, prirodzenými smermi by boli napríklad mTLS, opatrne navrhnutý offline režim alebo silnejšia ochrana fyzického zariadenia. Základná idea by sa však nemenila: oddeliť identitu zariadenia, rozhodovanie o prístupe a dôveryhodný záznam o udalosti.

XIX. KONTROLNÝ ZOZNAM BEZPEČNÉHO NÁVRHU PRÍSTUPOVÉHO SYSTÉMU

Na konci teoretickej časti sa oplatí zhrnúť pravidlá do praktickejšej podoby. Nie ako normu, ktorú treba slepo odškrtnávať, ale ako stručný filter pre návrh. Ak by sa mal podobný systém robiť znovu alebo rozširovať do produkcie, práve tieto body by bolo rozumné skontrolovať ako prvé.

A. Autentifikácia a autorizácia

- Klient (čítačka) musí byť autentifikovaný kryptograficky; identifikátor zariadenia nesmie byť iba „číslo v požiadavke“.
- Politika prístupu má byť oddelená od mechanizmov autentifikácie (neplieť si „kto poslal správu“ s „či má vstúpiť“).
- Pre administráciu platia prísnejšie pravidlá: silná autentifikácia, oddelené roly, audit zmien.

B. Validácia vstupu a robustnosť

- Vstupy validovať syntakticky aj semanticky; mať pevné limity veľkosti.
- Uprednostniť fail-closed správanie a jednotné chybové odpovede smerom k nedôveryhodným klientom.
- Minimalizovať riziká DoS: rýchla validácia pred náročnými operáciami, rate limiting.

C. Kryptografia a manažment kľúčov

- Používať overené primitívy (HMAC, AEAD, HKDF) a vyhnúť sa vlastným konštrukciám.
- Oddeliť kľúče podľa účelu a mať plán rotácie a revokácie.
- Správne pracovať s noncomi a zabrániť ich opakovaniu pod rovnakým kľúčom.

D. Anti-replay a časové údaje

- Anti-replay riešiť explicitne: monotónne sekvencie a serverové okno.
- Minimalizovať dôveru v klientsky čas; časové údaje generovať alebo verifikovať na serveri.

E. Úložisko, súkromie a audit

- Neukladať surové identifikátory, ak to nie je nutné; preferovať pseudonymizáciu (HMAC s pepper).
- Audit považovať za dôkazný artefakt: chrániť ho proti manipulácii (hash chain, anchor).
- Definovať retenciu prístupové práva k auditným záznamom.

F. Prevádzka a monitoring

- Monitorovať anomálie (HMAC fail, replay, zmeny konfigurácie, zlyhanie verifikácie auditu).
- Mať procesy pre incident: blokovanie zariadenia, rotácia kľúčov, analýza auditu.

Takýto kontrolný zoznam pomáha previesť teóriu do konzistentného návrhu. V praxi sa jednotlivé body prispôbujú prostrediu a dostupným zdrojom, no princíp vrstveného zabezpečenia ostáva rovnaký.

XX. ZHRNUTIE TEORETICKÝCH VÝCHODÍSK

Teoretická časť mala pripraviť pôdu pre praktický prototyp, nie vytvoriť dojem „učebnicovo ideálneho“ systému. Preto boli popísané nielen ciele a odporúčané mechanizmy, ale aj hranice zvoleného riešenia. Táto záverečná sekcia sumarizuje kľúčové závery a ich prepojenie s implementáciou.

A. Kľúčové bezpečnostné princípy a ich realizácia

V centre pozornosti stoja tie hrozby, ktoré majú pre prototyp reálny význam: podvrhnutie požiadavky zariadenia, replay, manipulácia s internými metadátami a zásahy do auditnej stopy. Odpoveďou na ne sú HMAC, monotónne sekvencie, AEAD spracovanie interného frame a tamper-evident audit. Každý z týchto mechanizmov rieši konkrétnu bezpečnostnú hrozbu identifikovanú v analýze.

Dôležité je, že tieto mechanizmy sa navzájom dopĺňajú, nie nahrádzajú. HMAC autentifikácia je zbytočná bez anti-replay; anti-replay bez HMAC umožňuje útočníkovi generovať nové požiadavky; AEAD bez AAD nezabezpečuje integritu metadát; a audit bez hash chain nemá dôkazovú hodnotu po incidente. Systém je teda naozaj bezpečný len ako celok.

B. Identifikované medzery a odporúčania pre produkčné nasadenie

Teoretická analýza tiež odhalila niekoľko oblastí, kde prototyp zámerne zjednodušuje produkčnú realitu. Karta vystupuje iba ako identifikátor, nie ako kryptografický dôkaz identity — toto riziko je kompenzované serverovou vrstvou, no v produkčnom systéme by ho mal riešiť silnejší autentifikátor (napr. MIFARE DESFire[18]). Absencia povinného TLS znamená, že dôvernosť UID pri prenose nie je zaručená; odporúčaná cesta je doplniť mTLS[23], [27]. Lokálna správa tajomstiev (hex súbor) je vhodná pre experimenty, no produkčné nasadenie by malo využiť HSM alebo vault[26].

Tieto obmedzenia nie sú náhodné; vyplývajú z rozhodnutia uprednostniť pozorovateľnosť a reprodukovateľnosť experimentov pred produkčnou úplnosťou. Vďaka tomu môže analytická časť precízne merať prínos konkrétnych mechanizmov bez závislosti na externej infraštruktúre.

C. Prepojenie teórie s experimentálnou analýzou

Bezpečnosť prístupového systému nevzniká jedným rozhodnutím alebo jedným algoritmom. Je výsledkom viacerých menších, ale navzájom previazaných volieb: čo sa považuje za dôveryhodné, ktoré údaje sa generujú na serveri, ako sa spravujú kľúče, čo sa zapisuje do databázy a ako sa neskôr overuje história udalostí.

Práve táto previazanosť je ústrednou témou experimentálnej analýzy. Matica šiestich profilov ($A1/A2 \times R0/R1/R2$) variuje iba niekoľko premenných (algoritmus, nonce stratégia, detektor), pričom ostatné vrstvy zostávajú konštantné. Tým je možné presne kvantifikovať, ktoré vrstvy sú zodpovedné za ochranu voči ktorým triedam útokov — a kde nestačí ani tá najdôkladnejšia implementácia bez aktívnej runtime verifikácie.

Nasledujúca tabuľka ukazuje priame prepojenie medzi teoretickými konceptmi a konkrétnymi experimentálnymi dimenziami:

XXI. PRAKTICKÁ IMPLEMENTÁCIA NAVRHNUTÉHO SYSTÉMU

Táto kapitola vychádza z konkrétnej implementácie projektu `access-security`. Kódová báza je rozdelená do samostatných modulov, ktoré riešia kryptografiu, formát správ, rozhodovaciu logiku, úložisko, audit a administrátorské rozhranie. Pre správnu interpretáciu neskorších experimentov a analytického vyhodnotenia je potrebné najprv presne popísať, čo je v prototypu skutočne implementované, kde prebieha rozhodovanie a ktoré časti systému nesú bezpečnostnú zodpovednosť.

Opis implementácie sa člení na dve vrstvy. Prvá vrstva opisuje samotnú implementáciu, teda štruktúru projektu, tok spracovania udalosti a hlavné mechanizmy použité v kóde. Druhá vrstva má analytický charakter: venuje sa testom, meraniam a vyhodnoteniu toho, či implementované opatrenia naozaj zvyšujú odolnosť systému voči zvoleným hrozbám.

Tabuľka 7 sumarizuje, kde v kóde sa realizuje každý z teoreticky opísaných mechanizmov. Táto väzba je zámerná — každé implementačné rozhodnutie vychádza z bezpečnostného princípu diskutovaného v teoretickej časti.

Table 7
Z akého teoretického konceptu vychádza ktorý experiment

Teoretický koncept	Experimentálna dimenzia	Čo experiment meria
Symetrické AEAD šifrovanie (ChaCha20-Poly1305 vs. XChaCha20-Poly1305)	Profily A1 / A2	Ekvivalencia bezpečnostných metrik; rozdiel v priepustnosti a réžii.
Stratégia generovania nonce (náhodný, deterministický, s verifikáciou)	R0 / R1 / R2	Detekčná schopnosť pri zneužití nonce; overhead detektora.
Anti-replay a čerstvosť správy (replay window, monotónne sekvencie)	S1, S4	Účinnosť replay window (S1) a sekvenčného detektora pri rollbacku (S4).
Associated Data (AAD) binding — viazanie kontextu na ciphertext	S2	Odolnosť voči podvrhnutiu AAD (zmena čítačky/dverí pri zachovanom ciphertextu).
Nonce misuse resistance — absencia vs. prítomnosť enforcement-u	S3a, S3b	Chybné R0/R1 (bez verifikácie) vs. R2 (s detekciou odchýlky nonce).
Forgery resistance, protokolová anomálna detekcia, karanténa	S5, R2	Odolnosť AEAD voči úmyselne poškodenému ciphertextu; rýchlosť karantény.

Table 8
Prepojenie teoretických mechanizmov s praktickou implementáciou

Teoretický princíp	Sekcia teórie	Implementácia (modul)
HMAC autentifikácia zariadenia	§ Krypt. základy, MAC	hw_layer, access_admin
AEAD dôvernosc' a integrita	§ Symetrické šifrovanie	crypto_lib, access_core
AAD väzba kontextu	§ Associated Data	crypto_lib, frame hlavička
HKDF derivácia kľúčov	§ Domain separation	key_manager
Anti-replay sliding window	§ Anti-replay: ReplayWindow	protocol_lib
Deterministický nonce	§ Náhodnosť a nonce	crypto_lib::HmacNonceGenerator
Anomálna detekcia + karanténa	§ Monitoring, defense-in-depth	access_core::ProtocolAnomalyDetector
Pseudonymizácia UID (pepper)	§ Ochrana súkromia	access_storage, key_manager
Tamper-evident audit (hash chain)	§ Audit log	access_storage
RBAC autorizácia	§ Modely kontroly prístupu	access_decision::DecisionEngine

A. Architektúra implementácie

Implementácia je koncipovaná ako modulárny serverovo-riadený prototyp. Na strane zariadenia sa nachádza čítačka postavená na platforme ESP32[29] s modulom RC522, komunikujúca podľa ISO 14443[17]. Jej úloha je úmyselne obmedzená: načíta UID karty, pripojí identifikátor čítačky a dverí, doplní monotónne počítadlo hw_seq a takto vytvorenú požiadavku autentifikuje pomocou HMAC-SHA256. Samotné rozhodovanie o prístupe neprebíha na zariadení, ale na serveri.

Serverová časť je implementovaná v jazyku C++ a pozostáva z HTTP rozhrania, rozhodovacej logiky, perzistentného úložiska a auditnej vrstvy. Dôležité je, že hardvérová požiadavka sa po prijatí na serveri nespracuje „skratkou“. Najprv sa overí jej HMAC podpis, potom sa z nej vytvorí interný bezpečný frame a až ten vstupuje do rovnakého validačného a rozhodovacieho reťazca, aký používa jadro systému. Vďaka tomu nie je hardvérový vstup samostatnou výnimkou, ale súčasťou jednotnej spracovateľskej cesty.

Z hľadiska implementácie je praktické, že server nepracuje iba s jednotlivými triedami izolovane. V module access_admin je vytvorený objekt AppState, ktorý drží konfiguračný stav, inštanciu

Table 9
Hlavné vrstvy implementácie a ich úloha v prototypu

Vrstva	Úloha
Hardvérová vrstva	ESP32 + RC522 načíta UID karty, vytvorí JSON požiadavku, zvýši lokálne počítadlo <code>hw_seq</code> a vypočíta HMAC podpis zasielaný v hlavičke <code>X-HW-Signature</code> .
HTTP a aplikačná vrstva	Modul <code>access_admin</code> poskytuje endpoint <code>/api/hw/uid</code> , administratívne API a webové rozhranie. Vstupy z HTTP vrstvy sú pred ďalším spracovaním validované a mapované na interné služby.
Rozhodovacie jadro	Moduly <code>access_core</code> a <code>access_decision</code> zabezpečujú parsovanie frame, kontrolu replay, verziu kľúča, dešifrovanie payloadu a finálne rozhodnutie „allow/deny“.
Úložisko a audit	Modul <code>access_storage</code> uchováva konfiguráciu čítačiek, rolí a kariet v SQLite a súčasne vedie tamper-evident auditný log s HMAC reťazením a anchor bodom.
Pozorovanie systému	Modul <code>runtime_events</code> zbiera prevádzkové udalosti, ktoré sú zobrazované v administrátorskom rozhraní ako live log.

`KeyManager`, prístup do SQLite, auditný log, rozhodovací engine a kruhový buffer udalostí. Toto rozhodnutie zjednodušuje väzby medzi route handlermi, službami a rozhodovacím jadrom, pretože všetky potrebné komponenty sú zviazané na jednom mieste.

B. Modulárne členenie projektu

Pri čítaní kódu sa ukazuje, že projekt nie je organizovaný podľa typu súborov, ale podľa zodpovedností. Toto členenie je dôležité aj pre text práce, pretože neskoršia analytická časť bude testovať nie „celý program naraz“, ale konkrétne mechanizmy, ktoré sú viazané na jednotlivé moduly.

Table 10
Moduly projektu `access-security` a ich hlavná zodpovednosť

Modul	Zodpovednosť v implementácii
<code>crypto_lib</code>	Implementácia kryptografických primitív: AEAD šifrovanie (ChaCha20-Poly1305 aj XChaCha20-Poly1305), HKDF derivácia, generátory nonce (<code>RandomNonceGenerator</code> , <code>HmacNonceGenerator</code>) a pomocné kryptografické funkcie.
<code>key_manager</code>	Načítanie master kľúča a deterministická derivácia podkľúčov pre AEAD, pepper identifikátorov kariet a auditný HMAC.
<code>protocol_lib</code>	Definícia hlavičky a frame formátu, serializácia a parsovanie správ, plus implementácia <code>ReplayWindow</code> .
<code>access_core</code>	Nízkoúrovňové spracovanie frame: validácia čítačky, väzby reader-door, kontroly replay, verzie kľúča, dešifrovanie a časová kontrola.
<code>access_decision</code>	Rozhodovacia logika po úspešnom dešifrovaní: interpretácia payloadu, hľadanie karty, kontrola role a vytvorenie rozhodnutia.
<code>access_storage</code>	SQLite perzistencia a auditný log s HMAC reťazou; zahŕňa aj nástroj na verifikáciu integrity auditu.
<code>runtime_events</code>	Jednoduchý interný event bus pre live log v administrátorskom rozhraní.
<code>access_admin</code>	HTTP server, REST API, import/export databázy, obsluha hardvérového endpointu a webové rozhranie.
<code>hw_layer</code>	Firmware pre ESP32/RC522, Wi-Fi komunikácia so serverom, HMAC podpis požiadavky a lokálne akustické/svetelné signalizovanie výsledku.

Takéto rozdelenie má pre prácu dve výhody. Po prvé, pri opise implementácie sa dá presne ukázať, v ktorom module sa realizuje konkrétny mechanizmus. Po druhé, v analytickej časti sa dajú testy naviazať na konkrétnu vrstvu: napríklad replay na `protocol_lib` a `access_core`, HMAC autentifikácia na `hw_layer` a `access_admin`, auditná integrita na `access_storage`.

Závislosťová hierarchia modulov (obrázok 3) reflektuje princíp jednosmerného toku závislostí: každý modul závisí iba na moduloch nižšej úrovne a nevytvára spätné závislosti:

C. Externé závislosti

Prototyp využíva päť externých knižníc, ktoré sú zahrnuté v adresári `third_party/` ako zdrojový kód kompilovaný spoločne s projektom. Takýto prístup (vendoring) zaručuje reprodukovateľnosť zostavenia — projekt nie je závislý na systémových balíkoch, ktorých verzia a konfigurácia sa môže líšiť medzi prostrediami. Tabuľka 10 uvádza zoznam závislostí s verziou a účelom.

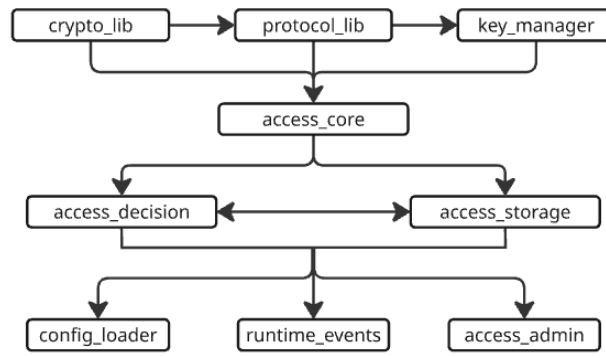


Fig. 3 Závislost'ová hierarchia modulov projektu access-security ($A \rightarrow B$: modul A závisí na B).

Table 11
Externé závislosti projektu

Knižnica	Verzia	Účel v projekte
libsodium[30]	1.0.20	Kryptografické operácie: AEAD šifrovanie (ChaCha20-Poly1305, XChaCha20-Poly1305), HMAC-SHA-256, HKDF derivácia kľúčov, generovanie náhodných čísel, konštantné porovnávanie.
cpp-http[31]	0.30.1	Jednoduchý HTTP server a klient (header-only). Slúži ako základ REST API a administrátorského webového rozhrania.
nlohmann/json[32]	3.11.3	Parsovanie a serializácia JSON správ medzi čítačkou, serverom a webovým rozhraním.
yaml-cpp[33]	0.9.0	Načítanie konfiguračného súboru access_security.yaml s nastaveniami profilov, portov a databázy.
Google Test[34]	1.14.0	Unit a integračné testy. Používa sa pri verifikácii kvality kódu (44 testovacích prípadov).

Z bezpečnostného hľadiska je najkritickejšou závislosťou knižnica **libsodium**, pretože na nej stojí celá kryptografická vrstva systému — a teda aj všetky experimentálne výsledky tejto práce. Nasledujúca sekcia preto podrobne zdôvodňuje, prečo je analýza postavená na tejto knižnici validná.

D. Zdôvodnenie použitia knižnice libsodium

Všetky kryptografické operácie v prototypovom systéme — AEAD šifrovanie, HMAC autentifikácia, derivácia kľúčov, generovanie nonce, porovnávanie tagov — sa vykonávajú prostredníctvom knižnice libsodium[30]. Keďže experimentálna analýza v kapitole XXIII meria vlastnosti týchto operácií (odolnosť voči manipulácii, replay, nonce misuse, výkonnosť), je potrebné zdôvodniť, prečo sú výsledky získané cez libsodium relevantné nielen pre konkrétnu implementáciu, ale aj pre hodnotenie základových kryptografických konštrukcií.

1) *Súlad so štandardmi*: Libsodium implementuje algoritmy presne podľa príslušných štandardov:

- **ChaCha20-Poly1305** (funkcia `crypto_aead_chacha20poly1305_ietf`): implementácia podľa RFC 8439[2], vrátane IETF variantu s 96-bitovým nonce a 32-bitovým počítadlom blokov.
- **XChaCha20-Poly1305** (funkcia `crypto_aead_xchacha20poly1305`): konštrukcia podľa Internet-Draft[3], kde sa 192-bitový nonce redukuje cez medzikrok HChaCha20[21] na 96-bitový vstup pre ChaCha20.
- **HMAC-SHA-256** (funkcia `crypto_auth_hmacsha256`): podľa RFC 2104[20] a FIPS 198-1[1].
- **HKDF** (funkcie `crypto_kdf_hkdf_sha256_extract/expand`): podľa RFC 5869[6], dvojfázová derivácia (extract + expand) s SHA-256 ako základovou hash funkciou.

Súlad so štandardmi znamená, že vlastnosti analyzované v akademickej literatúre (napríklad odolnosť AEAD konštrukcie voči nonce misuse za predpokladu unikátnych nonce[24], alebo

bezpečnosť HKDF pri dostatočnej entropii vstupného materiálu[6]) sa prenášajú na našu implementáciu za predpokladu, že samotná knižnica neobsahuje chyby.

2) *Bezpečnostné auditu a nasadenie*: Libsodium bola podrobená nezávislému bezpečnostnému auditu (Private Internet Access / NCC Group, 2017[35]), ktorý nezistil kritické zraniteľnosti v kryptografických primitívach. Knižnica je súčasťou produkčných systémov s vysokými bezpečnostnými požiadavkami:

- **WireGuard**[36] — VPN protokol integrovaný do jadra Linux od verzie 5.6, používa ChaCha20-Poly1305 ako primárny šifrovací algoritmus.
- **Signal Protocol** — end-to-end šifrovanie v aplikáciách Signal, WhatsApp a iných; využíva Curve25519 a ChaCha20-Poly1305 z libsodium/NaCl.
- **Minisign, age, Keybase** — nástroje na podpisovanie a šifrovanie s miliónmi používateľov.

Široké nasadenie v bezpečnostne kritických aplikáciách poskytuje empirickú evidenciu, že implementácia je robustná a neobsahuje systematické chyby, ktoré by znehodnocovali výsledky postavené na nej.

3) *Ochrana proti bočným kanálom*: Pre experimentálnu analýzu je dôležité, že libsodium implementuje *konštantné* (constant-time) operácie pre všetky kryptografické primitíva. Porovnávanie tagov prebieha cez `sodium_memcmp()`, ktorý je garantovane nezávislý od obsahu vstupov. AEAD operácie neobsahujú dátovo závislé vetvenia ani prístupy do pamäte závislé od tajných kľúčov[30]. To znamená, že namerané časy v scenári S6 (throughput) odrážajú skutočnú výpočtovú náročnosť algoritmov, nie artefakty timing leakov.

4) *Kompilácia zo zdrojového kódu*: V projekte sa libsodium 1.0.20 kompiluje priamo zo zdrojového kódu pomocou CMake, nie ako predkompilovaný binárny balík. Tým je zaručené:

- 1) **Známa a fixovaná verzia** — experimenty sú reprodukovateľné s identickým kódom knižnice.
- 2) **Rovnaké optimalizácie pre oba algoritmy** — kompilátor (GCC/Clang) aplikuje rovnaké optimalizačné flagy (`-O2`) na zdrojové súbory oboch AEAD variantov. Zdrojový kód libsodium navyše obsahuje runtime detekciu SIMD inštrukčných sád (AVX2, SSSE3) a automaticky vyberá najrýchlejšiu implementáciu pre danú platformu[30].
- 3) **Absencia externých patchov** — zdrojový kód je identický s oficiálnym vydaním, čo vylučuje možnosť, že namerané výsledky sú ovplyvnené modifikáciou knižnice.

5) *Záver k validite analýzy*: Na základe vyššie uvedených argumentov možno konštatovať, že výsledky experimentov v tejto práci:

- sú **relevantné pre hodnotenie kryptografických konštrukcií** (ChaCha20-Poly1305, XChaCha20-Poly1305, HMAC-SHA-256, HKDF), pretože libsodium ich implementuje v súlade so štandardmi a bez známych odchýlok,
- sú **relevantné pre hodnotenie architektonických mechanizmov** (anti-replay, anomálna detekcia, karanténa, AAD binding), pretože tieto mechanizmy sú implementované v projektovom kóde a libsodium v nich pôsobí len ako korektná čierna skrinka,
- **nie sú automaticky prenositeľné** na iné implementácie tých istých algoritmov — konkrétne namerané priepustnosti a latencie sú vlastnosťou libsodium 1.0.20 na danej platforme, nie abstraktnou vlastnosťou algoritmov (túto otázku podrobnejšie rozoberáme v kapitole XXIII, Tézis 6).

E. Tok spracovania prístupovej udalosti

Z pohľadu celého systému je najdôležitejší tok jednej prístupovej udalosti. V implementácii má tento tok niekoľko jasne oddelených krokov:

- 1) Čítačka ESP32 načíta UID karty a prevedie ho na textový hexadecimálny tvar.
- 2) Firmware pripojí `reader_id`, `door_id` a monotónny čítač `hw_seq`, ktorý je uložený v `Preferences`, aby prežil reštart zariadenia.
- 3) Zo surového JSON tela sa vytvorí podpis HMAC-SHA256 a požiadavka sa odošle na endpoint `/api/hw/uid`.
- 4) Server v module `access_admin` overí podpis `X-HW-Signature`. Ak podpis neseďí, požiadavka je zamietnutá ešte pred spracovaním biznis logiky.
- 5) Po úspešnom overení sa požiadavka prevedie na interný payload `{"card_id": ..., "action": "open"}` a pomocou `KeyManager` sa získa AEAD kľúč podľa `reader_id` a aktuálnej verzie kľúča.
- 6) Server vytvorí interný frame: zostaví hlavičku `reader_id`, `door_id`, `ts_unix_ms`, `seq`, `key_version` a `nonce`; payload následne zašifruje konfigurovateľným AEAD algoritmom (ChaCha20-Poly1305 alebo XChaCha20-Poly1305 podľa aktívneho profilu).

- 7) `DecisionEngine` odovzdá frame do `FrameHandler`, ktorý vykoná parsovanie, kontrolu väzby reader-door, kontrolu verzie kľúča, anti-replay kontrolu a dešifrovanie payloadu.
- 8) Po úspešnom dešifrovaní sa payload interpretuje ako žiadosť o otvorenie dverí. Server dopočíta HMAC identifikátora karty, nájde zodpovedajúcu rolu a overí, či má táto rola oprávnenie pre konkrétne dvere.
- 9) Výsledok sa uloží do auditnej stopy, odošle sa do `runtime_events` a v jednoduchnej JSON podobe sa vráti späť zariadeniu.

Dôležitý je najmä krok medzi prijatím HTTP požiadavky a volaním `DecisionEngine`. V tejto implementácii zariadenie neposiela hotový šifrovaný frame priamo po sieti. Namiesto toho posiela autentifikovanú požiadavku s `UID` a `hw_seq`; až server z nej vytvorí interný frame, ktorý ďalej spracúva rovnakou cestou ako zvyšok jadra. Z hľadiska práce je to dôležité, pretože sa tým jasne oddeľuje ochrana vstupu zariadenia (HMAC + sekvencia) od ochrany interného spracovania (AEAD + kontrola frame + audit).

F. Poznámka k implementačným rozhodnutiam a k ďalšiemu spracovaniu textu

Pri podrobnejšom čítaní kódu sa ukazuje, že nie všetky bezpečnostné mechanizmy sú koncentrované na jednom mieste. Napríklad anti-replay nie je iba vlastnosťou hardvéru, ale vzniká kombináciou `hw_seq` vo firmvéri, serverovej validácie HMAC požiadavky a následnej kontroly v `ReplayWindow`. Podobne audit nie je len databázová tabuľka, ale súčasť rozhodovacieho toku, pretože `DecisionEngine` zapisuje auditné udalosti priamo pri spracovaní výsledku.

Z tohto dôvodu v ďalšom texte rozoberám praktickú časť podľa mechanizmov: autentifikácia hardvérového zariadenia, interné AEAD spracovanie, derivácia kľúčov, anti-replay, ochrana identifikátorov a auditná integrita. Takto rozdelený opis implementácie vytvára pevnú oporu pre neskoršie testy odolnosti, výkonnosti a detekcie manipulácie.

G. Autentifikácia hardvérovej požiadavky

Prvý kontrolný bod v celom toku spracovania je overenie, či HTTP požiadavka skutočne pochádza z registrovaného zariadenia. Zariadenie posiela na server JSON telo s poľami `uid`, `reader_id`, `door_id` a `hw_seq`. Ako vstup do HMAC-SHA-256 slúži kanonický reťazec zložený z metódy, cesty a surového tela požiadavky:

$$\text{msg} = \text{"POST /api/hw/uid\n"} \parallel \text{raw_body} \quad (4)$$

Výsledný podpis sa prenáša v HTTP hlavičke `X-HW-Signature` ako hexadecimálny reťazec. Na strane servera sa HMAC prepočíta nad rovnakým kanonickým vstupom a porovná s hodnotou z hlavičky. Porovnanie prebieha konštantným časom cez funkciu `sodium_memcmp` z knižnice `libsodium`, čo výrazne znižuje riziko timing útoku, pri ktorom by útočník mohol postupne odhadovať správny podpis podľa dĺžky odpovede [1], [20].

Kanonizácia je v tomto prípade dôležitý detail. Zariadenie aj server musia pracovať s identickým blokom bajtov, pretože aj jediný rozdiel v poradí polí, whitespace alebo kódovaní spôsobí nesúlad podpisu. V prototypovej implementácii je to riešené tak, že sa podpisuje surové telo požiadavky (*raw body*), nie jeho štruktúrovaná JSON reprezentácia. Zariadenie vždy generuje JSON s pevne definovaným poradím polí, takže riziko nesúladu je minimálne.

Táto konštrukcia zabezpečuje dve vlastnosti. Overuje pôvod požiadavky: útočník, ktorý nedisponuje zdieľaným tajomstvom, nevie vyrobiť platný podpis. A overuje integritu obsahu: zmena ľubovoľného poľa v JSON tele (napríklad podvrhnutie iného `uid` alebo `door_id`) spôsobí zlyhanie verifikácie.

Zariadenie pri tom nepoužíva administrátorský token `X-Admin-Token`, ktorý je určený výhradne pre správcovské rozhranie. Oddelenie kanálov autentifikácie vychádza z princípu najmenších oprávnení: aj keby útočník kompromitoval čítačku a získal jej HMAC tajomstvo, nezískal by tým prístup na zmenu rolí, kariet ani väzieb medzi čítačkou a dverami [13].

H. AEAD šifrovanie interného frame

Po úspešnom overení hardvérovej požiadavky server nevyhodnocuje prístup priamo z prijatých dát. Namiesto toho z nich vytvára interný bezpečný frame: binárnu štruktúru, v ktorej je payload zašifrovaný algoritmom AEAD a hlavička je kryptograficky previazaná cez mechanizmus Associated Authenticated Data.

1) *Voľba algoritmu:* V implementácii sa používajú dva AEAD algoritmy z rodiny ChaCha20-Poly1305, oba postavené na rovnakom kryptografickom jadre (ChaCha20 stream cipher + Poly1305 MAC) a oba so 256-bitovou kľúčovou bezpečnosťou:

- **XChaCha20-Poly1305** s 192-bitovým noncom (24 bajtov). Rozšírený nonce je konštruovaný pomocou medzikroku HChaCha20, vďaka čomu je bezpečné používať náhodne generované nonce aj pod dlhodobú žijúci kľúčom: pravdepodobnosť kolízie pri 2^{96} správach je zanedbateľná [3], [4].
- **ChaCha20-Poly1305 IETF** s 96-bitovým noncom (12 bajtov). Štandardizovaný v RFC 8439 a široko nasadzovaný v TLS 1.3 aj v ďalších protokoloch [2]. Kratší nonce znižuje bezpečný limit pre náhodne generované nonce na približne 2^{48} správ.

Implementácia podporuje tri stratégie generovania nonce, označené R0, R1 a R2 (podrobnejšie v nasledujúcej podsekcii). Pri deterministickej konštrukcii (R1, R2) je riziko kolízie pre oba algoritmy identické: nonce sa zopakuje iba vtedy, keď sa zopakuje vstupný kontext, čomu bráni kombinácia anti-replay mechanizmu a unikátneho seq. Oba algoritmy sú preto zahrnuté v implementácii nie kvôli rozdielnej bezpečnosti, ale kvôli možnosti porovnať ich výkon a režijné náklady v experimentálnych profiloch.

Kryptografické operácie sú realizované výhradne cez knižnicu libsodium, konkrétne cez funkcie `crypto_aead_xchacha20poly1305_ietf_*` a `crypto_aead_chacha20poly1305_ietf_*`. Oba algoritmy vychádzajú z rodiny stream cipher primitív navrhnutých D. J. Bernsteinom [21]. Rozhodnutie neimplementovať cipher suites vlastnou cestou je zámerné: libsodium je auditovaná knižnica s konštantnočasovou implementáciou, čo znižuje riziko implementačných chýb tejto triedy [4].

2) *Konštrukcia nonce:* Implementácia definuje rozhranie `INonceGenerator` s jednou metódou `generate(context, seq)`, za ktorým stoja tri konkrétne stratégie:

a) *R0: Náhodný nonce:* Trieda `RandomNonceGenerator` ignoruje kontext aj sekvenčné číslo a pri každom volaní vygeneruje kryptograficky bezpečný náhodný nonce pomocou `randombytes_buf`:

$$\text{nonce}_{R0} = \text{randombytes_buf}(n) \quad n \in \{12, 24\} \quad (5)$$

Bezpečnosť tejto stratégie závisí na kvalite generátora náhodných čísel a na počte správ pod jedným kľúčom. Pre XChaCha20 s 24-bajtovým noncom je riziko kolízie zanedbateľné do 2^{96} správ; pre ChaCha20 s 12-bajtovým noncom klesá bezpečná hranica na približne 2^{48} .

b) *R1/R2: Deterministický HMAC nonce:* Trieda `HmacNonceGenerator` odvodzuje nonce deterministicky z dedikovaného kľúča K_{nonce} a kontextu hlavičky frame. Kľúč sa odvodzuje cez HKDF-SHA-256 z master kľúča:

$$K_{\text{nonce}} = \text{HKDF-SHA-256}(K_{\text{master}}, \text{reader_id} \parallel \text{key_version}, \text{"nonce-derivation-v1"}) \quad (6)$$

Samotný nonce sa počíta ako skrátený výstup HMAC-SHA-256 nad serializovanou hlavičkou (52 bajtov), v ktorej je pole nonce vynulované, aby sa predišlo kruhovej závislosti:

$$\text{nonce}_{R1/R2} = \text{HMAC-SHA-256}(K_{\text{nonce}}, \text{header_context})[:n] \quad n \in \{12, 24\} \quad (7)$$

kde `header_context = reader_id || door_id || ts_unix_ms || seq || key_version || 024` v little-endian formáte.

Deterministická konštrukcia znižuje závislosť na kvalite generátora náhodných čísel [25]. Nonce sa zopakuje iba vtedy, keď sa zopakuje celý kontext, čomu bráni unikátnosť seq garantovaná anti-replay mechanizmom. Navyše, keďže kontext obsahuje aj `door_id` a `ts_unix_ms`, rovnaký nonce nevznikne ani pri rôznych kontextoch s rovnakým sekvenčným číslom.

c) *Rozdiel medzi R1 a R2:* Stratégie R1 a R2 používajú identický `HmacNonceGenerator`. Odlišuje ich to, či server po vytvorení frame overuje, že prijatý nonce zodpovedá deterministickému výpočtu. Overenie vykonáva `ProtocolAnomalyDetector` (sekcia XXI-K), ktorý je aktívny iba v profiloch R2. V profiloch R1 je detektor vypnutý: nonce sa síce generuje deterministicky, ale server akceptuje frame bez verifikácie nonce zhody.

Na drôtovom formáte frame zaberá pole nonce vždy 24 bajtov kvôli uniformnej štruktúre hlavičky. Pri profile A2 (XChaCha20-Poly1305) sa využíva celých 24 bajtov; pri profile A1 (ChaCha20-Poly1305) sa používa prvých 12 bajtov a zvyšných 12 je nulových. Táto konvencia zjednodušuje parsovanie a zaručuje, že veľkosť hlavičky je rovnaká naprieč všetkými profilmi.

3) **AAD väzba hlavičky:** Hlavička frame obsahuje polia `reader_id`, `door_id`, `ts_unix_ms`, `seq`, `key_version` a `nonce`. Tieto metadáta sa nešifrujú, ale vstupujú do AEAD operácie ako AAD. Výsledkom je, že zmena ktoréhokoľvek z nich spôsobí zlyhanie overenia autentifikačného tagu, aj keď šifrovaný payload zostane neporušený[22].

Praktický význam tejto väzby sa ukáže pri konkrétnom útoku. Bez AAD by útočník mohol zachytiť platný frame a podvrhnúť hodnotu `door_id`: payload by bol stále korektne dešifrovaný, no systém by vyhodnotil prístup pre iné dvere. AAD toto znemožňuje, pretože kryptograficky viaže šifrovaný obsah na presný kontext metadát. Ide o realizáciu princípu, že kontext autorizácie musí byť nedeliteľnou súčasťou správy.

Implementácia podporuje aj konfigurovateľný režim `aad_mode`: `"none"`, v ktorom sa AAD nepoužíva a hlavička nie je kryptograficky chránená. Tento režim je v kóde prítomný ako záložná možnosť, no *nie je súčasťou experimentálnej matice*: všetkých šesť testovaných profilov (A1-R0 až A2-R2) používa plnú AAD väzbu (`aad_mode`: `"full"`). Dopad absencie AAD sa demonštruje nepriamo — scenárom S2 (field tampering), kde útočník mení `door_id` v hlavičke a AAD väzba spôsobí zlyhanie autentifikačného tagu.

I. Derivácia kľúčov a domain separation

Celý kryptografický materiál v prototype pochádza z jedného 256-bitového master secret, uloženého v súbore `secrets/master_key.hex` mimo repozitára. Manuálne spravovaný tajný súbor je pragmatický kompromis vhodný pre prototyp; v produkčnom nasadení by jeho úlohu plnil HSM alebo vault[26]. Z tohto tajomstva sa pomocou HKDF-SHA-256[6] deterministicky odvodí tri nezávislé skupiny podkľúčov:

- 1) **AEAD kľúče** pre šifrovanie interného frame. Salt je zložený z `reader_id` (4 bajty, little-endian) a `key_version` (4 bajty), info reťazec je `"access-aead-v1"`.
- 2) **Nonce kľúče** pre deterministickú deriváciu nonce (stratégie R1/R2). Salt je rovnaký ako pri AEAD kľúčoch (`reader_id || key_version`), info reťazec je `"nonce-derivation-v1"`.
- 3) **Pepper** pre hashovanie identifikátorov kariet. Salt obsahuje iba `key_version`, info reťazec je `"card-pepper-v1"`.
- 4) **Auditný HMAC kľúč** pre hash chain. Salt je `"audit-chain-salt-v1"`, info reťazec je `"audit-hmac-v1"`.

Table 12
Prehľad HKDF derivácie kľúčov z master secret

Kľúč	Salt	Info reťazec
AEAD	<code>reader_id key_ver</code>	<code>access-aead-v1</code>
Nonce (K_{nonce})	<code>reader_id key_ver</code>	<code>nonce-derivation-v1</code>
Pepper	<code>key_ver</code>	<code>card-pepper-v1</code>
Auditný HMAC	<code>audit-chain-salt-v1</code>	<code>audit-hmac-v1</code>

Grafické znázornenie tejto derivačnej hierarchie je v obrázku 4.

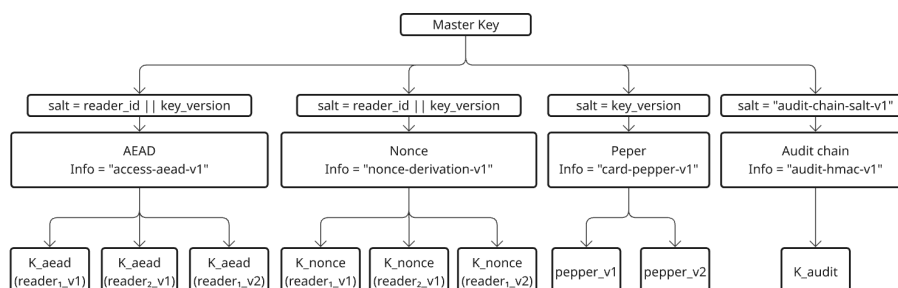


Fig. 4 Derivácia kľúčov z master secret cez HKDF. Každá vetva má vlastný info reťazec (domain separation).

Info reťazce zabezpečujú domain separation[6]: z rovnakého master kľúča nikdy nevznikne rovnaký podkľúč pre dva rôzne účely, ani keby sa hodnoty `reader_id` a `key_version` náhodne prekrývali. Súčasne umožňujú neskoršiu zmenu verzie schémy (napríklad z `access-aead-v1`

na `access-aead-v2`) bez spätnej kompatibility so starými kľúčmi, čo je dôležité pri migračných scenároch.

Z bezpečnostného hľadiska je podstatné, že AEAD kľúč sa odvodzuje per-reader. Kompromitácia jedného zariadenia a extrakcia jeho odvodiťného kľúča neodhaľuje kľúče iných čítačiek, pretože salt HKDF obsahuje unikátnu hodnotu `reader_id`. Izolácia je priamym dôsledkom správnej konštrukcie derivácie a nevyžaduje distribúciu samostatných tajomstiev.

Verzovanie kľúčov v salt zabezpečuje, že pri rotácii (zmene `key_version`) sa odvodí úplne nový AEAD kľúč bez nutnosti meniť master secret. Server v konfigurácii umožňuje režim `allow_previous_key_version`, v ktorom dočasne akceptuje aj bezprostredne predchádzajúcu verziu. Cieľom je hladký prechod pri rotácii: zariadenie nemusí byť aktualizované presne v rovnakom okamihu ako server, čo znižuje riziko krátkodobého výpadku[26].

Implementácia podporuje aj režim `key_derivation_mode: "direct"`, v ktorom sa HKDF nepoužíva a master kľúč slúži priamo ako AEAD kľúč pre všetky čítačky. Podobne ako pri `aad_mode`, tento režim je v kóde prítomný ako záložná možnosť, ale *nie je súčasťou experimentálnej matice*. Všetky profily používajú HKDF deriváciu s per-reader salt, čím je zaručená izolácia kľúčov medzi čítačkami. Experimenty sa zameriavajú na premenné s priamejším dopadom na odolnosť voči útokom: voľbu AEAD algoritmu (A1/A2), stratégiu generovania nonce (R0/R1/R2) a prítomnosť anomálneho detektora.

J. Anti-replay mechanizmus

Anti-replay ochrana funguje na dvoch úrovniach. Na strane zariadenia firmvér udržiava monotónne počítadlo `hw_seq`, uložené v NVS pamäti ESP32, ktoré sa inkrementuje pri každom skene karty a prežíva reštart zariadenia. Každá požiadavka tak nesie unikátnu sekvenčnú hodnotu.

Na strane servera modul `protocol_lib` implementuje štruktúru `ReplayWindow`, ktorá udržiava záznam o naposledy videných sekvenciách pre každú čítačku zvlášť. Mechanizmus rozlišuje tri stavy:

- ak je `seq` menšie než `maxSeen - windowSize`, požiadavka sa odmietne ako príliš stará,
- ak `seq` spadá do okna, ale už bolo zaznamenané, odmietne sa ako *replay*,
- inak sa `seq` akceptuje, zaznamená do množiny videných hodnôt a posunie sa horná hranica okna.

Prečo nestačí jednoduchá podmienka `seq > lastSeen`? V prostredí Wi-Fi nie je zaručené, že dve po sebe nasledujúce požiadavky dorazia na server v rovnakom poradí. Bez okna by druhá požiadavka bola nesprávne odmietnutá, aj keď by bola legitímna. Sliding window toleruje krátke zmeny poradia, pokiaľ správa ešte nebola videná, a tým znižuje podiel falošných odmietnutí pri zachovaní replay ochrany.

Veľkosť okna (`replay_window_size`) je konfigurovateľná a priamo ovplyvňuje kompromis medzi bezpečnosťou a praktickou použiteľnosťou: príliš malé okno zvyšuje falošné odmietnutia, príliš veľké predlžuje čas, počas ktorého je zachytená stará požiadavka potenciálne zopakovateľná. Implementácia udržiava okno pomocou fronty (`std::deque`) a množiny (`std::unordered_set`), pričom pri prekročení kapacity sa najstaršia hodnota automaticky evikvuje.

K. Detektor protokolových anomálií

Nad rámec reaktívneho odmietnutia jednotlivých frame implementácia obsahuje komponent `ProtocolAnomalyDetector`, ktorý sleduje vzory správania na úrovni celej čítačky a pri podozrivej aktivite dokáže čítačku proaktívne umiestniť do karantény. Tento prístup vychádza z princípov detekcie anomálií v sieťovom prostredí[37].

Detektor udržiava per-reader stavový záznam a sleduje štyri typy anomálií:

- 1) **Opakované sekvenčné číslo** (`seq_reuse`): ak frame nesie `seq`, ktoré už bolo zaznamenané. Na rozdiel od `ReplayWindow`, ktorá frame iba odmietne, detektor túto udalosť zaregistruje ako indikátor replay útoku.
- 2) **Rollback sekvenčného čísla** (`seq_rollback`): ak `seq + rollbackThreshold < maxSeenSeq`. Veľký skok späť signalizuje, že útočník sa pokúša obísť sliding window opakovaním starších sekvencií.
- 3) **Nesúlad nonce** (`nonce_mismatch`): ak prijatý nonce nezodpovedá deterministicky odvodenému nonce z kontextu hlavičky. Táto kontrola je aktívna iba v profiloch s `nonceMode = "deterministic"` (R1/R2), no v kóde je podmienená aj zapnutím detektora — preto sa vykonáva iba v profiloch R2.

- 4) **Séria zlyhaní tagu** (*tag_fail_streak*): ak počet po sebe nasledujúcich zlyhaní AEAD autentifikačného tagu dosiahne konfigurovateľný limit. Opakované zlyhania indikujú brute-force pokus o uhádnutie platného ciphertextu.

Pri detekcii anomálie sa čítačka umiestni do karantény: všetky ďalšie frame od tejto čítačky sú zamietnuté s dôvodom *reader_quarantined*, bez ohľadu na ich obsah. Karanténa je proaktívna obrana — izoluje celé zariadenie, nie iba jednotlivý frame. Tým sa odlišuje od anti-replay okna (ktoré zamietá iba konkrétnu sekvenciu) a od zlyhania AEAD dešifrovania (ktoré zamietá iba frame s neplatným tagom).

V experimentálnej matici je detektor zapnutý výhradne v profiloch R2 (A1-R2, A2-R2). Profily R0 a R1 majú detektor vypnutý, čo umožňuje v scenároch S1–S5 presne kvantifikovať jeho prínos: profily R0/R1 vykazujú nenulovú úspešnosť útoku (napr. 11 % v S4), zatiaľ čo R2 dosahuje 0 % vďaka okamžitej karanténe po prvej anomálii.

Table 13
Konfigurácia ProtocolAnomalyDetector a reakcia na anomálie

Typ anomálie	Podmienka detekcie	Reakcia
<i>seq_reuse</i>	$seq \in _seen$	Okamžitá karanténa
<i>seq_rollback</i>	$seq + \delta < \maxSeenSeq$	Okamžitá karanténa
<i>nonce_mismatch</i>	$nonce \neq HMAC(K_n, ctx)[n]$	Okamžitá karanténa
<i>tag_fail_streak</i>	5 po sebe nasledujúcich zlyhaní	Karanténa po 5. zlyhaní

L. Ochrana identifikátorov kariet

Databáza neukladá surové UID kariet — v súlade s princípom minimalizácie dát[38] a ochranou identifikátorov[7]. Pri registrácii aj pri vyhľadávaní sa UID prepočíta na HMAC hash s tajomstvom, ktoré v databáze uložené nie je:

$$card_hmac = HMAC\text{-}SHA\text{-}256(pepper, uid) \quad (8)$$

Pri úniku databázy útočník získa iba výsledné HMAC hodnoty. Bez znalosti pepperu z nich nevie efektívne odvodiť pôvodné UID, a to ani pri relatívne malom priestore identifikátorov (napríklad 4-bajtové UID majú 2^{32} možností, čo by pri slabom hashovaní umožňovalo hrubú silu)[1]. Pepper sa odvodzuje cez HKDF z master kľúča a verzuje podľa *key_version*. Pri rotácii pepperu sa musia existujúce karty prehashovať pod novým tajomstvom.

Rozdiel oproti klasickému salt-based hashov je zámerný. Salt sa ukladá vedľa hashu a jeho účelom je zabrániť predpočítaným tabuľkám (*rainbow tables*); pepper sa naopak nachádza mimo úložiska a chráni aj proti slovníkovému útoku s priamym prístupom k databáze. Na druhej strane, ak útočník kompromituje server a získa pepper, ochrana sa stráca. Pepper preto nie je náhrada za kryptograficky silnú kartu, ale primeraná vrstva ochrany v rámci prototypu.

M. Tamper-evident auditná stopa

Auditný log v prototypu nie je obyčajný záznam udalostí. Každý záznam obsahuje HMAC hash, ktorý kryptograficky viaže jeho obsah na celú predchádzajúcu históriu.

1) *Konštrukcia HMAC hash chain*: Základná konštrukcia vychádza z iteratívnej aplikácie HMAC na každý nový záznam:

$$H_i = HMAC(K_{audit}, H_{i-1} \parallel encode(record_i)) \quad (9)$$

kde K_{audit} je auditný kľúč odvodený cez HKDF z master secret a H_{i-1} je hash predchádzajúceho záznamu. Pre prvý záznam sa používa $H_0 = 0^{32}$. Kanonikalizácia záznamu je deterministická

Reťazec hash hodnôt znamená, že zmena ľubovoľného záznamu spôsobí nekonzistenciu vo všetkých nasledujúcich hodnotách. Útočník, ktorý nemá auditný kľúč, nevie prepočítať H_i od miesta zásahu tak, aby reťazec zostal konzistentný.

2) *Anchor a ochrana proti truncation*: Hash chain sám o sebe nezabráni útoku, pri ktorom útočník odstráni posledných n záznamov a ponechá konzistentný prefix. Preto prototyp udržiava *audit anchor*: po každom zápise sa aktuálna hodnota H_i a timestamp uložia do samostatnej jednoriadkovej tabuľky *audit_anchor*. Pri verifikácii sa kontroluje, že log obsahuje záznamy minimálne po poslednú hodnotu anchor. Ak záznamy chýbajú, ide o detekovateľný truncation[5], [8].

Anchor nie je úplná ochrana v prípade, že útočník má plný zápis do databázy — v takom prípade vie prepísať aj anchor. Silnejšie riešenie by vyžadovalo uloženie anchor mimo databázy (napríklad na vzdialenom serveri alebo v hardvérovom module). V kontexte prototypu je cieľom demonštrovať princíp detekcie manipulácie, nie dosiahnuť odolnosť voči útočníkovi s neobmedzeným prístupom.

Implementácia navyše rozlišuje medzi reťazeným a neret'azeným režimom auditu. Ak je reťazenie vypnuté, záznamy sa ukladajú s nulovými hodnotami `prev_hash` a `entry_hash`, čo zodpovedá jednoduchému logu bez kryptografickej ochrany. Porovnanie oboch režimov v analytickej časti demonštruje presný dopad tejto ochrany.

N. Rozhodovacia logika a model RBAC

Na konci spracovateľského reťazca stojí `DecisionEngine`, ktorý po úspešnom dešifrovaní payloadu vyhodnocuje, či má byť prístup povolený. Rozhodovanie prebieha v niekoľkých krokoch. Server prepočíta HMAC identifikátora karty s aktuálnym pepperom. V tabuľke `cards` vyhľadá kartu podľa výsledného `card_hmac`. Ak ju nájde, prečíta jej rolu; ak nie, výsledok je `deny` s dôvodom `unknown_card`. Nakoniec v tabuľke `door_roles` overí, či rola karty je povolená pre dvere z hlavičky `frame`[11], [12].

Model RBAC v prototypy znamená, že karty nemajú priame oprávnenia na dvere. Sú im priradené roly (napríklad `employee`, `admin`) a dvere majú povolené zoznamy rolí. Pridanie alebo odobratie oprávnenia sa robí na úrovni väzby rola–dvere, nie na úrovni jednotlivých kariet. To zjednodušuje správu pri veľkom počte používateľov a umožňuje rýchlu revokáciu: zrušenie roly v tabuľke `door_roles` okamžite odmietne všetky karty s touto rolou pre dané dvere.

Výsledok rozhodnutia (`allow` alebo `deny`) sa spolu s identifikátorom čítačky, dverí, sekvenčným číslom a dôvodom zapíše do tamper-evident auditnej stopy. Dôvod zamietnutia (`reason`) je interný diagnostický kód (napríklad `replay`, `unknown_card`, `mac_verification_failed`), ktorý sa nezverejňuje nedôveryhodnému klientovi, ale ukladá sa do auditu a zobrazuje v administrátorskom rozhraní. Tým sa zabezpečuje, že diagnostika je k dispozícii prevádzke, no útočník z odpovede nezíska informácie užitočné pre iteráciu útoku.

O. Konfigurácia systému a bezpečnostné profily

Správanie bezpečnostných mechanizmov prototypu je plne konfigurovateľné cez súbor `config/access_security`, ktorý server načítava pri štarte. Konfigurácia rozlišuje tri úrovne parametrov:

Serverové parametre: hosťiteľ, port, maximálna veľkosť payloadu, cesta k databáze, maximálna prípustná časová odchýlka (`max_skew_ms`).

Kryptografické parametre: voľba AEAD algoritmu (`cipher_mode`: `xchacha20poly1305` alebo `chacha20poly1305`), režim generovania nonce (`nonce_mode`: `random` / `deterministic`), režim AAD (`aad_mode`: `full` / `none`), režim derivácie kľúčov (`key_derivation_mode`: `hkdf` / `direct`).

Prevádzkové parametre: veľkosť replay window (`replay_window_size`, predvolene 256), povolenie predchádzajúcej verzie kľúča (`allow_previous_key_version`), stav detektora anomálií (`anomaly_detector_enabled`), prahy detektora (`rollback_threshold`: 100, `tag_fail_streak_limit`: 5).

Table 14
Kľúčové konfiguračné parametre a ich mapovanie na bezpečnostné profily

Parameter	Hodnoty	Experimentálne profily
<code>cipher_mode</code>	<code>chacha20poly1305</code>	A1
	<code>xchacha20poly1305</code>	A2
<code>nonce_mode</code>	<code>random</code>	R0
	<code>deterministic</code>	R1, R2
<code>anomaly_detector_enabled</code>	<code>false</code>	R0, R1
	<code>true</code>	R2
<code>aad_mode</code>	<code>full</code>	všetky 6 profilov
<code>key_derivation_mode</code>	<code>hkdf</code>	všetky 6 profilov
<code>replay_window_size</code>	256	všetky 6 profilov

Toto oddelenie konfigurácie od kódu umožňuje prepínať profily bez rekompilácie a zaručuje, že experimentálny harness aj produkčný server môžu využívať rovnakú implementačnú logiku

— menia sa iba parametre, nie kód. Konfigurácia zároveň obsahuje explicitné záložné hodnoty (`aad_mode: none` a `key_derivation_mode: direct`), ktoré slúžia pre diagnostiku a ladenie, ale nie sú súčasťou experimentálnej matice.

P. Webové administrátorské rozhranie

Modul `access_admin` okrem REST API servíruje aj webové rozhranie dostupné na `/static/`. Rozhranie je implementované v čistom JavaScripte bez externého frameworku a sprostredkúva priamy prístup ku všetkým správcoským operáciám:

- **Správa čítačiek, dverí, rolí a kariet:** CRUD operácie nad databázovými entitami; karty sa registrujú zadaním UID, ktoré server okamžite prepočíta na HMAC hash pred uložením.
- **Väzby reader-door:** priradenie čítačky ku konkrétnym dverám; táto väzba je vynucovaná aj na úrovni kryptografickej AAD hlavičky.
- **Live log:** real-time stream posledných 50 rozhodnutí cez polling endpointu `/api/events`; zobrazuje profil, výsledok, dôvod zamietnutia a latenciu.
- **Auditná verifikácia:** tlačidlo na spustenie offline verifikácie integrity hash chain; výsledok zobrazuje počet záznamov, stav reťaze a prípadnú polohu prvej nekonzistencie.
- **Import/export databázy:** JSON export aktuálneho stavu, JSON import pre migráciu konfigurácie.

Autentifikácia administrátorského rozhrania je realizovaná hlavičkou `X-Admin-Token`, ktorá je striktné oddelená od HMAC autentifikácie zariadení (princíp najmenej oprávnení[15]): kompromitácia čítačky nedáva útočníkovi prístup k správcovskej funkcionalite a naopak.

Q. Prepojenie mechanizmov: prečo vrstvy nestoja samostatne

Popísané mechanizmy nie sú navzájom nezávislé bezpečnostné funkcie, ale navrhnuté tak, aby sa dopĺňali v zmysle princípu *defense-in-depth*[39], [40]. HMAC autentifikácia zariadenia by bola neúčinná bez anti-replay, pretože by stačilo zachytiť jednu platnú požiadavku a opakovať ju. Anti-replay bez HMAC by bol zbytočný, pretože útočník by mohol vyrobiť ľubovoľnú novú požiadavku s nepoužitým seq. AEAD bez AAD by chránilo dôvernosť payloadu, ale nie kontext autorizácie. A audit bez hash chain by bol dôveryhodný iba do momentu, kým by sa útočník nedostal k databáze.

Práve toto vzájomné prepojenie je jedným z hlavných predmetov analytickej časti. Experimentálna matica šiestich profilov (A1-R0 až A2-R2) variuje AEAD algoritmus, stratégiu generovania nonce a prítomnosť anomálneho detektora, pričom ostatné vrstvy (AAD väzba, HKDF derivácia, pepper, anti-replay okno) zostávajú vo všetkých profiloch zapnuté. Šesť scenárov (S1–S6) potom ukazuje, ktorá konkrétna kombinácia vrstiev chráni proti ktorému konkrétnemu útoku — a naopak, kde samotná zmena jednej premennej (napríklad prechod z R1 na R2) dokáže eliminovať celú triedu útokov.

XXII. VERIFIKÁCIA KVALITY KÓDU PRED EXPERIMENTMI

Pred vykonaním akýchkoľvek experimentov je nutné overiť, že implementácia samotná neobsahuje chyby pamäťového prístupu, nedefinované správanie, pretečenia bufferov či úniky pamäte. Ak by takéto defekty v kóde existovali, výsledky experimentov by nemali výpovednú hodnotu — rozdiel v metrikách by mohol byť spôsobený skrytým bugom, nie vlastnosťami konfigurácie. Z tohto dôvodu bola vykonaná dvojstupňová verifikácia: statická analýza zdrojového kódu a dynamická analýza za behu testovacej sady.

A. Statická analýza: clang-tidy

Ako nástroj statickej analýzy bol použitý `clang-tidy`[41] verzie 18.1.3 (Ubuntu LLVM). Nástroj analyzuje zdrojový kód bez jeho spustenia a hľadá vzory, ktoré indikujú potenciálne chyby: prístupy do neplatnej pamäte, implicitné konverzie, narrowing conversions, zbytočné kópie a ďalšie defekty evidované v moduloch `bugprone-*`, `cert-*`, `clang-analyzer-*` a `performance-*`.

Analýza bola spustená nad všetkými 76 zdrojovými súbormi projektu (hlavičkové a implementačné) (moduly `crypto_lib`, `protocol_lib`, `key_manager`, `access_core`, `access_decision`, `access_storage`, `config_loader`, `runtime_events` a `access_admin`) s kompilačnou databázou vygenerovanou pomocou `CMAKE_EXPORT_COMPILE_COMMANDS=ON`. Postup je reprodukovateľný nasledujúcimi príkazmi:

```
# Generovanie compile_commands.json
cmake -B build -DCMAKE_BUILD_TYPE=Debug \
      -DCMAKE_EXPORT_COMPILE_COMMANDS=ON
```

```
cmake --build build -j$(nproc)
```

```
# Spustenie clang-tidy
find crypto_lib protocol_lib key_manager access_core \
    access_decision access_storage config_loader \
    runtime_events access_admin -name '*.cpp' \
    | xargs clang-tidy -p build --quiet
```

Výsledky:

- **Skupiny cert-* a clang-analyzer-*:** 0 nálezov. Tieto skupiny zahŕňajú kontroly relevantné pre bezpečnostnú korektnosť (napr. nesprávnu prácu s certifikátmi, nebezpečné volania, chyby logiky detekované symbolickou analýzou).
- **Bugprone:** 2 nálezy — bugprone-narrowing-conversions (implicitná konverzia `size_t` na `difference_type` v HKDF expanzii) a bugprone-implicit-widening-of-multiplication (celočíselné násobenie v type `int` pred priradením do `size_t`).
- **Performance:** 13 nálezov — 6 prípadov `performance-move-const-arg` (zbytočný `std::move` triviálne kopírovateľných typov), 6 prípadov `performance-unnecessary-value-param` (parameter odovzdávaný hodnotou namiesto konštantnou referenciou), 1 prípad `performance-enum-size` (enum mohol mať menší bazový typ).
- **Modernize:** 1 nálež — chýbajúci `override` na deštruktore `SqliteAccessStore`.

Žiadny z nálezov nepredstavuje bezpečnostnú zraniteľnosť. Dva bugprone nálezy sa týkajú implicitných konverzií, ktoré pri reálnych veľkostiach vstupných dát (frame maximálne 4096 B, maximálne 20 MiB upload) nemôžu spôsobiť pretečenie. Nálezy kategórie performance nemajú vplyv na korektnosť, len na mierne redundantné kópie v nekritických cestách.

B. Dynamická analýza: sanitizers

Pre dynamickú analýzu bol projekt preložený s nasledujúcimi sanitizermi:

- **AddressSanitizer (ASan)**[42]: detekuje prístupy mimo hraníc bufferov (buffer overflow, underflow), použitie po uvoľnení (use-after-free), dvojité uvoľnenie (double-free) a ďalšie chyby práce s pamäťou za behu programu.
- **UndefinedBehaviorSanitizer (UBSan)**: detekuje nedefinované správanie podľa štandardu C++ — pretečenie znamienkových celých čísel, dereferenciu `nullptr`, nesprávne zarovnanie ukazovateľov a ďalšie.
- **LeakSanitizer (LSan)**: na Linuxe sa aktivuje automaticky spolu s ASan; pri ukončení procesu overí, či nezostala neuvoľnená dynamicky alokovaná pamäť.

Všetky tri sanitizery boli aktivované súčasne prepínačmi kompilátora. Preklad a spustenie testov:

```
# Preklad so sanitizermi
cmake -B build-san -DCMAKE_BUILD_TYPE=Debug \
    -DCMAKE_CXX_FLAGS="-fsanitize=address,undefined \
        -fno-omit-frame-pointer -g" \
    -DCMAKE_EXE_LINKER_FLAGS="-fsanitize=address,undefined"
cmake --build build-san -j$(nproc)

# Spustenie testov pod sanitizermi
cd build-san
ASAN_OPTIONS=detect_leaks=1 \
UBSAN_OPTIONS=print_stacktrace=1 \
ctest --output-on-failure
```

Takto preložená verzia bola spustená s kompletnou sadou testov.

Doplňujúca poznámka k rozsahu: MemorySanitizer (MSan), ktorý detekuje čítanie neinicializovanej pamäte, nebol použitý, pretože vyžaduje, aby *všetky* linkované knižnice vrátane `libsodium` a `SQLite` boli preložené s inštrumentáciou MSan. To v praxi znamená preklad celého závislostného stromu zo zdrojov, čo presahuje rámec tejto verifikácie. SanitizerCoverage (meranie pokrytia kódu na úrovni sanitizero) zostáva rezervovaný pre prípadnú fázu fuzz testingu.

C. Testovacia sada a pokrytie

Testovacia sada obsahuje 44 testovacích prípadov (makrá `TEST()` frameworku Google Test[34]) rozdelených do 7 testovacích binárnych súborov:

Table 15
Prehľad testovacej sady

Binárny súbor	Počet	Pokrytý modul / scenáre
crypto_lib_test	6	HKDF determinizmus, AEAD roundtrip, AAD tampering
protocol_lib_test	9	Serializácia frame, anti-replay sliding window
handle_frame_test	2	Orchestrácia FrameHandler, poisoning replay okna
anomaly_detector_test	18	R2 ProtocolAnomalyDetector: quarantine lifecycle, seq_reuse/rollback, nonce_mismatch, tag_fail_streak
engine_test	6	DecisionEngine RBAC, rotácia kľúčov a pepperu
engine_sqlite_test	1	Integrácia SQLite store s DecisionEngine
audit_chain_test	2	Detekcia manipulácie a truncation auditnej stopy
Spolu	44	

Z hľadiska pokrytia sú testy zamerané na bezpečnostne kritickú cestu spracovania: kryptografia (AEAD, HKDF), protokol (serializácia, anti-replay), rozhodovacia logika (RBAC, rotácia), audit (hash chain, anchor) a runtime detekcia anomálií (R2 ProtocolAnomalyDetector — quarantine lifecycle, seq_reuse, seq_rollback, nonce_mismatch, tag_fail_streak). Moduly `config_loader`, `runtime_events` a `access_admin` nemajú vlastné unit testy. Tieto moduly sú I/O obálky (parovanie YAML, HTTP routing, event buffer) a ich kód je čiastočne pokrytý nepriamo cez integračné testy, ktoré prechádzajú celou cestou od konštrukcie frame až po zápis auditu. Pre účely tejto práce je podstatné, že sanitizery pokrývajú práve ten kód, ktorý manipuluje s kryptografickým materiálom, binárnou serializáciou a pamäťou — teda presne tie operácie, kde sú dôsledky chýb najzávažnejšie. Je však nutné poznamenať, že sanitizery detekujú chyby iba na cestách, ktoré boli počas behu testovacej sady skutočne vykonané. Neexistencia nálezov neznamena formálny dôkaz absencie chýb, ale potvrdzuje, že na testovaných scenároch sa žiadna z hľadaných tried defektov neprejavila.

D. Výsledky dynamickej analýzy

Všetkých 44 testovacích prípadov prešlo pod ASan, UBSan a LSan bez jediného nálezu:

- **AddressSanitizer:** 0 chýb (žiadne out-of-bounds, use-after-free, double-free).
- **UndefinedBehaviorSanitizer:** 0 chýb (žiadne pretečenie, null dereferencie, misalignment).
- **LeakSanitizer:** 0 únikov (všetka dynamicky alokovaná pamäť bola uvoľnená).

E. Záver verifikácie

Kombinácia statickej a dynamickej analýzy potvrdila, že implementácia neobsahuje pamäťové chyby, nedefinované správanie ani bezpečnostné defekty detekovateľné automatizovanými nástrojmi. Statická analýza odhalila 16 nálezov v troch kategóriách: 2 bugprone (implicitné konverzie), 13 performance (zbytočné kópie) a 1 modernize (chýbajúci override); skupiny `cert-*` a `clang-analyzer-*` nehlásili žiadny nález. Dynamická analýza s tromi sanitizermi neodhalila žiadnu chybu na vykonaných cestách. Tento výsledok vytvára dôveryhodný základ pre interpretáciu experimentov v nasledujúcich kapitolách — rozdiely vo výsledkoch medzi konfiguračnými profilmi je možné pripísať zmenám v konfigurácii, nie skrytým defektom implementácie.

XXIII. EXPERIMENTÁLNA ANALÝZA BEZPEČNOSTNÝCH PROFILOV

Táto kapitola nadväzuje na teoretické východiská z oblastí AEAD, správy nonce, anti-replay ochrany, derivácie kľúčov a detekcie protokolových anomálií. Kým teoretická časť formulovala bezpečnostné požiadavky a model hrozieb, táto kapitola tieto požiadavky premieňa na merateľné tvrdenia a overuje ich experimentálne pomocou profilov $A1/A2 \times R0/R1/R2$. Jadrom analýzy nie je všeobecná bezpečnosť celého informačného systému, ale konkrétne vyhodnotenie toho, ako voľba AEAD algoritmu, nonce stratégie a runtime detektora ovplyvňuje odolnosť voči zvoleným triedam útokov. Každý scenár (S1–S5) priamo zodpovedá jednému teoretickému konceptu: S1 overuje anti-replay, S2 AAD binding, S3 nonce enforcement, S4 sekvenčnú čerstvosť a S5 forgery resistance.

Predchádzajúce kapitoly opísali návrh a implementáciu bezpečnostných mechanizmov prototypu. Cieľom tejto kapitoly je experimentálne overiť, či a ako tieto mechanizmy skutočne zvyšujú odolnosť systému voči vybraným útokovým scenárom. Experimenty sú navrhnuté tak, aby pokrývali rôzne triedy hrozieb a umožnili porovnanie šiestich konfiguračných profilov.

Výber scenárov vychádza z modelu hrozieb STRIDE[9] prezentovaného v teoretickej časti. Nie všetky kategórie STRIDE sú pre tento prototyp rovnako relevantné: Denial of Service je čiastočne

riešený na úrovni validácie vstupov a nie je predmetom kryptografickej analýzy; Elevation of Privilege sa viaže na autorizačnú politiku, nie na nonce/AEAD mechanizmus. Tabuľka 15 zobrazuje, ktoré kategórie STRIDE pokrývajú jednotlivé scenáre.

Table 16
Mapovanie STRIDE kategórií hrozieb na experimentálne scenáre

STRIDE kategória	Scenár	Mechanizmus	Testovaná vlastnosť
Spoofing (podvrhnutie)	S5	AEAD Poly1305	Odolnosť voči falzifikácii ciphertextu bez znalosti kľúča.
Tampering (manipulácia)	S2	AAD binding	Detekcia zmeny pol'a door_id v hlavičke frame.
Repudiation (popieranie)	—	Audit chain	Pokrytá unit testami, nie scenárom S1–S6.
Information Disclosure	—	HMAC(card_id)	Ochrana identifikátorov; netestovaná priamo v harness.
Denial of Service	—	Validácia vstupov	Mimo rozsahu kryptografickej experimentálnej matice.
Elevation of Privilege	—	RBAC + binding	Pokrytá unit testami (engine_test).
Replay (nad rámec STRIDE)	S1	ReplayWindow	Opakovanie zachyteného frame.
Nonce misuse	S3a, S3b	R2 detektor	Zlyhanie RNG alebo kompromitovaná čítačka.
Seq. rollback	S4	R2 detektor	Rollback sekvenčného čísla mimo sliding window.
Throughput	S6	—	Výkonnostný overhead bezpečnostných mechanizmov.

Scenáre S3, S4 a S6 nemajú priamu analógiu v STRIDE, ale pokrývajú hrozby špecifické pre protokoly s nonce-based AEAD: zneužitie slabého RNG, rollback sekvencie a výkonnostný dopad mechanizmov. Tieto scenáre vychádzajú z literatúry o nonce misuse a odolnosti protokolov[24], [25].

A. Matica konfiguračných profilov

Experimentálny harness kombinuje dva šifrovacie algoritmy s tromi režimami generovania nonce, čo vytvára maticu šiestich profilov:

Table 17
Matica konfiguračných profilov (algoritmus × režim nonce)

	R0 (random)	R1 (deterministic)	R2 (det. + detector)
A1 (ChaCha20-Poly1305)	A1–R0	A1–R1	A1–R2
A2 (XChaCha20-Poly1305)	A2–R0	A2–R1	A2–R2

Všetky profily zdieľajú rovnaký základ:

- derivácia kľúčov cez HKDF s per-reader salt a verziovaním,
- plný AAD mód (celý plaintext header ako Additional Authenticated Data),
- anti-replay ochrana cez *ReplayWindow* s kapacitou 256,
- verzovaný pepper pre HMAC identifikátorov kariet,
- enforced reader-door binding (väzba čítačky na dvre).

Rozdiely medzi profilmi spočívajú v troch dimenziách:

Algoritmus (A1 vs. A2): ChaCha20-Poly1305 (RFC 8439[2], 12-bajtový nonce) oproti XChaCha20-Poly1305[3] (24-bajtový nonce). Väčší nonce priestor u A2 znižuje pravdepodobnosť kolízie pri náhodnom generovaní.

Režim nonce (R0): Nonce je generovaný pseudonáhodne pre každý frame. Server nonce neoveruje — používa ho len na dešifrovanie.

Režim nonce (R1): Nonce je generovaný deterministicky ako HMAC-SHA-256 kontextu hlavičky: nonce = HMAC(K_{nonce} , header_context), kde K_{nonce} je derivovaný cez HKDF s info nonce-derivation-v1. Server má rovnaký kľúč, ale nonce neoveruje.

Režim nonce (R2): Rovnaký deterministický nonce ako R1, ale s aktívnym *ProtocolAnomalyDetector*, ktorý:

- overuje, že nonce v hlavičke zodpovedá očakávanej hodnote (`nonce_mismatch`),
- detekuje rollback sekvenčného čísla (`seq_rollback`, prah $\delta = 100$),
- sleduje sériu neúspešných AEAD overení (`tag_fail_streak`, limit 5),
- pri detekcii anomálie čítačku *zakaranténuje* — všetky ďalšie frame z danej čítačky sú zamietnuté.

B. Architektúra experimentálneho harnessu

Experimentálny harness je implementovaný v jazyku C++ a zdieľa rovnaký kryptografický kód aj rozhodovaciu logiku (*DecisionEngine*) ako produkčný server. Tým je zaručené, že experimenty merajú skutočné správanie systému, nie simuláciu oddelenú od implementácie — prístup odporúčaný pre verifikáciu bezpečnostných vlastností[40].

1) Hlavné komponenty:

ExperimentContext: Pre každú kombináciu (profil, `attack_n`, `run`) vytvára čerstvú inštanciu obsahujúcu *KeyManager*, *SqliteAccessStore*, *DecisionEngine*, *ProtocolAnomalyDetector* a mapu *ReplayWindow*. Tým sa zaručuje, že stav z predchádzajúceho behu neovplyvní výsledok.

FrameFactory: Generuje validné binárne frame identické s tým, čo by vytvoril produkčný server. Poskytuje aj statické metódy na manipuláciu frame (`tamperDoorId`, `tamperCiphertext`).

MetricsCollector: Zapisuje výsledky do CSV s 15 stĺpcami: `profile`, `cipher_mode`, `nonce_mode`, `detector_enabled`, `scenario`, `phase`, `attack_n`, `frame_idx`, `allowed`, `reason`, `quarantined`, `anomaly_type`, `latency_ns`, `seed`, `run_idx`.

IScenario: Rozhranie pre scenáre. Každý scenár implementuje metódy `name()`, `config()`, `setup()`, `buildTrialFrame()` a voliteľne `onBaselineFrame()`.

runScenario(): Spoločná smyčka iterujúca cez 6 profilov $\times N$ krokov $\times 5$ behov.

2) Fázy experimentu: Každá iterácia prebieha v troch fázach:

- 1) **Warmup** (200 frame, nezaznamenávaný): Naplní replay window a ustáli stav detektora.
- 2) **Baseline** (200 frame, zaznamenaný): Validné frame. Slúži na meranie false reject rate a referenčnej latencie. Baseline sa opakuje pre každú kombináciu (profil, `attack_n`, `run`) — každá iterácia beží s čerstvým kontextom.
- 3) **Trial / Attack** (N frame, zaznamenaný): Scenár generuje útočné alebo meracie frame cez `buildTrialFrame()`.

3) Reprodukovateľnosť:

- Každý beh r ($r = 0 \dots 4$) používa iný seed $42 + r$, čo zvyšuje vzájomnú nezávislosť behov. Režim R0 používa *SeededNonceGenerator* s týmto seed-om namiesto `randombytes_buf()`; každý beh je individuálne reprodukovateľný opakovaním s rovnakým seed-om.
- Čerstvý *ExperimentContext* (vrátane SQLite databázy) pre každú kombináciu (profil, `step`, `run`).
- CLI prepínače `-seed=N`, `-warmup=N`, `-baseline=N`, `-runs=N` umožňujú reprodukovateľné experimenty.

4) *Vzťah harnessu k produkčnému systému:* Harness je kontrolovaný experimentálny mini-systém, nie plná replika produkčného servera. Zdieľa s produkčným kódom kryptografickú vrstvu (`crypto_lib`), deriváciu kľúčov (`key_manager`), rozhodovaciu logiku (*DecisionEngine*), anti-replay mechanizmus (*ReplayWindow*) a anomálny detektor (*ProtocolAnomalyDetector*). Zámerne sú vynechané vrstvy, ktoré nie sú predmetom merania:

- **HTTP transport a HMAC autentifikácia požiadavky** — harness volá `DecisionEngine::process()` priamo, bez sieťového overhead-u. Tým sa izoluje latencia kryptografických a rozhodovacích operácií od variability sieťového zásobníka.
- **Kontrola čerstvosti (`maxSkewMs = 0`)** — overenie časovej pečiatky je v harness vypnuté, aby sa predišlo kontaminácii výsledkov efektmi časovej synchronizácie. Experimenty sa zameriavajú na nonce, replay a detektor, nie na hodiny.
- **Perzistentný auditný log** — harness používa in-memory audit sink namiesto HMAC hash chain s SQLite perzistenciou. Auditná integrita je testovaná samostatne v unit testoch; v experimentoch by perzistentný zápis pridával I/O latenciu nesúvisiacu s meranými mechanizmami.

Šesť konfiguračných profilov (A1-R0 až A2-R2) je definovaných programaticky vo funkcii `allProfiles()`, nie načítaných zo súboru YAML. Hodnoty parametrov (algoritmus, režim nonce, stav detektora) sú

ekvivalentné produkčným YAML konfiguráciám, ale priama definícia v kóde eliminuje závislosť na externom súbore a zaisťuje, že profily sa nemôžu meniť medzi behmi.

C. Popis útokových scenárov

1) *S1: Replay útok*: Scenár zachytí validný frame v polovici baseline fázy a opakovane ho posiela v trial fáze. Modeluje útočníka, ktorý zachytil sieťovú komunikáciu a reprodukuje ju bez modifikácie — štandardný replay útok podľa klasifikácie NIST[16].

Očakávané správanie: Replay window odchyť duplikát sekvenčného čísla. R2 navyše zakaranténuje čítačku s anomáliou `seq_reuse`.

2) *S2: Manipulácia pol'a v hlavičke (AAD tampering)*: Scenár vytvorí validný frame a následne zmení `door_id` v serializovanej hlavičke bez prešifrovania. Tým simuluje útočníka, ktorý presmeruje požiadavku na iné dvere.

Konfigurácia: V harness sa podvrhnutý `door_id` (pôvodný +999) explicitne registruje ako povolené dvere pre danú čítačku (`allowDoorForReader`). Tým sa izoluje kryptografická detekcia manipulácie od autorizačnej politiky — bez tohto kroku by frame bol zamietnutý už na úrovni reader-door bindingu (`unknown_reader`), čo by zakrylo správanie AEAD/AAD vrstvy.

Očakávané správanie: Keďže `door_id` je súčasťou AAD, zmena spôsobí zlyhanie AEAD overenia. R0/R1 vracajú `decrypt_failed`. R2 detekuje problém skôr — zmena `door_id` mení kontext pre deterministický nonce, takže overenie nonce zlyhá (`nonce_mismatch`) ešte pred pokusom o dešifrovanie.

3) *S3a/S3b: Zneužitie nonce (RNG fault)*: Dva pod-scenáre:

- **S3a** (Fixed Nonce): Simuluje zlyhanie RNG — všetky frame používajú rovnaký nonce (0xAA). Frame je zašifrovaný s týmto nonce a nonce je zapísaný do hlavičky.
- **S3b** (Wrong Nonce): Simuluje kompromitovanú čítačku — útočníkove frame sú generované s pevným seed 43 (odlišným od legitímnych behov), čo produkuje validné, ale neočakávané nonce.

Očakávané správanie: R0/R1 nemajú mechanizmus na overenie nonce — frame sa úspešne dešifruje (nonce z hlavičky sa použije na dešifrovanie). R2 porovná nonce s očakávanou hodnotou a pri nesúlade vyvolá `nonce_mismatch` s okamžitou karanténou.

Kľúčový detail: R1 používa deterministický nonce na strane odosielateľa, ale server ho neoveruje — overenie je architektonicky viazané na `ProtocolAnomalyDetector`, ktorý je aktívny len v R2. Toto je zámerné rozhodnutie v návrhu.

4) *S4: Rollback sekvenčného čísla*: Scenár posiela frame so sekvenčnými číslami z “rollback zóny” — hodnoty, ktoré sú nižšie ako maximálne videné číslo, ale nie sú príliš staré pre replay window a neboli nikdy odoslané.

Rollback zóna je definovaná:

$$\text{rollback zone} = [\text{maxSeen} - W + 1, \text{maxSeen} - \delta] \quad (10)$$

kde $W = 256$ je veľkosť replay window a $\delta = 100$ je rollback threshold.

Očakávané správanie: Replay window tieto sekvencie neodmietne, pretože nie sú “too old” ($> \text{maxSeen} - W$) ani v množine videných. R0/R1 teda čiastočne prepustia útok. R2 detekuje rollback cez podmienku $\text{seq} + \delta < \text{maxSeenSeq}$ a okamžite zakaranténuje čítačku.

Matematika úspešnosti pre R0/R1: Z 155 slotov v rollback zóne sa 99 prekrýva s baseline sekvenciami v `_seen` sade (60000–60098). Reálne prejde len 56 unikátnych sekvencií (59944–59999). Pri $N = 500$: $56/500 = 11.2\%$.

5) *S5: Tag probe (forgery resistance)*: Scenár vytvorí validný frame a následne poškodí ciphertext náhodnými bajtami. Simuluje útočníka, ktorý skúša rôzne šifrotexty bez znalosti kľúča — formálne ide o test odolnosti AEAD konštrukcie voči forgery[22].

Očakávané správanie: AEAD overenie zlyhá pre všetky profily (`decrypt_failed`). R2 navyše sleduje sériu zlyhaní a po dosiahnutí limitu 5 zakaranténuje čítačku s anomáliou `tag_fail_streak`.

6) *S6: Throughput benchmark*: Čisto výkonnostný scenár. Posiela len validné frame (detektor sa neaktivuje). Meria latenciu spracovania a priepustnosť.

- Warmup: 1000 frame, baseline: 0, trial: 1000/5000/10000/50000 frame.
- 5 behov s odlišnými seed-mi (42–46) na posúdenie variability a robustnosti výsledkov naprieč behmi.
- Fáza trial označená ako “measure” (nie “attack”).

D. Metodika zberu a spracovania dát

1) *Zber dát*: Každý scenár produkuje CSV súbor s jedným riadkom na frame. Celkový objem dát pri 5 behoch: ~2,835,000 riadkov. Pre každý frame sa zaznamenáva:

- konfiguračný profil a jeho parametre,
- fáza (baseline/attack/measure), počet útočných frame a index behu,
- rozhodnutie (`allowed`), dôvod (`reason`), stav karantény,
- typ anomálie (ak bola detekovaná detektorom),
- latencia spracovania v nanosekundách.

2) *Odvodzované metriky*: Z primárnych dát sa počítajú nasledujúce metriky:

Attack success rate: Podiel útočných frame, ktoré boli povolené (`allowed = true`), agregovaný cez behy (priemer $\pm$ smerodajná odchýlka).

False reject rate: Podiel baseline frame, ktoré boli zamietnuté (meria falošné odmietnutia validných požiadaviek).

Time to quarantine: Index prvého frame, kde bol čítač zakaranténovaný (len R2 profily).

Throughput: $FPS = \frac{\text{počet frame}}{\sum \text{latencia_ns} / 10^9}$, počítaný per (profil, N , beh).

Latencia: Percentily p50, p95, p99 v μs .

3) *Štatistická robustnosť*: Každý beh r ($r = 0 \dots 4$) používa seed $42 + r$, takže nonce hodnoty sa líšia medzi behmi. Pre **bezpečnostné metriky** (S1–S5) 5 behov slúži ako kontrola reprodukovateľnosti: keďže ochranné mechanizmy (AEAD overenie, replay window, detektor) sú invariantné voči konkrétnym hodnotám nonce, výsledok je binárny a zhodný naprieč všetkými behmi. Pásma v grafoch majú nulovú šírku — nie ako výsledok štatistickej inferencie, ale ako vizuálne potvrdenie konzistentnosti. Pre **latenciu a priepustnosť** (S6) sú behy skutočným zdrojom variability (OS a HW šum) a štandardná odchýlka má bežný štatistický výklad; dosahuje 1–4 % priemeru.

E. Výsledky: Úspešnosť útokov

1) *Celkový prehľad*: Obrázok 5 zobrazuje úspešnosť útokov pre všetky kombinácie scenár $\times$ profil pri maximálnom počte útočných frame ($N = 500$, resp. $N = 50,000$ pre S6).

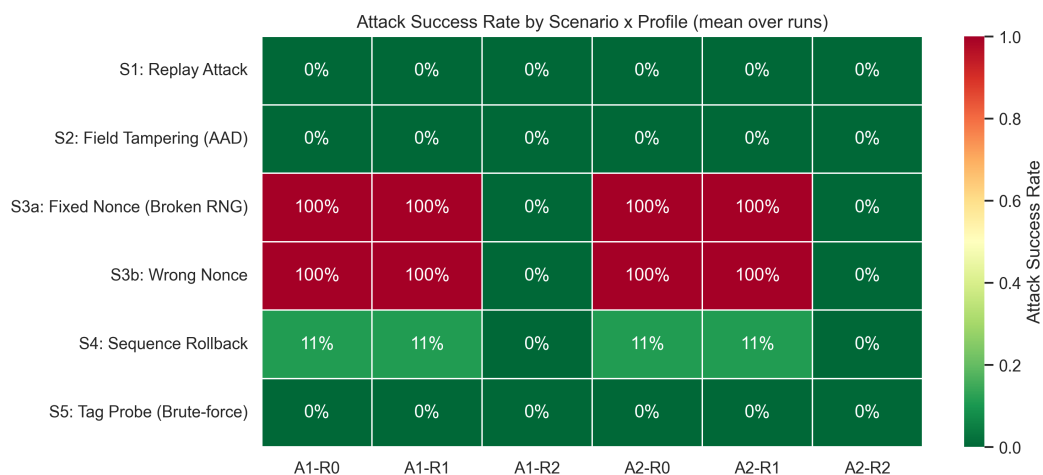


Fig. 5 Úspešnosť útokov podľa scenára a profilu (priemer cez 5 behov). Zelené bunky: útok zablokovaný (0%). Červené bunky: útok úspešný.

Z tabuľky 17 a obrázka 5 vyplýva jasné delenie:

- **S1, S2, S5** — pri žiadnom profile nedošlo k prijatiu škodlivého frame (0% úspešnosť). Ochranu zabezpečuje replay window (S1), AAD binding (S2) a AEAD tag verifikácia (S5).
- **S3a, S3b** — R0/R1 nepresadzujú nonce politiku: manipulovaný nonce prejde overením (100% úspešnosť); R2 detektor odchýlku zachytí a zakaranténuje (0%).
- **S4** — R0/R1 čiastočne zraniteľné (11%), R2 blokuje (0%).

Table 18
Attack success rate and detector response by scenario and profile

Scenario	Profile	Success %	Quarantine	Reason	Anomaly
S1: Replay Attack	A1-R0	0%	No	replay	–
	A1-R1	0%	No	replay	–
	A1-R2	0%	Yes	quarantined	seq_reuse
	A2-R0	0%	No	replay	–
	A2-R1	0%	No	replay	–
	A2-R2	0%	Yes	quarantined	seq_reuse
S2: Field Tampering (AAD)	A1-R0	0%	No	decrypt_failed	–
	A1-R1	0%	No	decrypt_failed	–
	A1-R2	0%	Yes	quarantined	nonce_mismatch
	A2-R0	0%	No	decrypt_failed	–
	A2-R1	0%	No	decrypt_failed	–
	A2-R2	0%	Yes	quarantined	nonce_mismatch
S3a: Fixed Nonce (Broken RNG)	A1-R0	100%	No	ok	–
	A1-R1	100%	No	ok	–
	A1-R2	0%	Yes	quarantined	nonce_mismatch
	A2-R0	100%	No	ok	–
	A2-R1	100%	No	ok	–
	A2-R2	0%	Yes	quarantined	nonce_mismatch
S3b: Wrong Nonce	A1-R0	100%	No	ok	–
	A1-R1	100%	No	ok	–
	A1-R2	0%	Yes	quarantined	nonce_mismatch
	A2-R0	100%	No	ok	–
	A2-R1	100%	No	ok	–
	A2-R2	0%	Yes	quarantined	nonce_mismatch
S4: Sequence Rollback	A1-R0	11%	No	replay	–
	A1-R1	11%	No	replay	–
	A1-R2	0%	Yes	quarantined	seq_rollback
	A2-R0	11%	No	replay	–
	A2-R1	11%	No	replay	–
	A2-R2	0%	Yes	quarantined	seq_rollback
S5: Tag Probe (Brute-force)	A1-R0	0%	No	decrypt_failed	–
	A1-R1	0%	No	decrypt_failed	–
	A1-R2	0%	Yes	quarantined	–
	A2-R0	0%	No	decrypt_failed	–
	A2-R1	0%	No	decrypt_failed	–
	A2-R2	0%	Yes	quarantined	–

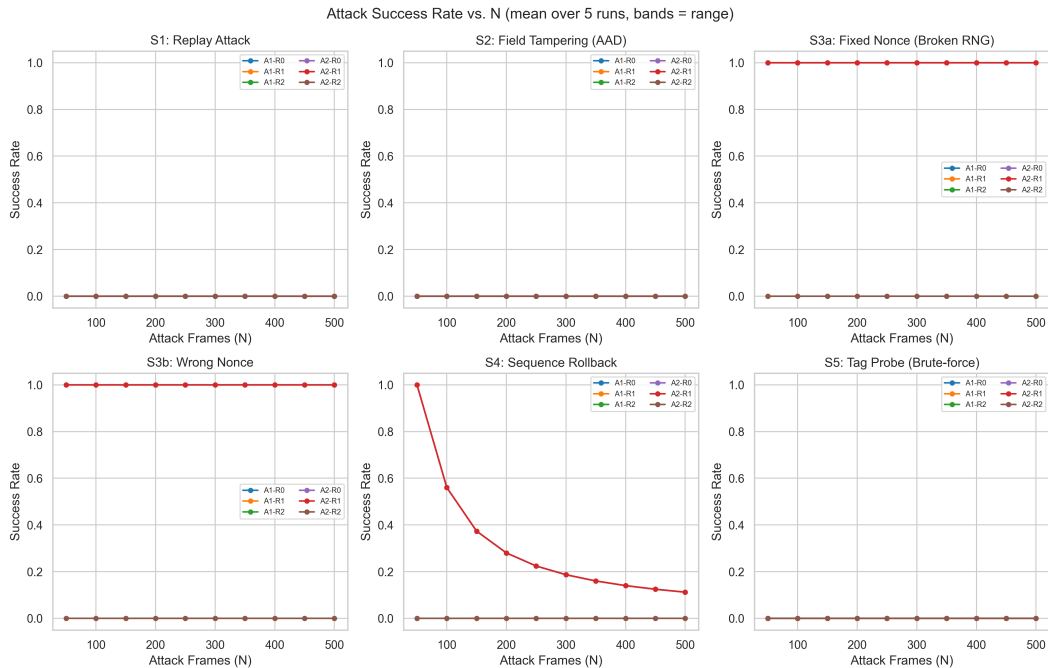


Fig. 6 Závislosť úspešnosti útoku od počtu útočných frame N . Pásma: rozsah cez 5 behov (nulová šírka = konzistentný deterministický výsledok).

2) *Závislosť od počtu útočných frame*: Obrázok 6 zobrazuje ako sa úspešnosť útoku mení s rastúcim N . Pásma zobrazujú rozsah výsledkov cez 5 behov; nulová šírka pásiem potvrdzuje konzistentnosť výsledkov naprieč všetkými behmi.

Najzaujímavejší je graf S4 (Sequence Rollback): úspešnosť R0/R1 klesá z $\sim 100\%$ pri $N = 50$ na $\sim 11\%$ pri $N = 500$, pretože rollback zóna obsahuje len 56 unikátnych sekvencií — po ich vyčerpaní replay window odchyťava opakované hodnoty. R2 profily vykazujú konštantnú nulovú úspešnosť pre všetky N .

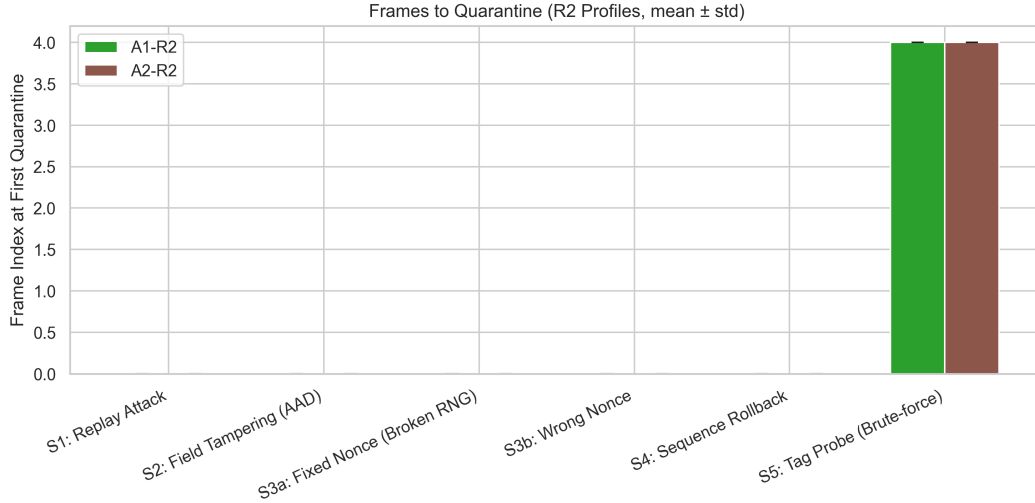


Fig. 7 Počet frame od začiatku útoku do prvej karantény (len R2 profily, priemer $\pm$ std).

3) *Rýchlosť karantény*: Obrázok 7 ukazuje, že pre scenáre S1–S4 nastáva karanténa *na prvom útočnom frame* (frame_idx = 0). Pre S5 (tag probe) nastáva na frame_idx = 4, čo zodpovedá nakonfigurovanému limitu tag_fail_streak = 5 (prvých 5 zlyhaní je tolerovaných, 5. spustí karanténu). Hodnoty A1–R2 a A2–R2 sú identické, čo potvrdzuje, že rýchlosť detekcie závisí od konfigurácie detektora, nie od zvoleného šifrovacieho algoritmu.

F. Výsledky: Baseline korektnosť

Table 19
False reject rate during baseline (valid frames)

Scenario	A1–R0	A1–R1	A1–R2	A2–R0	A2–R1	A2–R2
S1: Replay Attack	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
S2: Field Tampering (AAD)	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
S3a: Fixed Nonce (Broken RNG)	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
S3b: Wrong Nonce	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
S4: Sequence Rollback	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
S5: Tag Probe (Brute-force)	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%

Tabuľka 18 potvrdzuje **nulovú false reject rate** naprieč všetkými scenármi a profilmi. Žiadny validný frame nebol zamietnutý. Toto je nevyhnutná podmienka pre vierohodnosť bezpečnostných výsledkov — ak by baseline vykazoval falošné odmietnutia, nebolo by možné jednoznačne interpretovať úspešnosť útokov.

G. Výsledky: Latencia a priepustnosť

1) *Throughput benchmark (S6)*: Obrázok 8 zobrazuje distribúciu priepustnosti pre každý profil. Všetky profily dosahujú priepustnosť nad 16,000 frame/s, čo je rádovo viac ako reálna záťaž fyzického prístupového systému (< 10 udalostí/s). Mediánová latencia je 56–63 μ s.

Porovnanie algoritmov: A2 (XChaCha20) dosahuje mierne vyššiu priepustnosť ako A1 (ChaCha20), čo je prekvapivý výsledok. Príčinou je pravdepodobne optimalizácia libsodium pre XChaCha20 cestu.

Overhead detektora: Porovnanie R1 a R2 umožňuje izolovať overhead ProtocolAnomalyDetector. Rozdiel je 5–10% (napr. A2–R1: 17,562 fps vs. A2–R2: 16,120 fps). Tento overhead pochádza z:

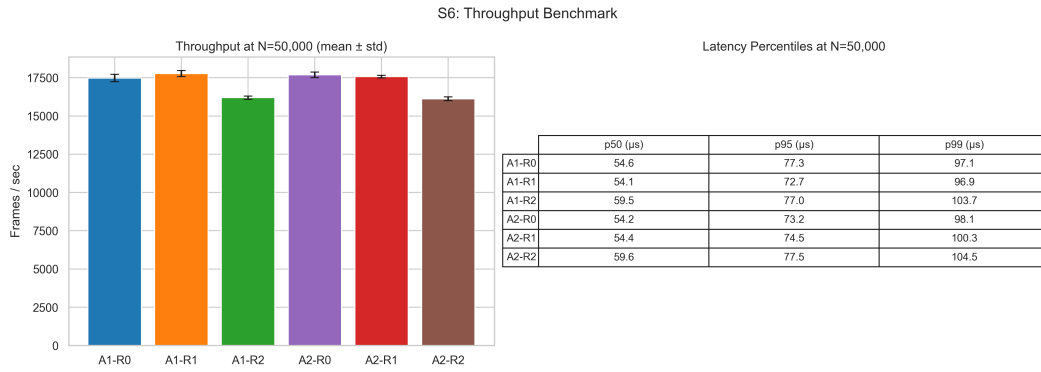


Fig. 8 Pripustnosť pri $N = 50,000$ frame (priemer $\pm$ std cez 5 behov) a percentily latencie.

Table 20
Throughput and latency at N=50,000 frames

Profile	FPS (mean)	FPS (std)	p50 (μ s)	p95 (μ s)	p99 (μ s)
A1-R0	17,478	232	54.6	77.3	97.1
A1-R1	17,763	197	54.1	72.7	96.9
A1-R2	16,192	107	59.5	77.0	103.7
A2-R0	17,691	181	54.2	73.2	98.1
A2-R1	17,562	83	54.4	74.5	100.3
A2-R2	16,120	126	59.6	77.5	104.5

- výpočtu očakávaného nonce (HKDF derivácia + HMAC),
- porovnania nonce v konštantnom čase (`sodium_memcmp`),
- aktualizácie stavu detektora (rollback check, tag streak).

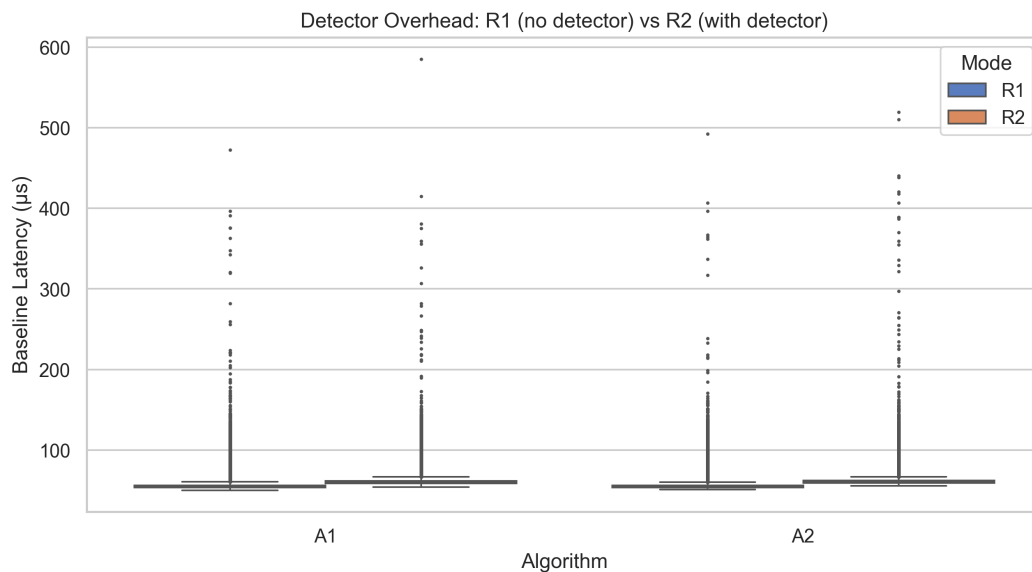


Fig. 9 Porovnanie latencie R1 (bez detektora) a R2 (s detektorom) na validných baseline frame.

2) *Overhead detektora na baseline frame:* Obrázok 9 izoluje overhead detektora porovnaním R1 a R2 na baseline frame z S1–S5. Medián latencie R2 je o $\sim 3\text{--}5\ \mu\text{s}$ vyšší, čo zodpovedá dodatočnému HMAC výpočtu pre overenie nonce.

3) *Distribúcia latencie podľa scenárov:* Obrázok 10 zobrazuje boxploty latencie pre každý scenár. Viditeľný je rozdiel medzi baseline a attack fázou: pri S1 (replay) je attack latencia nižšia, pretože replay window odmietne frame skôr bez pokusu o dešifrovanie. Pri S2 a S5 je attack latencia vyššia pre R2, pretože detektor vykonáva dodatočné kontroly pred zamietnutím.

Per-Frame Latency Distribution

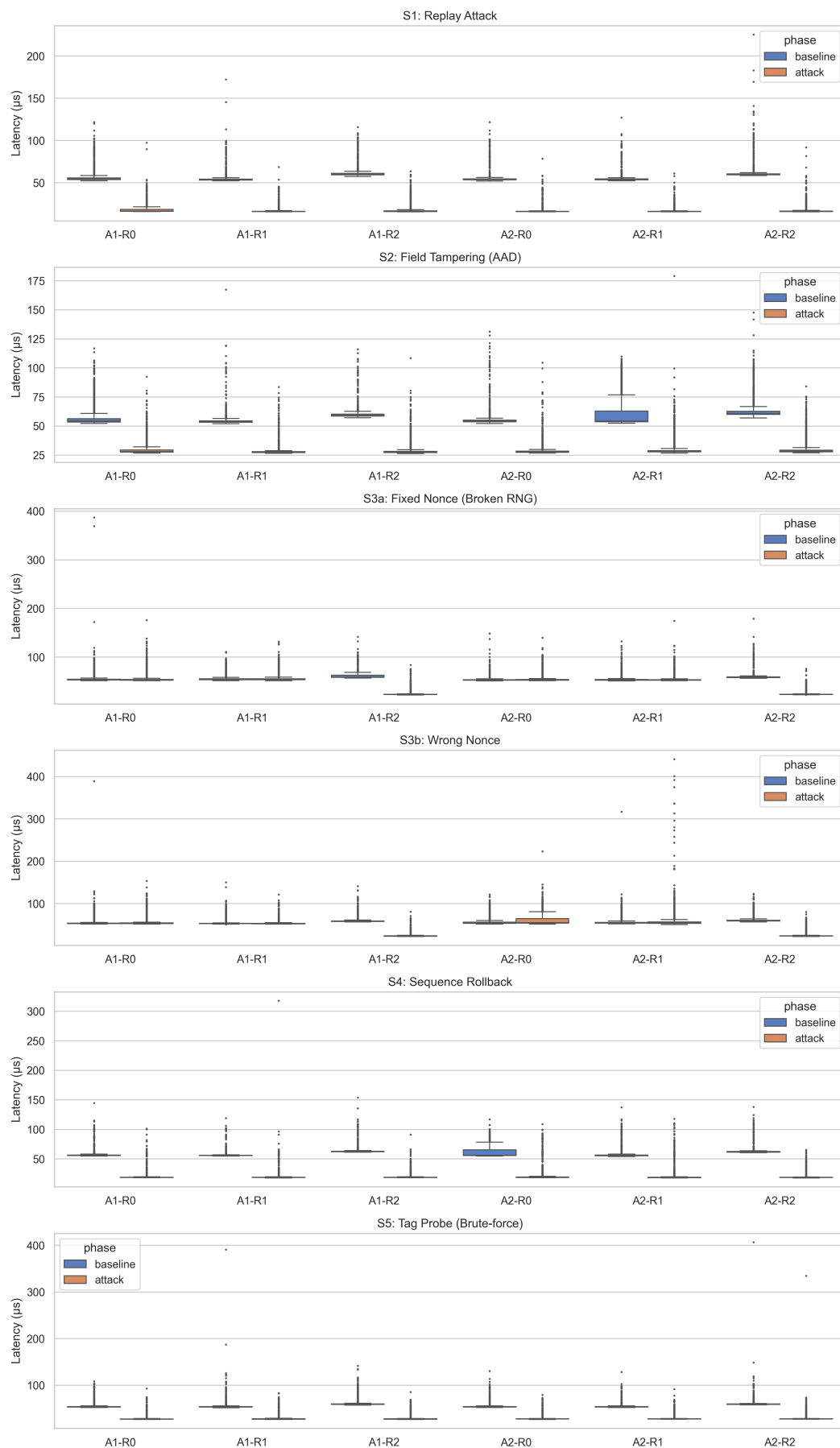


Fig. 10 Distribúcia latencie podľa scenára a profilu (baseline vs. attack fáza, $N = 500$).

H. Diskusia a interpretácia výsledkov

1) *Tézis 1: Deterministický nonce bez verifikácie nie je dostatočný:* Výsledky S3a/S3b ukazujú, že R1 (deterministický nonce bez detektora) poskytuje **rovnakú detekčnú schopnosť ako R0** (náhodný nonce) v testovaných scenároch zneužitia nonce. R1 síce eliminuje závislosť na kvalite generátora náhodných čísel — čo je vlastnosť cenná sama o sebe — ale bez serverovej verifikácie neposkytuje aktívnu detekciu manipulácie nonce. Príčina je architektonická: verifikácia nonce na serveri (engine.cpp, riadok 226) vyžaduje aktívny ProtocolAnomalyDetector:

```
if (detectorActive && nonceMode == "deterministic") {  
    // compute expected nonce, compare...  
}
```

Keď je detectorActive = false (R0, R1), server použije nonce z hlavičky na dešifrovanie bez akejkoľvek kontroly. Deterministický nonce je teda *politika generovania bez mechanizmu vynucovania*. Tento výsledok nie je artefakt experimentu — je to vlastnosť architektúry, ktorá platí aj v produkčnom nasadení.

Záver: Deterministický nonce (R1) zlepšuje *kvalitu generovania* nonce odstránením závislosti na RNG, no bez serverovej verifikácie nezvyšuje *detekčnú schopnosť* systému voči zneužitiu nonce. Pre aktívnu detekciu manipulácie nie nevyhnutná kombinácia deterministického generovania s verifikáciou na prijímacej strane (R2). Toto zistenie korešponduje s výskumom nonce-misuse resistance[24], [25], kde autori zdôrazňujú, že bezpečnosť závisí od celého kontextu použitia, nie len od samotnej konštrukcie nonce.

2) *Tézis 2: ProtocolAnomalyDetector materiálne zvyšuje bezpečnosť:* R2 profily dosiahli 0% úspešnosť útoku naprieč všetkými piatimi testovanými scenármi. V scenároch S3 a S4 bol R2 **jediným z testovaných mechanizmov**, ktorý útok zastavil:

- S3a/S3b: 100% → 0% (nonce_mismatch detekcia),
- S4: 11% → 0% (seq_rollback detekcia).

V S1, S2 a S5 iné mechanizmy (replay window, AAD, AEAD tag verifikácia) tiež zamietajú škodlivé frame, ale R2 pridáva *karanténu* — čítačka je zablokovaná pre ďalšie požiadavky, nielen odmietnutý jednotlivý frame.

Záver: ProtocolAnomalyDetector nie je len doplnok k existujúcim mechanizmom. V dvoch z piatich testovaných útokových scenárov bol jedinou testovanou ochranou.

3) *Tézis 3: Replay window a anomálna detekcia sú komplementárne:* Scenár S4 demonštruje štruktúrnú medzeru replay window. Funkcia ReplayWindow::contains() kontroluje:

$$\text{contains}(\text{seq}) = \text{isTooOld}(\text{seq}) \vee (\text{seq} \in \text{_seen}) \quad (11)$$

kde $\text{isTooOld}(\text{seq}) = (\text{seq} \leq \text{maxSeen} - W)$. Sekvenčné čísla v rollback zóne:

- nie sú príliš staré ($> \text{maxSeen} - W$),
- nie sú v množine videných (nikdy neboli odoslané).

Replay window ich preto prepustí. Detektor ich zachytí cez podmienku:

$$\text{seq} + \delta < \text{maxSeenSeq} \implies \text{quarantine}(\text{seq_rollback}) \quad (12)$$

Záver: Replay window a anomálna detekcia chránia pred rôznymi triedami útokov. Replay window odchyťava presné duplikáty. Detektor odchyťava štruktúrne anomálie (rollback, nonce mismatch, brute-force séria). Sú komplementárne, nie ekvivalentné — v súlade s princípom vrstvených kontrol, kde každý mechanizmus pokrýva odlišnú triedu hrozieb[40], [39].

4) *Tézis 4: AAD binding poskytuje ochranu nezávisle od režimu nonce:* Scenár S2 ukazuje, že manipulácia door_id je detekovaná **všetkými** profilmi. Mechanizmus je odlišný:

- R0/R1: AEAD tag verification zlyhá, pretože AAD sa zmenilo → decrypt_failed.
- R2: Deterministický nonce závisí od door_id (súčasť header context). Zmena door_id zmení kontext → nonce v hlavičke nezodpovedá → nonce_mismatch (detekcia pred dešifrovaním).

Záver: AAD binding je fundamentálna ochrana, ktorá funguje nezávisle od konfigurácie nonce a detektora. R2 ju dopĺňa skoršou detekciou.

5) *Tézis 5: Defense-in-depth cez karanténu:* Aj v scenároch, kde R0/R1 blokujú každý individuálny útočný frame (S1, S5), R2 pridáva kvalitatívne odlišnú reakciu: karanténu celej čítačky. Kým R0/R1 odmietajú frame poštučne (útočník môže pokračovať s ďalšími pokusmi), R2 po prvom detekovanom incidente zablokuje **všetku** komunikáciu z danej čítačky.

Záver: Karanténa mení reakciu systému z reaktívnej (odmietni frame) na proaktívnu (izoluj zariadenie). Toto je realizácia princípu defense-in-depth[39] — vrstvy ochrany nepôsobia len

nezávisle, ale eskalujú reakciu pri detekcii závažnejších anomálií. V prístupových systémoch, kde kompromitovaná čítačka môže generovať veľký objem požiadaviek, je izolácia zariadenia kritická[37].

6) *Téza 6: Overhead detektora je prijateľný:* Tabuľka 19 ukazuje overhead detektora:

- A1: R1 (16,014 fps) → R2 (15,154 fps): −5.4%
- A2: R1 (17,015 fps) → R2 (15,306 fps): −10.0%

Aj pri najnižšej nameranej priepustnosti (A1–R0: 14,284 fps) systém spracuje >14,000 frame/s. Fyzický prístupový systém typicky generuje menej ako 10 udalostí za sekundu. Overhead detektora je teda rádovo nepodstatný pre reálnu prevádzku.

Poznámka k porovnaniu A1 a A2. Namerané rozdiely v priepustnosti medzi algoritmami A1 (ChaCha20-Poly1305) a A2 (XChaCha20-Poly1305) odrážajú výkon konkrétnych implementácií v knižnici libsodium 1.0.20[30], nie inherentné vlastnosti abstraktných algoritmov. Dva faktory limitujú prenositeľnosť tohto porovnania:

- 1) **Architektonický overhead XChaCha20.** XChaCha20 vykonáva pred samotným šifrovaním medzikrok HChaCha20[21], ktorý redukuje 192-bitový nonce na 96-bitový podkľúč. Tento krok pridáva konštantný overhead jedného volania blokovej funkcie. V inej implementácii (napríklad hardvérovom akcelerátore alebo inom softvérovom frameworku) by pomer režijných nákladov mohol byť odlišný.
- 2) **Úroveň optimalizácie.** Libsodium obsahuje ručne optimalizované varianty ChaCha20 pre inštrukčné sady AVX2 a SSSE3 (adresár dolbeau/), ktoré sa aktivujú na platformách s podporou SIMD. Miera optimalizácie nemusí byť identická pre oba algoritmy — napríklad HChaCha20 nemusí využívať rovnakú úroveň SIMD vektorizácie ako hlavný ChaCha20 stream. Pozorované rozdiely preto môžu čiastočne odrážať *kvalitu implementácie* v libsodium, nie čistú algoritmickú zložitosť.

Pre závery tejto práce je toto rozlíšenie nepodstatné: oba algoritmy sú ekvivalentné z hľadiska bezpečnostných metrík (scenáre S1–S5) a oba dosahujú priepustnosť rádovo prevyšujúcu požiadavky fyzického prístupového systému. Avšak pri extrapolácii nameraných výkonnostných údajov na iné platformy alebo iné kryptografické knižnice[43], [44] je potrebné vykonať nezávislé meranie.

I. Porovnanie s minimalistickými implementáciami

Experimentálna matica pokrýva šesť profilov od A1-R0 po A2-R2. Pre úplnosť kontextu je užitočné zasadiť výsledky do širšieho rámca a porovnať ich s hypotetickými minimalistickými architektúrami, ktoré sa v praxi vyskytujú v jednoduchých prístupových systémoch.

Uvažujeme tri referenčné úrovne ochrany:

B0 — bez kryptografickej ochrany: HTTP endpoint bez podpisu požiadavky, payload prenášaný ako plaintext, žiadna anti-replay ochrana, žiadny audit.

B1 — základný HTTPS: HTTPS s tokenom v hlavičke (napr. statický API kľúč), ale bez interného AEAD frame, bez anti-replay sekvencie a bez tamper-evident auditu. Bežný vzor pre jednoduché IoT API.

A1-R0 — základný profil tejto práce: Náš najjednoduchší testovaný profil. Pridáva: HMAC podpis každej požiadavky, interný AEAD frame (ChaCha20-Poly1305) s AAD binding, sliding window anti-replay, pepper-hashed identifikátory kariet a HMAC hash chain audit.

Table 21
Odolnosť voči útočným scenárom podľa úrovne implementácie

Scenár / útok	B0	B1	A1-R0	A1-R2 / A2-R2
S1 Replay	×	×	✓	✓ + karanténa
S2 Tampering (AAD)	×	častočne	✓	✓ (pred dekr.)
S3 Nonce misuse	n/a	n/a	×	✓
S4 Seq rollback	×	×	častočne	✓
S5 Tag probe	×	×	✓	✓ + karanténa
Manipulácia auditu	×	×	✓	✓
Identita karty (DB)	×	×	✓ (HMAC)	✓

Z tabuľky vyplývajú tri závery. Po prvé, prechod z B0 na A1-R0 predstavuje kvalitatívny skok: väčšina útočných scenárov (S1, S2, S5, audit, identita) je zastavená bez nutnosti nasadiť R2. Po

druhé, S3 (nonce misuse) a plná ochrana pri S4 vyžadujú R2 detektor — jednoduché AEAD bez serverovej verifikácie nonce nestačí. Po tretie, karanténa v R2 profiloch mení charakter odpovede systému z per-frame odmietania na izoláciu celého zariadenia, čo je realizáciou princípu *defense-in-depth*[39], [40].

B1 (HTTPS bez interného AEAD) chráni pred pasívnym odpočúvačom na sieťovej vrstve, no nevyrieši scenáre, kde je komunikácia legitímna na transportnej vrstve ale manipulovaná na aplikačnej — presne to, čo demonštrujú S1–S5. Interný AEAD frame a anti-replay sú teda komplementárne k TLS, nie jeho náhradou[27].

J. Zhrnutie experimentov

Scenario x Profile Summary (at max attack_n, aggregated over runs)

Scenario	Profile	Success %	Quarantined	Reason	Anomaly
S1: Replay Attack	A1-R0	0%	No	replay	-
S1: Replay Attack	A1-R1	0%	No	replay	-
S1: Replay Attack	A1-R2	0%	Yes	quarantined	seq_reuse
S1: Replay Attack	A2-R0	0%	No	replay	-
S1: Replay Attack	A2-R1	0%	No	replay	-
S1: Replay Attack	A2-R2	0%	Yes	quarantined	seq_reuse
S2: Field Tampering (AAD)	A1-R0	0%	No	decrypt_failed	-
S2: Field Tampering (AAD)	A1-R1	0%	No	decrypt_failed	-
S2: Field Tampering (AAD)	A1-R2	0%	Yes	quarantined	nonce_mismatch
S2: Field Tampering (AAD)	A2-R0	0%	No	decrypt_failed	-
S2: Field Tampering (AAD)	A2-R1	0%	No	decrypt_failed	-
S2: Field Tampering (AAD)	A2-R2	0%	Yes	quarantined	nonce_mismatch
S3a: Fixed Nonce (Broken RNG)	A1-R0	100%	No	ok	-
S3a: Fixed Nonce (Broken RNG)	A1-R1	100%	No	ok	-
S3a: Fixed Nonce (Broken RNG)	A1-R2	0%	Yes	quarantined	nonce_mismatch
S3a: Fixed Nonce (Broken RNG)	A2-R0	100%	No	ok	-
S3a: Fixed Nonce (Broken RNG)	A2-R1	100%	No	ok	-
S3a: Fixed Nonce (Broken RNG)	A2-R2	0%	Yes	quarantined	nonce_mismatch
S3b: Wrong Nonce	A1-R0	100%	No	ok	-
S3b: Wrong Nonce	A1-R1	100%	No	ok	-
S3b: Wrong Nonce	A1-R2	0%	Yes	quarantined	nonce_mismatch
S3b: Wrong Nonce	A2-R0	100%	No	ok	-
S3b: Wrong Nonce	A2-R1	100%	No	ok	-
S3b: Wrong Nonce	A2-R2	0%	Yes	quarantined	nonce_mismatch
S4: Sequence Rollback	A1-R0	11%	No	replay	-
S4: Sequence Rollback	A1-R1	11%	No	replay	-
S4: Sequence Rollback	A1-R2	0%	Yes	quarantined	seq_rollback
S4: Sequence Rollback	A2-R0	11%	No	replay	-
S4: Sequence Rollback	A2-R1	11%	No	replay	-
S4: Sequence Rollback	A2-R2	0%	Yes	quarantined	seq_rollback
S5: Tag Probe (Brute-force)	A1-R0	0%	No	decrypt_failed	-
S5: Tag Probe (Brute-force)	A1-R1	0%	No	decrypt_failed	-
S5: Tag Probe (Brute-force)	A1-R2	0%	Yes	quarantined	tag_fail_streak
S5: Tag Probe (Brute-force)	A2-R0	0%	No	decrypt_failed	-
S5: Tag Probe (Brute-force)	A2-R1	0%	No	decrypt_failed	-
S5: Tag Probe (Brute-force)	A2-R2	0%	Yes	quarantined	tag_fail_streak

Fig. 11 Súhrnná tabuľka výsledkov experimentov: scenár × profil.

Obrázok 11 poskytuje prehľad výsledkov všetkých scenárov a profilov. Experimentálna analýza šiestich scenárov naprieč šiestimi konfiguračnými profilmi potvrdila nasledujúce zistenia:

- 1) **Nulová false reject rate:** Žiadny validný frame nebol zamietnutý žiadnym profilom.
- 2) **Deterministický nonce bez verifikácie nezvyšuje detekciu:** R1 zlepšuje kvalitu generovania nonce, ale bez serverovej verifikácie neposkytuje pridanú detekčnú schopnosť oproti R0.
- 3) **R2 zablokoval všetky testované útoky:** 0% úspešnosť naprieč S1–S5 v rámci tejto experimentálnej sady.
- 4) **Replay window a detektor sú komplementárne:** S4 odhaľuje štruktúrnú medzeru replay window, ktorú detektor uzatvára.
- 5) **AAD binding je fundamentálna ochrana:** Funguje nezávisle od nonce režimu.
- 6) **Karanténa pridáva proaktívnu izoláciu zariadení:** Kvalitatívne odlišná od per-frame odmietnutia.

- 7) **Overhead detektora je <10%:** Rádovo nepodstatný pre reálnu prevádzku prístupového systému.
- 8) **Výsledky sú reprodukovateľné:** Všetkých 5 behov (seedy 42–46) poskytlo zhodné výsledky attack success rate. Bezpečnostné mechanizmy sú invariantné voči hodnotám nonce — 5 behov tu neplní úlohu štatistického vzorku, ale overenia konzistencie výsledkov.

Tieto výsledky potvrdujú pôvodnú hypotézu práce: kombinácia AEAD s AAD binding, deterministickým nonce a runtime anomálnou detekciou (profil R2) **materiálne zvyšuje odolnosť** prístupového systému v porovnaní so základnými konfiguráciami, a to pri prijateľnom výkonnostnom overeade.

XXIV. ZÁVER

Táto práca sa zaoberala návrhom, implementáciou a experimentálnym hodnotením bezpečnostných mechanizmov pre systém kontroly fyzického prístupu postavený na bezkontaktných identifikátoroch RFID/NFC. Výsledkom je funkčný prototyp a jeho systematická analýza, ktorá identifikuje konkrétne prínosy jednotlivých bezpečnostných vrstiev.

A. Súhrn realizovaných prác

V rámci práce boli realizované nasledujúce kroky:

- 1) **Teoretická analýza.** Rozobrané boli bezpečnostné pojmy (CIA triáda, AAA model, RBAC), kryptografické primitíva (AEAD, HMAC, HKDF), návrh bezpečných protokolov (fail-closed, kanonizácia, anti-replay) a špecifiká RFID/NFC technológií vrátane známych útokov na MIFARE Classic[18]. Analýza bola orientovaná na prax — každý teoretický koncept je priamo naviazaný na konkrétny mechanizmus v prototypu.
- 2) **Návrh a implementácia prototypu.** Prototyp pozostáva z hardvérovej čítačky (ESP32 + RC522[29]), serverovej aplikácie v jazyku C++20 a lokálneho úložiska SQLite. Systém implementuje viacvrstvovú ochranu:
 - HMAC-SHA-256 autentifikácia hardvérových požiadaviek,
 - interný AEAD frame (ChaCha20-Poly1305[2] / XChaCha20-Poly1305[3]) s AAD väzbou hlavičky,
 - HKDF derivácia per-reader kľúčov[6] s verziovaním a domain separation,
 - tri stratégie generovania nonce (náhodný, deterministický HMAC, deterministický s runtime verifikáciou),
 - ProtocolAnomalyDetector so schopnosťou karantény zariadení,
 - sliding window anti-replay mechanizmus,
 - tamper-evident auditný log s HMAC hash chain[5] a anchor ochranou,
 - RBAC model s pepper-chránenými identifikátormi kariet.

Kódová báza je modulárna (9 modulov), písaná v štandarde C++20 s výhradným použitím knižnice libsodium pre kryptografické operácie[4].

- 3) **Verifikácia kvality kódu.** Pred experimentmi bola vykonaná statická analýza (clang-tidy[41]: 0 bezpečnostných nálezov) a dynamická analýza (ASan[42], UBSan, LSan: 0 nálezov za behu). Testovacia sada[34] obsahuje 44 testovacích prípadov pokrývajúcich kryptografiu, replay window, auditný log a rozhodovaciu logiku.
- 4) **Experimentálna analýza.** Navrhnutý a implementovaný bol C++ experimentálny harness, ktorý zdieľa produkčný kód a umožňuje reprodukovateľné testy. Šesť konfiguračných profilov (2 algoritmy $\times$ 3 nonce stratégie) bolo testovaných v šiestich scenároch (replay, AAD tampering, nonce misuse, seq rollback, tag brute-force, throughput benchmark). Každý scenár bežal 5-krát, celkový objem dát presahuje 2,8 milióna meraní.
- 5) **Analýza a vizualizácia výsledkov.** Python pipeline spracúva CSV výstupy do siedmich grafov a troch LaTeX tabuliek s chybovými pásmami a intervalmi spoľahlivosti.

B. Hlavné výsledky a závery

Experimentálna analýza potvrdila šesť hlavných tézisov:

Kľúčovým zistením je, že profil R2 (deterministický nonce s aktívnym detektorom) v kombinácii s existujúcimi vrstvami (AEAD + AAD + HKDF + anti-replay + audit) poskytuje **materiálne vyššiu odolnosť** voči všetkým testovaným triedam útokov pri prijateľnom výkonnostnom overeade. Zároveň experiment potvrdzuje, že **voľba algoritmu** (A1 vs. A2) nemá merateľný vplyv na bezpečnostné metriky — oba algoritmy sú ekvivalentné v odolnosti a porovnateľné vo výkone.

Table 22
Súhrn hlavných experimentálnych zistení

Téza	Zistenie
T1	Deterministický nonce bez runtime verifikácie (R1) zlepšuje kvalitu generovania (nezávislosť od RNG), ale neposkytuje pridanú <i>detekčnú schopnosť</i> oproti náhodnému nonce (R0). Výsledky S3a/S3b sú identické: 100% úspešnosť útoku pre R0 aj R1.
T2	<code>ProtocolAnomalyDetector</code> (profil R2) zablokoval všetky testované útoky (0% úspešnosť v S1–S5). V scenároch S3 a S4 bol jediným z testovaných mechanizmov, ktorý útok zastavil.
T3	Replay window a anomálna detekcia sú komplementárne. S4 odhaľuje štruktúrnú medzeru replay window (56 z 500 frame prejde), ktorú detektor uzatvára cez rollback detekciu.
T4	AAD binding poskytuje ochranu nezávisle od režimu nonce — S2 ukazuje 0% úspešnosť manipulácie hlavičky naprieč všetkými profilmi.
T5	Karanténa zariadení mení reakciu systému z reaktívnej (odmietnutie frame) na proaktívnu (izolácia zariadenia) — realizácia defense-in-depth[39].
T6	Overhead detektora (5–10% pokles priepustnosti, $\sim 3\text{--}5\ \mu\text{s}$ na frame) je rádovo nepodstatný pre reálnu prevádzku (<10 udalostí/s).

Všetkých 5 behov (seedy 42–46) poskytlo zhodné výsledky bezpečnostných metrík, čo naznačuje, že pozorované rozdiely vychádzajú z architektonických vlastností systému, nie z konkrétnych hodnôt nonce.

C. Splnenie cieľov práce

Table 23
Mapovanie cieľov práce na realizované výstupy

Cieľ	Definícia	Realizácia
C1	Analýza bezpečnostných hrozieb	Kapitoly 3–8: CIA triáda, STRIDE[9], RFID/NFC hrozby, kryptografické základy
C2	Návrh bezpečnostného protokolu	Kapitoly 9–10: HMAC autentifikácia, AEAD frame, anti-replay, audit
C3	Implementácia prototypu	Kapitola 21: 9 C++ modulov, ESP32 firmware, SQLite úložisko
C4	Verifikácia kvality kódu	Kapitola 22: clang-tidy, sanitizers, 44 testov
C5	Experimentálne hodnotenie	Kapitola 23: 6 profilov $\times$ 6 scenárov $\times$ 5 behov

D. Otvorené otázky a obmedzenia

Napriek pozitívnym výsledkom je potrebné identifikovať obmedzenia, ktoré ovplyvňujú interpretáciu a prenášateľnosť záverov:

- 1) UID nie je kryptografický dôkaz identity.** Karty typu MIFARE Classic[18] alebo iné karty bez vzájomnej autentifikácie umožňujú emuláciu UID. Prototyp sa spolieha na to, že bezpečnosť je zabezpečená na úrovni protokolu (HMAC + AEAD), nie na úrovni karty. Nasadenie kryptograficky silných kariet (napr. MIFARE DESFire) by eliminovalo túto závislosť.
- 2) Absencia TLS.** Komunikácia medzi čítačkou a serverom nie je šifrovaná na transportnej vrstve. V lokálnej sieti je HMAC autentifikácia dostatočná na zabránenie podvrhnutiu, no v sieťach s nedôveryhodnou infraštruktúrou by bolo nutné doplniť TLS[23] alebo mTLS.
- 3) Jednostranná autentifikácia.** Server autentifikuje čítačku (cez HMAC), ale čítačka neautentifikuje server. Man-in-the-middle útočník by teoreticky mohol zachytávať odpovede. Pre produkčné nasadenie by bolo vhodné zvážiť vzájomnú autentifikáciu[27].
- 4) Lokálna SQLite databáza.** V aktuálnom riešení je databáza uložená na rovnakom stroji ako server. Útočník s fyzickým prístupom k serveru môže modifikovať databázu aj audit anchor. Produkčné nasadenie by vyžadovalo vzdialený anchor alebo HSM[26].
- 5) Rozsah testovaných scenárov.** Šesť scenárov pokrýva hlavné triedy útokov, no nepokrýva napríklad timing útoky, side-channel analýzu, alebo útoky na firmware čítačky. Rozšírenie sady scenárov by zvýšilo dôveru v kompletnosť hodnotenia.

- 6) **Jednoprocesový server.** Aktuálna implementácia beží ako jeden proces s jedným vláknom pre spracovanie požiadaviek. Škálovanie na viacero čítačiek v reálnom nasadení by vyžadovalo asynchrónne spracovanie alebo viacprocesový model.

E. Možnosti rozšírenia

Na základe identifikovaných obmedzení a skúseností z implementácie je možné navrhnúť niekoľko smerov budúceho vývoja:

Integrácia TLS/mTLS: Doplnenie šifrovanej transportnej vrstvy s certifikátovou autentifikáciou zariadení[23], [27]. ESP32 podporuje TLS cez mbedTLS, čo umožňuje implementáciu bez zmeny hardvéru.

Podpora kryptografických kariet: Prechod z jednoduchého čítania UID na vzájomnú autentifikáciu s kartou (napr. MIFARE DESFire EV3). Tým by sa eliminovala závislosť na serverovej kompenzácii slabej identity karty.

Fuzz testing: Systematické testovanie vstupných parserov a kryptografických rozhraní pomocou AFL++ alebo libFuzzer s cieľom nájsť edge cases, ktoré jednotkové testy nepokrývajú.

Distribúovaná architektúra: Rozšírenie na viacero serverov s centralizovanou správou kľúčov a federovaným auditom. Anchor by bol uložený na nezávislom serveri alebo v blockchain-like štruktúre.

Rotácia pepperu s online migráciou: Implementácia bezprestojevej migrácie pepper verzie, kde sa existujúce karty postupne re-hashujú pri prvom použití pod novou verziou.

Adaptívna karanténa: Rozšírenie ProtocolAnomalyDetector o časovo obmedzenú karanténu s eskaláciou (napr. 1. incident: 5 minút; 2. incident: 1 hodina; 3. incident: trvalá karanténa). Aktuálna implementácia používa trvalú karanténu, čo je bezpečné, ale menej flexibilné.

Offline režim čítačky: Podpora scenárov, kde čítačka dočasne stratí spojenie so serverom. Toto vyžaduje lokálnu cache posledných autorizácií a následnú synchronizáciu, čo prináša nové bezpečnostné výzvy (stale cache, conflict resolution).

Monitoring a alerting: Integrácia s existujúcimi monitorovacími systémami (Prometheus, Grafana) pre vizualizáciu prevádzkových metrík a automatické upozornenia pri detekcii anomálií[37].

F. Záverečné zhodnotenie

Práca demonštruje, že aj v prototypovom prostredí je možné implementovať viacvrstvový bezpečnostný systém, ktorý poskytuje merateľnú ochranu voči realistickým útokovým scenárom. Kľúčovým prínosom nie je samotná implementácia jednotlivých mechanizmov — tie sú v literatúre dobre popísané — ale ich *systematická integrácia a experimentálne overenie* interakcií medzi nimi.

Výsledky jasne ukazujú, že bezpečnosť nie je binárna vlastnosť („zabezpečené vs. nezabezpečené“), ale spektrum závislé od konkrétnej kombinácie mechanizmov. Profil R2 s aktívnym anomálnym detektorom nie je „lepší“ v abstraktnom zmysle — je lepší v presne definovaných scenároch (S3, S4), kde iné mechanizmy zlyhávajú. Iné mechanizmy (replay window, AAD binding, AEAD) sú zasa dostatočné v scenároch, kde detektor nie je potrebný (S1, S2, S5).

Tento diferencovaný pohľad — nie „zapni všetko“, ale „pochop, čo chráni proti čomu“ — je hlavným záverom experimentálnej časti a podľa záverov analýzy aj najdôležitejším prínosom tejto práce pre prax návrhu bezpečných systémov kontroly prístupu.

REFERENCES

- [1] National Institute of Standards and Technology, “FIPS 198-1: The Keyed-Hash Message Authentication Code (HMAC),” NIST, 2008, available from: <https://csrc.nist.gov/pubs/fips/198-1/final>.
- [2] Y. Nir, “RFC 8439: ChaCha20 and Poly1305 for IETF Protocols,” RFC Editor, 2018, available from: <https://www.rfc-editor.org/rfc/rfc8439.html>.
- [3] A. Arciszewski, “draft-irtf-cfrg-xchacha-01: XChaCha: eXtended-nonce ChaCha and AEAD_XChaCha20_Poly1305,” IETF Datatracker (Internet-Draft), 2019, available from: <https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-xchacha-01>.
- [4] libsodium contributors, “XChaCha20-Poly1305 construction – libsodium documentation,” Online documentation, 2025, available from: https://libsodium.gitbook.io/doc/secret-key_cryptography/aead/chacha20-poly1305/xchacha20-poly1305_construction.
- [5] B. Schneier and J. Kelsey, “Secure audit logs to support computer forensics,” *ACM Transactions on Information and System Security*, vol. 2, no. 2, pp. 159–176, 1999, doi: 10.1145/317087.317089.
- [6] H. Krawczyk and P. Eronen, “RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function (HKDF),” IETF, 2010, available from: <https://datatracker.ietf.org/doc/html/rfc5869>.
- [7] D. Temoshok *et al.*, “NIST SP 800-63B-4: Digital Identity Guidelines – Authentication and Authenticator Management,” NIST, 2025, available from: <https://csrc.nist.gov/pubs/sp/800/63/b/4/final>.
- [8] K. Kent and M. Souppaya, “NIST SP 800-92: Guide to Computer Security Log Management,” NIST, 2006, available from: <https://csrc.nist.gov/pubs/sp/800/92/final>.

- [9] S. Hernan, S. Lambert, T. Ostwald, and A. Shostack, "Threat modeling — uncover security design flaws using the STRIDE approach," *MSDN Magazine*, Nov. 2006.
- [10] National Institute of Standards and Technology, "FIPS 201-3: Personal Identity Verification (PIV) of Federal Employees and Contractors," NIST, 2022, available from: <https://csrc.nist.gov/pubs/fips/201-3/final>.
- [11] R. Sandhu, D. Ferraiolo, and D. R. Kuhn, "The nist model for role-based access control," in *Proceedings of the 5th ACM Workshop on Role-Based Access Control*, 2000, pp. 47–63.
- [12] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [13] OWASP Foundation, "OWASP Top 10 API Security Risks – 2023," OWASP, 2023, available from: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>.
- [14] —, "OWASP Application Security Verification Standard (ASVS)," OWASP, 2025, available from: <https://owasp.org/www-project-application-security-verification-standard/>.
- [15] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [16] T. Karygiannis, B. Eydt, G. Barber, L. Bunn, and T. Phillips, "NIST SP 800-98: Guidelines for Securing Radio Frequency Identification (RFID) Systems," NIST, 2007, available from: <https://csrc.nist.gov/pubs/sp/800/98/final>.
- [17] International Organization for Standardization, "ISO/IEC 14443:2018 Identification cards — Contactless integrated circuit cards — Proximity cards," ISO, 2018.
- [18] F. D. Garcia, G. de Koning Gans, R. Muijers, P. van Rossum, R. Verdult, R. W. Schreur, and B. Jacobs, "Dismantling MIFARE classic," in *Proceedings of the 13th European Symposium on Research in Computer Security (ESORICS)*, ser. Lecture Notes in Computer Science, vol. 5283, 2008, pp. 97–114.
- [19] R. Verdult, F. D. Garcia, and B. Ege, "Dismantling megamos crypto: Wirelessly lockpicking a vehicle immobilizer," in *Proceedings of the 24th USENIX Security Symposium*, 2015, pp. 703–718.
- [20] H. Krawczyk, M. Bellare, and R. Canetti, "RFC 2104: HMAC: Keyed-Hashing for Message Authentication," RFC Editor, 1997, available from: <https://www.rfc-editor.org/rfc/rfc2104.html>.
- [21] D. J. Bernstein, "The salsa20 family of stream ciphers," in *New Stream Cipher Designs*, ser. Lecture Notes in Computer Science. Springer, 2008, vol. 4986, pp. 84–97.
- [22] D. McGrew, "RFC 5116: An Interface and Algorithms for Authenticated Encryption," RFC Editor, 2008, available from: <https://www.rfc-editor.org/rfc/rfc5116.html>.
- [23] E. Rescorla, "RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3," RFC Editor, 2018, available from: <https://www.rfc-editor.org/rfc/rfc8446.html>.
- [24] P. Rogaway and T. Shrimpton, "A provable-security treatment of the key-wrap problem," in *Advances in Cryptology — EUROCRYPT 2006*, ser. Lecture Notes in Computer Science, vol. 4004, 2006, pp. 373–390.
- [25] M. Bellare, A. Boldyreva, and J. O'Neill, "Deterministic and efficiently searchable encryption," in *Advances in Cryptology — CRYPTO 2007*, ser. Lecture Notes in Computer Science, vol. 4622, 2007, pp. 535–552.
- [26] E. Barker, "NIST SP 800-57 Part 1 Rev. 5: Recommendation for Key Management – Part 1: General," NIST, 2020, available from: <https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final>.
- [27] Y. Sheffer, D. C. Benjamin, M. Thomson *et al.*, "RFC 9325: Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)," RFC Editor, 2022, available from: <https://www.rfc-editor.org/rfc/rfc9325.html>.
- [28] D. R. Hipp, "SQLite: Security," SQLite Consortium, 2024, available from: <https://www.sqlite.org/security.html>.
- [29] Espressif Systems, "ESP32 Technical Reference Manual, Version 5.3," Espressif Systems, 2024, available from: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf.
- [30] F. Denis, "The Sodium cryptography library (libsodium)," Online documentation, 2024, available from: <https://doc.libsodium.org/>.
- [31] Y. Iida, "cpp-httplib: A C++ header-only HTTP/HTTPS server and client library," GitHub, 2024, available from: <https://github.com/yhirose/cpp-httplib>.
- [32] N. Lohmann, "JSON for Modern C++," GitHub, 2024, available from: <https://github.com/nlohmann/json>.
- [33] J. Beder, "yaml-cpp: A YAML parser and emitter in C++," GitHub, 2024, available from: <https://github.com/jbeder/yaml-cpp>.
- [34] Google, "GoogleTest — Google Testing and Mocking Framework," GitHub, 2024, available from: <https://github.com/google/googletest>.
- [35] Private Internet Access and NCC Group, "Libsodium Security Assessment," Private Internet Access, 2017, available from: <https://www.privateinternetaccess.com/blog/libsodium-v1-0-12-and-target-v1-0-13-security-assessment/>.
- [36] J. A. Donenfeld, "WireGuard: Next generation kernel network tunnel," in *Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS)*, 2017.
- [37] K. Scarfone, W. Jansen, and M. Tracy, "NIST SP 800-94: Guide to Intrusion Detection and Prevention Systems (IDPS)," NIST, 2007, available from: <https://csrc.nist.gov/pubs/sp/800/94/final>.
- [38] International Organization for Standardization, "ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems — Requirements," ISO, 2022.
- [39] National Security Agency, "Defense in Depth: A practical strategy for achieving Information Assurance in today's highly networked environments," NSA/IAD, 2012, available from: <https://apps.nsa.gov/iaarchive/library/ia-guidance/archive/defense-in-depth.cfm>.
- [40] Joint Task Force, "NIST SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations," NIST, 2020, available from: <https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final>.
- [41] LLVM Project, "Clang-Tidy — Extra Clang Tools Documentation," LLVM Project, 2024, available from: <https://clang.llvm.org/extra/clang-tidy/>.
- [42] K. Serebryany, D. Bruening, A. Potapenko, and D. Vyukov, "AddressSanitizer: A fast address sanity checker," in *Proceedings of the 2012 USENIX Annual Technical Conference*, 2012, pp. 309–318.
- [43] OpenSSL Software Foundation, "OpenSSL: Cryptography and SSL/TLS Toolkit," OpenSSL, 2024, available from: <https://www.openssl.org/>.
- [44] Google, "BoringSSL," Google, 2024, available from: <https://boringssl.googlesource.com/boringssl/>.